

# **Bausteine Computergestützter Datenanalyse**

Lukas Arnold      Simone Arnold      Florian Bagemihl  
Matthias Baitsch      Marc Fehr      Franca Hollmann  
Maik Poetzsch      Sebastian Seipel

2025-10-02

# Inhaltsverzeichnis

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Methodenbaustein Sensordatenanalyse</b>                          | <b>5</b>  |
| <b>Voraussetzungen</b>                                              | <b>6</b>  |
| <b>Lernziele</b>                                                    | <b>7</b>  |
| <b>1 Das Prinzip von Messungen</b>                                  | <b>8</b>  |
| 1.1 Messung . . . . .                                               | 10        |
| 1.1.1 Direkte und indirekte Messung . . . . .                       | 10        |
| 1.1.2 Genauigkeit und Präzision . . . . .                           | 12        |
| 1.2 Messreihen . . . . .                                            | 12        |
| 1.3 Varianz . . . . .                                               | 12        |
| 1.4 Standardabweichung . . . . .                                    | 13        |
| 1.4.1 Experiment Verteilungskenngrößen . . . . .                    | 14        |
| 1.5 Ergebnisse . . . . .                                            | 15        |
| 1.6 grafische Darstellung . . . . .                                 | 15        |
| 1.7 Code . . . . .                                                  | 16        |
| 1.7.1 Aufgabe Verteilungskenngrößen . . . . .                       | 18        |
| 1.7.2 Varianz und Standardabweichung mit NumPy und Pandas . . . . . | 20        |
| <b>2 Die Normalverteilung</b>                                       | <b>22</b> |
| 2.1 ein Würfel . . . . .                                            | 22        |
| 2.2 zwei Würfel . . . . .                                           | 23        |
| 2.3 drei Würfel . . . . .                                           | 25        |
| 2.4 Grafik . . . . .                                                | 30        |
| 2.5 Code . . . . .                                                  | 30        |
| 2.6 absolute Häufigkeit . . . . .                                   | 33        |
| 2.7 relative Häufigkeit . . . . .                                   | 33        |
| 2.8 Häufigkeitsdichte . . . . .                                     | 33        |
| 2.9 3 Klassen . . . . .                                             | 33        |
| 2.10 5 Klassen . . . . .                                            | 33        |
| 2.11 7 Klassen . . . . .                                            | 33        |
| 2.12 Normalverteilung anpassen . . . . .                            | 34        |
| 2.13 Das Paket SciPy . . . . .                                      | 36        |
| 2.14 Aufgaben Normalverteilung . . . . .                            | 38        |
| 2.15 Konfidenzintervalle . . . . .                                  | 39        |

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| 2.16     | Standardnormalverteilung . . . . .                          | 40        |
| 2.17     | Einzelner Messwert . . . . .                                | 41        |
| 2.18     | Stichprobe $N = 12$ . . . . .                               | 41        |
| 2.19     | Code . . . . .                                              | 42        |
| 2.20     | Die t-Verteilung . . . . .                                  | 44        |
| 2.21     | Grafik . . . . .                                            | 45        |
| 2.22     | Code . . . . .                                              | 46        |
| 2.22.1   | Beispiel Gewicht weiblicher Pinguine . . . . .              | 47        |
| 2.23     | Grafik . . . . .                                            | 48        |
| 2.24     | Code . . . . .                                              | 48        |
| 2.25     | Aufgabe Konfidenzintervalle . . . . .                       | 50        |
| <b>3</b> | <b>Lineare Parameterschätzung</b>                           | <b>52</b> |
| 3.1      | Messreihe Hooke'sches Gesetz . . . . .                      | 52        |
| 3.1.1    | Deskriptive Statistik . . . . .                             | 54        |
| 3.1.2    | Explorative Statistik . . . . .                             | 59        |
| 3.2      | Federkonstante bestimmen . . . . .                          | 62        |
| 3.2.1    | Lineare Ausgleichsrechnung . . . . .                        | 64        |
| 3.3      | Grafik . . . . .                                            | 65        |
| 3.4      | Code . . . . .                                              | 65        |
| 3.5      | partielle Ableitung nach dem y-Achsenschnittpunkt . . . . . | 67        |
| 3.6      | partielle Ableitung nach dem Anstieg . . . . .              | 68        |
| 3.6.1    | Messabweichung quantifizieren . . . . .                     | 75        |
| 3.6.2    | Das Modul SciPy . . . . .                                   | 77        |
| 3.6.3    | Ergebnis Federkonstante . . . . .                           | 78        |
| <b>4</b> | <b>Fehlerrechnung</b>                                       | <b>79</b> |
| 4.1      | Bevor es losgeht . . . . .                                  | 79        |
| 4.1.1    | Der Begriff des Fehlers . . . . .                           | 79        |
| 4.1.2    | Konfidenzniveau . . . . .                                   | 80        |
| 4.1.3    | Notation . . . . .                                          | 80        |
| 4.2      | Fehlerarten . . . . .                                       | 80        |
| 4.3      | Grundbegriffe nach DIN 1319 . . . . .                       | 82        |
| 4.3.1    | Messabweichung . . . . .                                    | 83        |
| 4.3.2    | Messunsicherheit . . . . .                                  | 83        |
| 4.3.3    | Vollständiges Messergebnis . . . . .                        | 85        |
| 4.3.4    | Übersicht . . . . .                                         | 86        |
| 4.4      | Absolute und relative Fehler . . . . .                      | 87        |
| 4.5      | Größtfehler . . . . .                                       | 88        |
| 4.6      | Signifikante Stellen . . . . .                              | 89        |
| 4.7      | Methoden der Fehlerrechnung . . . . .                       | 90        |
| 4.7.1    | 1. Fehlerabschätzung . . . . .                              | 91        |
| 4.7.2    | 2. Fehlerstatistik . . . . .                                | 97        |

|          |                                                            |            |
|----------|------------------------------------------------------------|------------|
| 4.7.3    | 3. Fehlerfortpflanzung . . . . .                           | 100        |
| 4.8      | Vollständiges Messergebnis . . . . .                       | 108        |
| 4.9      | 95-%-Vertrauensintervall . . . . .                         | 108        |
| 4.10     | Lineare Fehlerfortpflanzung . . . . .                      | 111        |
| 4.11     | Gauß'sche Fehlerfortpflanzung . . . . .                    | 112        |
| <b>5</b> | <b>Übung Hooke'sches Gesetz</b>                            | <b>114</b> |
| 5.1      | Dateien einlesen . . . . .                                 | 114        |
| 5.2      | Aufgabe Dateien einlesen . . . . .                         | 116        |
| 5.3      | Musterlösung Daten aufbereiten . . . . .                   | 123        |
| 5.3.1    | Auf Ausreißer prüfen . . . . .                             | 124        |
| 5.4      | Ausreißer bestimmen . . . . .                              | 124        |
| 5.5      | Gemeinsame Ausgabe der Messreihen . . . . .                | 124        |
| 5.6      | Code . . . . .                                             | 126        |
| 5.6.1    | Umwandlung der Rechengrößen . . . . .                      | 127        |
| 5.6.2    | Grafische Darstellung . . . . .                            | 130        |
| 5.7      | Grafische Darstellung . . . . .                            | 132        |
| 5.8      | Mittelwerte und Standardfehler berechnen . . . . .         | 133        |
| 5.9      | Code . . . . .                                             | 133        |
| 5.10     | Auswertung . . . . .                                       | 134        |
| 5.11     | Musterlösung Federkonstanten bestimmen . . . . .           | 135        |
| 5.12     | $\alpha = 0.10$ . . . . .                                  | 135        |
| 5.13     | $\alpha = 0.05$ . . . . .                                  | 136        |
| 5.14     | Code . . . . .                                             | 136        |
| 5.15     | Auswertung . . . . .                                       | 137        |
| <b>6</b> | <b>Kalibrierung</b>                                        | <b>139</b> |
| 6.1      | Beispiel Pt100 . . . . .                                   | 139        |
| 6.1.1    | Aufgabe Pt100-Tabelle einlesen . . . . .                   | 140        |
| 6.1.2    | Nicht-Linearität darstellen . . . . .                      | 147        |
| 6.1.3    | Nicht-Linearität quantifizieren . . . . .                  | 151        |
| 6.2      | Kalibrieren und Justieren . . . . .                        | 154        |
| 6.3      | Kalibriermethoden . . . . .                                | 155        |
| 6.3.1    | Korrektionstabelle . . . . .                               | 156        |
| 6.3.2    | Ermittlung von Kalibrierfaktoren . . . . .                 | 156        |
| 6.3.3    | Ermittlung einer (empirischen) Kalibrierfunktion . . . . . | 156        |
| 6.4      | Resterampe . . . . .                                       | 156        |



# Methodenbaustein Sensordatenanalyse



Bausteine Computergestützter Datenanalyse von Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch und Sebastian Seipel. Methodenbaustein Sensordatenanalyse von Maik Poetzsch ist lizenziert unter [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). Das Werk ist abrufbar auf [GitHub](https://github.com/bausteine-der-datenanalyse). Ausgenommen von der Lizenz sind alle Logos Dritter und anders gekennzeichneten Inhalte. 2025

## Zitiervorschlag

Arnold, Lukas, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch, und Sebastian Seipel. 2025. „Bausteine Computergestützter Datenanalyse. Methodenbaustein Sensordatenanalyse. <https://github.com/bausteine-der-datenanalyse/m-sensordatenanalyse>.

## BibTeX-Vorlage

```
@misc{BCD-m-sensordatenanalyse-2025,  
  title={Bausteine Computergestützter Datenanalyse. Methodenbaustein Sensordatenanalyse},  
  author={Arnold, Lukas and Arnold, Simone and Bagemihl, Florian and Baitsch, Matthias and Fehr, Marc and Hollmann, Franca and Poetzsch, Maik and Seipel, Sebastian},  
  year={2025},  
  url={https://github.com/bausteine-der-datenanalyse/m-sensordatenanalyse}}
```

# Voraussetzungen

Die Bearbeitungszeit dieses Bausteins beträgt circa **Platzhalter**. Für die Bearbeitung dieses Bausteins werden folgende Bausteine vorausgesetzt:

- Werkzeugbaustein Python
- Werkzeugbaustein NumPy
- Werkzeugbaustein Pandas
- Werkzeugbaustein matplotlib

In diesem Baustein werden die folgenden Module und Pakete verwendet:

- numpy
  - numpy.polynomial
- pandas
- matplotlib
- glob
- scipy

Im Baustein werden folgende Daten verwendet:

- Zahnwachstum bei Meerschweinchen [CSV-Datei](#)
- Vermessung von Pinguinen an der Palmer Station [GitHub](#)
- Elektrische Widerstandswerte eines Pt100-Thermometers aus der DIN 60751 (Deutsches Institut für Normung [2023](#)) und von [hier](#) ([PDF](#))
- Abstandsmessungen mit einem Ultraschallsensor (Messungen an der FH Dortmund)

Querverweis auf:

- Methodenbaustein Datenfitting und Datenoptimierung

# Lernziele

In diesen Baustein lernen Sie ...

- Statistische Grundbegriffe
- Werkzeuge zur Auswertung und grafischen Darstellung von Sensordaten kennen
- Methoden zur Verarbeitung von mehreren Dateien anzuwenden
- Fehlerrechnung und -analyse
- Sensorkennlinien
- Kennlinienfehler und deren Korrektur

# 1 Das Prinzip von Messungen

“In der Physik existiert nur das, was gemessen worden ist” (Merz 1968, 14).  
Merz, Ludwig. 1968. “Grundkurs der Messtechnik. Teil I: Das Messen elektrischer Größen.” 2. Auflage. München; Wien. R. Oldenbourg Verlag.

In diesem Baustein werden die folgenden Module verwendet:

```
import numpy as np
import numpy.polynomial.polynomial as poly
import pandas as pd
import matplotlib.pyplot as plt
import scipy
import glob
```

Physikalische Größen werden mit der Hilfe von Messgeräten bestimmt. Diese ordnen der tatsächlichen Merkmalsausprägung eine numerische Entsprechung relativ zu einem Bezugssystem zu.

Ein Beispiel: “Johanna ist am Messbrett 173 Zentimeter groß.”

- Die tatsächliche Merkmalsausprägung ist Johannas Größe.
- Das Messgerät ist das Messbrett.
- Die numerische Entsprechung ist 173.
- Das Bezugssystem ist das metrische System.

Messwerte können aus verschiedenen Gründen immer nur eine Annäherungen an den wahren Wert der zugrundeliegenden physikalischen Größe sein. Zum einen variiert die [Größe eines Menschen im Tagesverlauf](#). Zum anderen ist das Messergebnis auch ein Ergebnis der verwendeten Skala. Wäre die Messung im imperialen Messsystem erfolgt, wäre Johannas Größe mit 68 Zoll bestimmt worden, was 172,72 Zentimetern entspricht.

Das Messergebnis ist also keine exakte Entsprechung der tatsächlichen Merkmalsausprägung. Ein bekanntes Beispiel für die mit dem Messvorgang verbundene Unsicherheit ist das Küstenlinienparadox: Das Ergebnis der Vermessung unregelmäßiger Küstenlinien wird umso größer, je kleiner die Messabschnitte gewählt werden.



- (a) Gerade Messabschnitte von 200 km Länge, Gesamtlänge ungefähr 2350 km  
 (b) Gerade Messabschnitte von 100 km Länge, Gesamtlänge ungefähr 2775 km  
 (c) Gerade Messabschnitte von 50 km Länge, Gesamtlänge ungefähr 3425 km

Abbildung 1.1: Küstenlinienparadox

Britain-fractal-coastline-200km, Britain-fractal-coastline-100km und Britain-fractal-coastline-50km von Maksim stehen unter der Lizenz [CC BY-SA 3.0](#) und sind abrufbar auf Wikipedia ([200km](#), [100km](#), [50km](#)). 2006

## 1.1 Messung

### ! Wichtig 1: Messung

“Eine Messung ist der experimentelle Vorgang, durch den ein spezieller Wert einer physikalischen Größe als Vielfaches einer Einheit oder eines Bezugswertes ermittelt wird.

Die Messung ergibt zunächst einen Messwert. Dieser stimmt aber aufgrund störender Einflüsse mit dem wahren Wert der Messgröße praktisch nie überein, sondern weist eine gewisse Messabweichung auf. Zum *vollständigen Messergebnis* wird der Messwert, wenn er mit quantitativen Aussagen über die zu erwartende Größe der Messabweichung ergänzt wird. Dies wird in der Messtechnik als Teil der Messaufgabe und damit der Messung verstanden.”

Messung. von verschiedenen [Autor:innen](#) steht unter der Lizenz [CC BY-SA 4.0](#) ist abrufbar auf [Wikipedia](#). 2025

- Die ideale Messung ist eine direkte Messung oder der gesuchte Wert hängt linear vom gemessenen Wert ab.
- Die ideale Messung ist *genau* und *präzise*.

### 1.1.1 Direkte und indirekte Messung

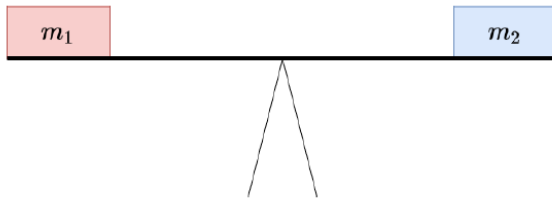
Bei einer direkten Messung wird die Messgröße durch den unmittelbaren Vergleich mit einem Normal oder einem genormten Bezugssystem gewonnen.

Gliedermaßstäbe von Fst76 ist lizenziert unter [CC-BY-SA 3.0](#) und ist abrufbar auf [Wikimedia](#). 2014

Bei einer indirekten Messung wird die Messgröße auf eine andere physikalische Größe zurückgeführt.

Spring scale von Amada44 steht unter der Lizenz [CC-BY-SA-3.0 unported](#) und ist abrufbar auf [Wikimedia](#). 2016

Observe the Moon wurde von der NASA veröffentlicht und ist abrufbar unter [nasa.gov](#). 2010



(a) Balkenwaage

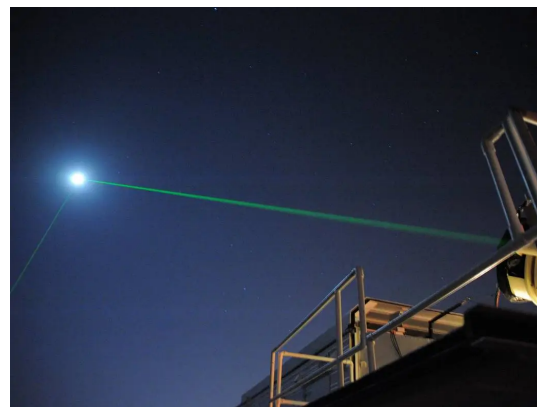


(b) Zollstock

Abbildung 1.2: Direkte Messung



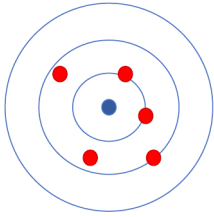
(a) Federwaage



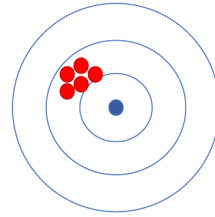
(b) Laserentfernungsmessung

Abbildung 1.3: Indirekte Messung

### 1.1.2 Genauigkeit und Präzision



(a) Genauigkeit



(b) Präzision

Die Genauigkeit einer Messung ist ein Maß für die Abweichung der Messwerte vom realen Wert. Die Genauigkeit ist nur bestimmbar, wenn anerkannte Referenzwerte vorhanden sind. Die Präzision einer Messung beschreibt, wie gut die einzelnen Messwerte miteinander übereinstimmen. Die Präzision einer Messung wird über die Standardabweichung der Stichprobe bestimmt.

Abbildung 1.4: Genauigkeit und Präzision

## 1.2 Messreihen

Um die Unsicherheit einer Messung zu verringern, kann man einen Messwert in Form einer Messreihe wiederholt aufnehmen. Die beste Schätzung der Messgröße bietet der arithmetische Mittelwert der Messreihe.

Der arithmetische Mittelwert einer Messreihe  $\bar{x}$  ist die Summe aller Einzelmesswerte  $x_i$  dividiert durch die Anzahl der Messwerte  $N$ .

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

Mit Hilfe des arithmetischen Mittelwerts kann eine Aussage über die Streuung der Messwerte und die Präzision der Messung getroffen werden. Dazu werden die Varianz und die Standardabweichung der Messreihe berechnet.

## 1.3 Varianz

Die Varianz ist der Mittelwert der quadrierten Abweichungen vom Mittelwert.



$$\text{Var}(x_i) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

## 1.4 Standardabweichung

Die Quadratwurzel der Varianz wird als Standardabweichung bezeichnet. Diese hat den Vorteil, dass sie in der Einheit der Messwerte vorliegt und dadurch leichter zu interpretieren ist. Die Standardabweichung  $s$  wird so berechnet:

$$s_N = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Für Stichproben wird die Stichprobenvarianz verwendet. Für die Standardabweichung einer Stichprobe gilt:

$$s_{N-1} = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Da die Varianz das Quadrat der Standardabweichung  $s$  ist, wird diese häufig mit  $s^2$  gekennzeichnet.

### **Warning 1: Standardabweichung und Varianz in der Grundgesamtheit**

In der Stochastik werden Formeln häufig auch mit griechischen Buchstaben geschrieben, wenn Sie sich statt auf eine Stichprobe auf die Grundgesamtheit beziehen. Der Mittelwert in der Grundgesamtheit wird auch Erwartungswert genannt und mit dem griechischen Buchstaben  $\mu$  (My) dargestellt. Die Standardabweichung des Erwartungswerts wird mit  $\sigma$  (Sigma) gekennzeichnet.

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

Mit Hilfe der Standardabweichung kann der *Standardfehler des Mittelwerts* bestimmt werden. Der Standardfehler des Mittelwerts ist ein Maß dafür, wie genau sich der arithmetische Mittelwert der Stichprobe an den tatsächlichen Mittelwert der Grundgesamtheit, den Erwartungswert, annähert (dazu gleich mehr) und wird auch *Stichprobenfehler* genannt. Der Standardfehler des Mittelwerts wird aus der Standardabweichung einer Messung und der Wurzel der Stichprobengröße berechnet.

$$\sigma_{\bar{x}} = \frac{s}{\sqrt{N}}$$

Da die Standardabweichung in der Grundgesamtheit in der Regel unbekannt ist, wird der Standardfehler des Mittelwerts mit der Stichprobenstandardabweichung geschätzt.

$$\sigma_{\bar{x}} = \frac{s_{n-1}}{\sqrt{N}}$$

Manchmal wird der Standardfehler zur längeren Schreibweise umgeformt.

$$\sigma_{\bar{x}} = \frac{s_{n-1}}{\sqrt{N}} = \sqrt{\frac{1}{N(N-1)} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Der Standardfehler wird umso kleiner (die Messung umso präziser), je kleiner die Varianz in der Grundgesamtheit und je größer der Stichprobenumfang ist.

Dies lässt sich mit einem simulierten Würfelexperiment verdeutlichen. Bei einem idealen, fairen Würfel kommt jede Augenzahl gleich oft vor. Der Erwartungswert eines sechseitigen Würfels ist:

$$\frac{1}{6} \sum_{i=1}^{i=6} (x_i) = 3,5$$

Die Standardabweichung eines fairen, sechseitigen Würfels beträgt:

$$\sqrt{\frac{1}{6} \sum_{i=1}^{i=6} (x_i - 3,5)^2} \approx 1,71$$

Da die Varianz in der Grundgesamtheit bekannt ist, hängt der Standardfehler des Mittelwerts eines fairen Würfels allein von der Stichprobengröße ab.

### 1.4.1 Experiment Verteilungskenngrößen

In einem simulierten Experiment würfeln 100 Personen jeweils 3, 10 und 50 Mal und bilden den Mittelwert der Augen. Weil ein fairer Würfel simuliert wird, kann der Standardfehler mit der Standardabweichung der Grundgesamtheit berechnet werden.

## 1.5 Ergebnisse

Würfe pro Person: 3  
kleinster Mittelwert: 1.00  
Stichprobenmittelwert: 3.57

Stichprobengröße: 300  
größter Mittelwert: 6.00  
Standardfehler: 0.10

Würfe pro Person: 10  
kleinster Mittelwert: 1.70  
Stichprobenmittelwert: 3.49

Stichprobengröße: 1000  
größter Mittelwert: 4.60  
Standardfehler: 0.05

Würfe pro Person: 50  
kleinster Mittelwert: 2.98  
Stichprobenmittelwert: 3.50

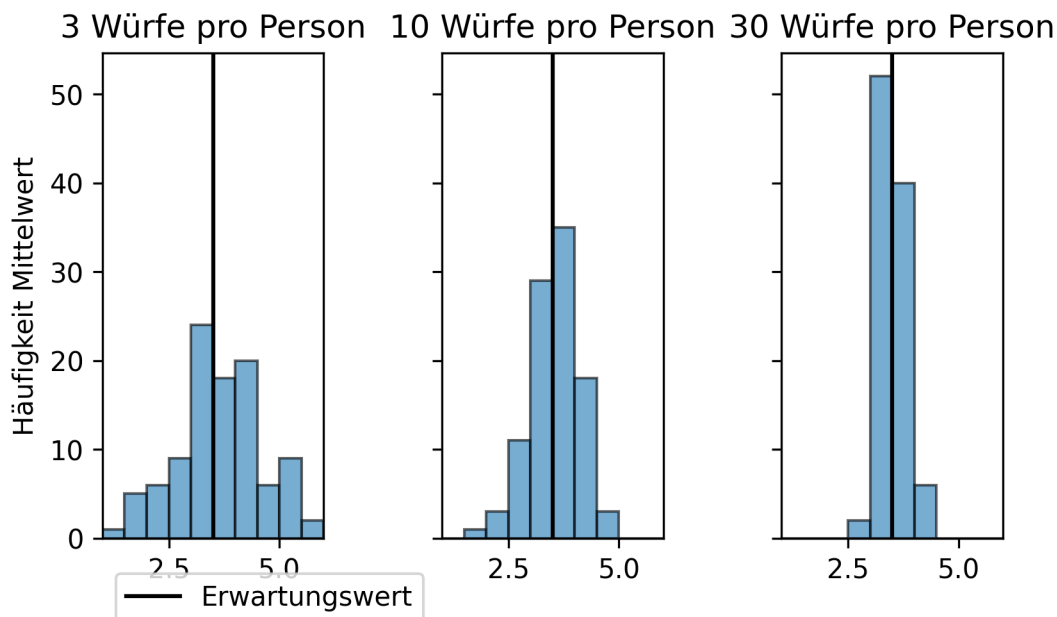
Stichprobengröße: 5000  
größter Mittelwert: 4.36  
Standardfehler: 0.02

Mit zunehmender Anzahl an Würfeln nähern sich Minimum und Maximum der individuellen Durchschnittswerte sowie der Stichprobenmittelwert dem Erwartungswert an.

*Hinweis: Da das Skript dynamisch generiert wird, wurden die Zufallszahlen mit einem festgelegten Startwert erzeugt.*

## 1.6 grafische Darstellung

Die Häufigkeit der individuellen Mittelwerte ist in den folgenden Histogrammen dargestellt.



## 1.7 Code

Berechnung

```
personen = 100
standardabweichung_grundgesamtheit = np.arange(1, 7).std(ddof = 0)
seed = 1

# 3 Würfe
würfe = 3

## Personen stehen in den Zeilen (axis = 0), Würfe in den Spalten (axis = 1)
augen3 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size=(personen, würfe))

## zeilenweise Mittelwert bilden mit np.array.mean(axis = 1)
print(f"Würfe pro Person: {würfe}\t\t\t\t",
      f"Stichprobengröße: {würfe * personen}\n",
      f"kleinster Mittelwert: {augen3.mean(axis = 1).min():.2f}\t\t\t",
      f"größter Mittelwert: {augen3.mean(axis = 1).max():.2f}\n",
      f"Stichprobenmittelwert: {augen3.mean():.2f}\t\t\t",
      f"Standardfehler: {standardabweichung_grundgesamtheit / ( augen3.size ** (1/2) ):.2f}\n",
      sep = "")

# 10 Würfe
würfe = 10

## Personen stehen in den Zeilen (axis = 0), Würfe in den Spalten (axis = 1)
augen10 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size=(personen, würfe))

## zeilenweise Mittelwert bilden mit np.array.mean(axis = 1)
print(f"Würfe pro Person: {würfe}\t\t\t\t",
      f"Stichprobengröße: {würfe * personen}\n",
      f"kleinster Mittelwert: {augen10.mean(axis = 1).min():.2f}\t\t\t",
      f"größter Mittelwert: {augen10.mean(axis = 1).max():.2f}\n",
      f"Stichprobenmittelwert: {augen10.mean():.2f}\t\t\t",
      f"Standardfehler: {standardabweichung_grundgesamtheit / ( augen10.size ** (1/2) ):.2f}\n",
      sep = "")

# 50 Würfe
würfe = 50

## Personen stehen in den Zeilen (axis = 1), Würfe in den Spalten (axis = 1)
```

```

augen50 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = 50)

## zeilenweise Mittelwert bilden mit np.array.mean(axis = 1)
print(f"Würfe pro Person: {würfe}\t\t\t",
      f"Stichprobengröße: {würfe * personen}\n",
      f"kleinster Mittelwert: {augen50.mean(axis = 1).min():.2f}\t\t",
      f"größter Mittelwert: {augen50.mean(axis = 1).max():.2f}\n",
      f"Stichprobenmittelwert: {augen50.mean():.2f}\t\t",
      f"Standardfehler: {standardabweichung_grundgesamtheit / ( augen50.size ** (1/2) ):.2f}\n",
      sep = "")

```

Darstellung

```

personen = 100
standardabweichung_grundgesamtheit = np.arange(1, 7).std(ddof = 0)
seed = 1

# 3 Würfe
würfe = 3
augen3 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = 3 * 100)

# 10 Würfe
würfe = 10
augen10 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = 10 * 100)

# 50 Würfe
würfe = 50
augen50 = np.random.default_rng(seed = seed).integers(low = 1, high = 6, endpoint = True, size = 50 * 100)

# plotten
bins = 10

# 3 Würfe
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, sharey = True)

ax1.hist(augen3.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax1.set_xlim(1, 6)
ax1.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black', label = 'Erwartungswert')
ax1.set_ylabel('mittleres Würfelergebnis')
ax1.set_ylabel('Häufigkeit Mittelwert')
ax1.set_title("3 Würfe pro Person")
ax1.legend(loc = 'lower left', bbox_to_anchor = (0, -0.2))

```

```
# 10 Würfe
ax2.hist(augen10.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax2.set_xlim(1, 6)
ax2.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black')
ax2.set_ylabel('mittleres Würfelergebnis')
ax2.set_title("10 Würfe pro Person")

# 30 Würfe
ax3.hist(augen50.mean(axis = 1), bins = bins, alpha = 0.6, edgecolor = 'black', range = (1, 6))
ax3.set_xlim(1, 6)
ax3.axvline(x = 3.5, ymin = 0, ymax = 1, color = 'black')
ax3.set_ylabel('mittleres Würfelergebnis')
ax3.set_title("30 Würfe pro Person")

plt.tight_layout()
plt.show()
```

### 1.7.1 Aufgabe Verteilungskenngrößen

Im Datensatz `ToothGrowth.csv` ist eine Messreihe zur Länge zahnbildender Zellen bei Meerschweinchen gespeichert. Die Tiere erhielten Vitamin C direkt (VC) oder in Form von Orangensaft (OJ) in unterschiedlichen Dosen.

---

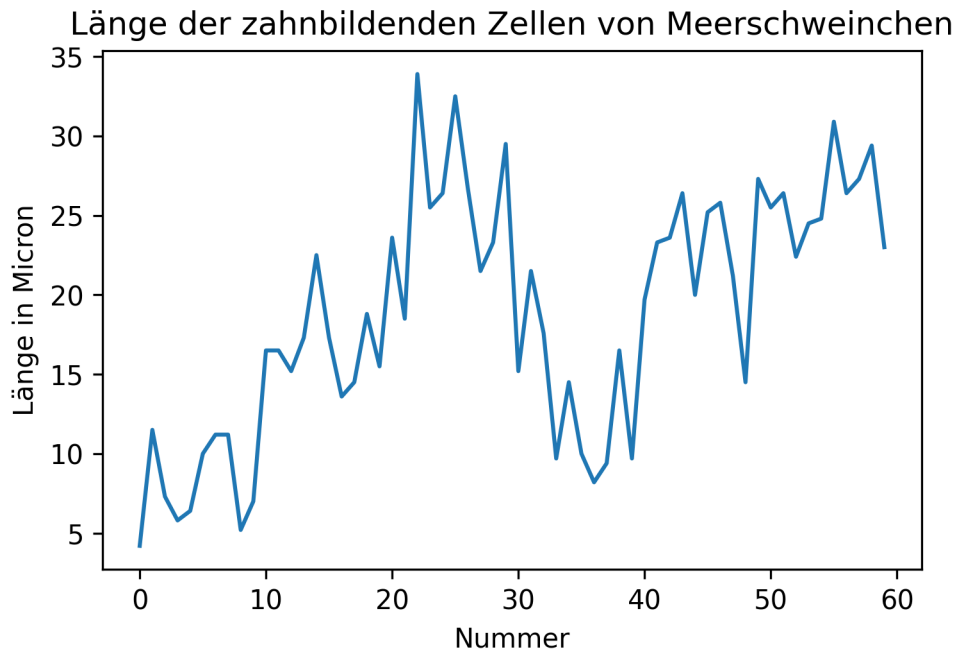
#### Code-Block 1.1

```
dateipfad = "01-daten/ToothGrowth.csv"
meerschweinchen = pd.read_csv(filepath_or_buffer = dateipfad, sep = ',', header = 0, \
    names = ['ID', 'len', 'supp', 'dose'], dtype = {'ID': 'int', 'len': 'float', 'dose': 'float'})
```

---

Crampton, E. W. 1947. „THE GROWTH OF THE ODONTOBLASTS OF THE INCISOR TOOTH AS A CRITERION OF THE VITAMIN C INTAKE OF THE GUINEA PIG“. The Journal of Nutrition 33 (5): 491–504. <https://doi.org/10.1093/jn/33.5.491>

Der Datensatz kann in R mit dem Befehl “`ToothGrowth`” aufgerufen werden.



**Berechnen Sie den arithmetischen Mittelwert, die Varianz, die Standardabweichung und den Stichprobenfehler der Messreihe zur Zahnlänge (len). Verwenden Sie dazu die vorgestellten Formeln.**

Das Ergebnis könnte so aussehen:

N: 60

arithmetisches Mittel: 18.81

Stichprobenfehler: 0.99

Stichprobenvarianz: 58.51

Stichprobenstandardabweichung: 7.65

### 💡 Tipp 1: Musterlösung Verteilungskenngrößen

```
def verteilungskennwerte(x, output = True):

    # Anzahl Messwerte bestimmen
    N = len(x)

    # arithmetisches Mittel bestimmen
    stichprobenmittelwert = sum(x) / N

    # Stichprobenvarianz bestimmen
    stichprobenvarianz = sum((x - stichprobenmittelwert) ** 2) / (N - 1)

    # Standardabweichung bestimmen
    standardabweichung = stichprobenvarianz ** (1/2)

    # Stichprobenfehler bestimmen
    stichprobenfehler = standardabweichung / (N ** (1/2))

    # Ausgabe
    if output: # output = True
        print(f"N: {N}\n",
              f"arithmetisches Mittel: {stichprobenmittelwert:.2f}\n",
              f"Stichprobenfehler: {stichprobenfehler:.2f}\n",
              f"Stichprobenvarianz: {stichprobenvarianz:.2f}\n",
              f"Standardabweichung: {standardabweichung:.2f}",
              sep = '')

    else: # output = False
        return N, stichprobenmittelwert, stichprobenfehler, stichprobenvarianz, standardabweichung

verteilungskennwerte(meerschweinchen['len'])
```

## 1.7.2 Varianz und Standardabweichung mit NumPy und Pandas

Die Module NumPy und Pandas verfügen über eigene Funktionen zur Berechnung der Varianz und der Standardabweichung. Die Varianz und Standardabweichung werden mit den Funktionen `np.var()` und `np.std()` bzw. den Methoden `pd.var()` und `pd.std()` berechnet. Der Parameter `ddof` (delta degrees of freedom) steuert, welcher Nenner zur Berechnung der Varianz verwendet wird in der Form  $N - \text{ddof}$ . Während der Standardwert in NumPy `ddof=0` ist, berechnet Pandas mit dem Standardwert `ddof=1` die Stichprobenvarianz.



```
print("Varianz:")
print(f"NumPy:\t{np.var(meerschweinchen['len']):.2f}")
print(f"Pandas:\t{meerschweinchen['len'].var():.2f}")

print("\nStandardabweichung:")
print(f"NumPy:\t{np.std(meerschweinchen['len']):.2f}")
print(f"Pandas:\t{meerschweinchen['len'].std():.2f}")
```

Varianz:

NumPy: 57.54

Pandas: 58.51

Standardabweichung:

NumPy: 7.59

Pandas: 7.65

## 2 Die Normalverteilung

Mit zunehmender Stichprobengröße wird eine immer bessere Schätzung des Erwartungswerts erreicht. Mathematisch liegt dieser Beobachtung der [zentrale Grenzwertsatz](#) zugrunde. So werden beim Würfeln mit mehreren Würfeln weit vom Erwartungswert entfernte Wurfresultate immer unwahrscheinlicher. Dies lässt sich bereits mit wenigen Würfeln zeigen (siehe Beispiel).

### **i** Hinweis 1: Häufigkeitsverteilung von Würfelresultaten

Für einen Würfel gibt es 6 mögliche Resultate, für 2 Würfel  $6 * 6$  mögliche Kombinationen, für 3 Würfel  $6 * 6 * 6$  Kombinationen und so weiter. Weil viele Kombinationen wertgleich sind, kommen Wurfresultate in der Nähe des Erwartungswerts häufiger vor als beispielsweise ein Einserpasch.

### 2.1 ein Würfel

```
ein_würfel = []

for i in range(1, 7):
    ein_würfel.append(i)

ein_würfel = pd.Series(ein_würfel)

print("Häufigkeitsverteilung der Augensumme:")
print(ein_würfel.value_counts(), "\n")
print(f"Durchschnitt: {ein_würfel.mean():.1f}")

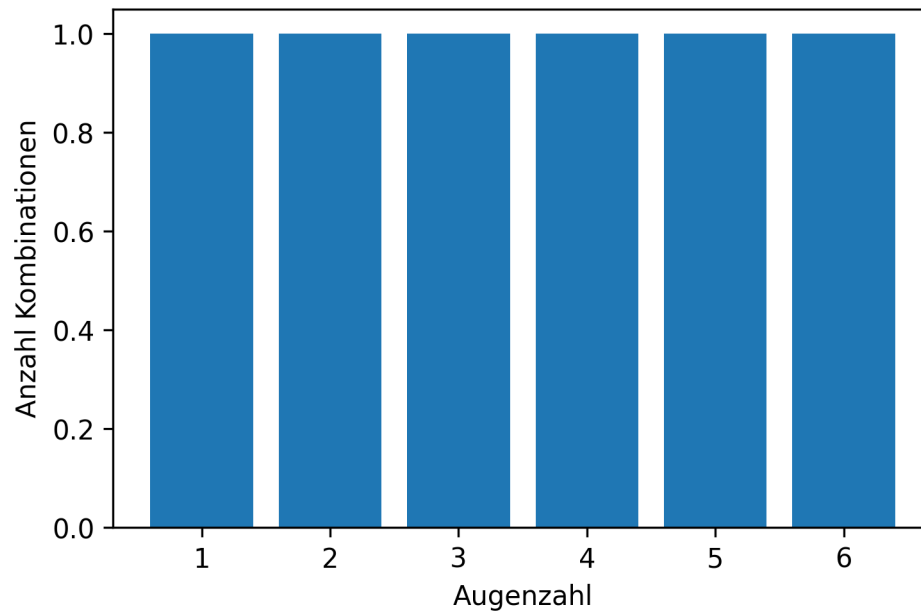
plt.bar(ein_würfel.unique(), ein_würfel.value_counts())
plt.xlabel('Augenzahl')
plt.ylabel('Anzahl Kombinationen')
plt.show()
```

Häufigkeitsverteilung der Augensumme:

|   |   |
|---|---|
| 1 | 1 |
| 2 | 1 |

```
3    1
4    1
5    1
6    1
Name: count, dtype: int64
```

Durchschnitt: 3.5



## 2.2 zwei Würfel

```

zwei_würfel = []

for i in range(1, 7):
    würfel_1 = i

    for j in range (1, 7):
        würfel_2 = j
        zwei_würfel.append(würfel_1 + würfel_2)

zwei_würfel = pd.Series(zwei_würfel)

print("Häufigkeitsverteilung der Augensumme:")
print(zwei_würfel.value_counts().sort_index(ascending = True), "\n")
print(f"Durchschnitt: {zwei_würfel.mean():.1f}")
print(f"Durchschnitt pro Würfel: {zwei_würfel.mean() / 2:.1f}")

plt.bar(zwei_würfel.unique(), zwei_würfel.value_counts().sort_index(ascending = True))
plt.xlabel('Augenzahl')
plt.ylabel('Anzahl Kombinationen')
plt.grid()
plt.show()

```

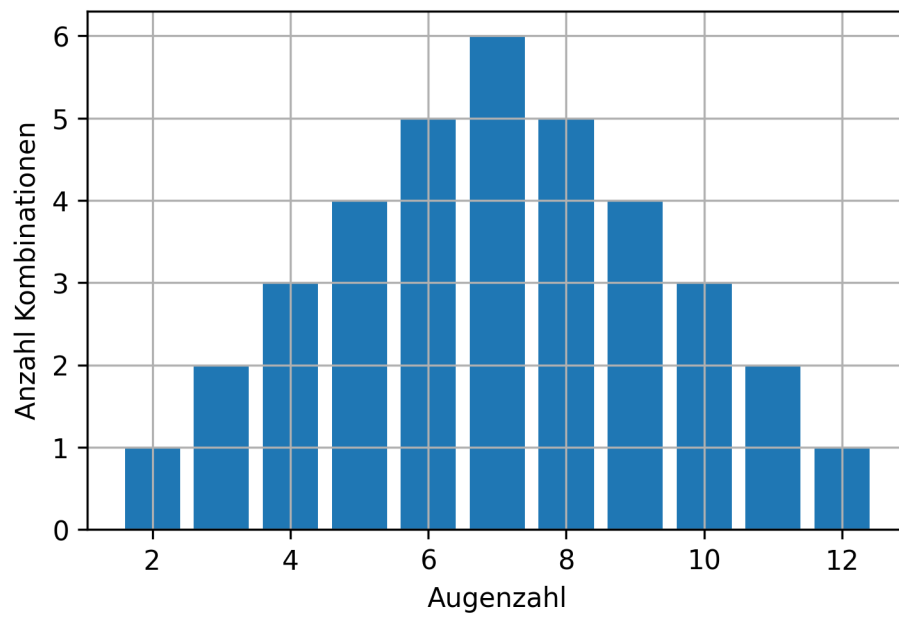
Häufigkeitsverteilung der Augensumme:

|    |   |
|----|---|
| 2  | 1 |
| 3  | 2 |
| 4  | 3 |
| 5  | 4 |
| 6  | 5 |
| 7  | 6 |
| 8  | 5 |
| 9  | 4 |
| 10 | 3 |
| 11 | 2 |
| 12 | 1 |

Name: count, dtype: int64

Durchschnitt: 7.0

Durchschnitt pro Würfel: 3.5



## 2.3 drei Würfel

```

dreiwürfel = []

for i in range(1, 7):
    würfel_1 = i

    for j in range (1, 7):
        würfel_2 = j

        for k in range (1, 7):
            würfel_3 = k
            dreiwürfel.append(würfel_1 + würfel_2 + würfel_3)

dreiwürfel = pd.Series(dreiwürfel)

print("Häufigkeitsverteilung der Augensumme:")
print(dreiwürfel.value_counts().sort_index(ascending = True), "\n")
print(f"Durchschnitt: {dreiwürfel.mean():.1f}")
print(f"Durchschnitt pro Würfel: {dreiwürfel.mean() / 3:.1f}")

plt.bar(dreiwürfel.unique(), dreiwürfel.value_counts().sort_index(ascending = True))
plt.xlabel('Augenzahl')
plt.ylabel ('Anzahl Kombinationen')
plt.grid()
plt.show()

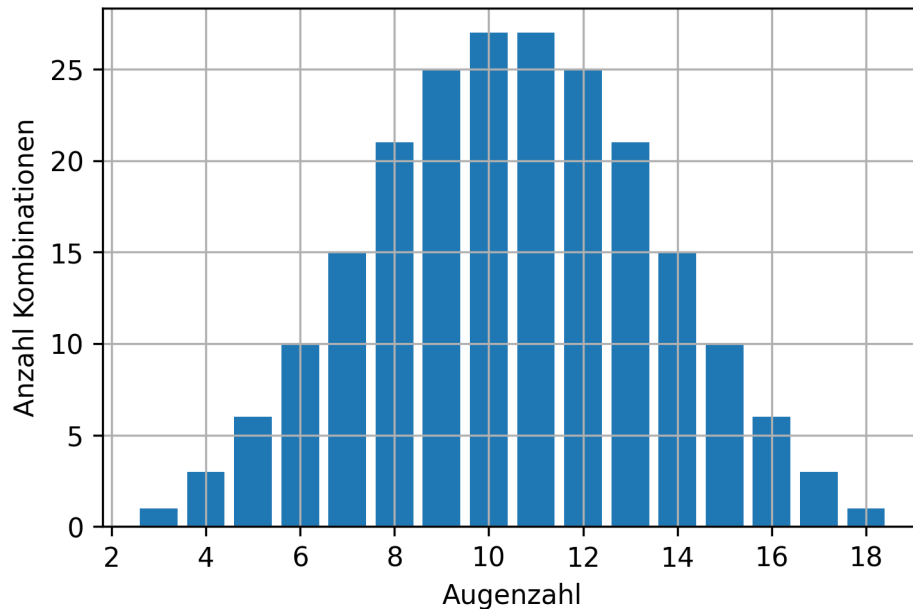
```

Häufigkeitsverteilung der Augensumme:

|    |    |
|----|----|
| 3  | 1  |
| 4  | 3  |
| 5  | 6  |
| 6  | 10 |
| 7  | 15 |
| 8  | 21 |
| 9  | 25 |
| 10 | 27 |
| 11 | 27 |
| 12 | 25 |
| 13 | 21 |
| 14 | 15 |
| 15 | 10 |
| 16 | 6  |
| 17 | 3  |

```
18      1
Name: count, dtype: int64

Durchschnitt: 10.5
Durchschnitt pro Würfel: 3.5
```



Die mit steigender Stichprobengröße zu beobachtende Annäherung von Messwerten an einen in der Grundgesamtheit geltenden Erwartungswert gilt auch, wenn der Erwartungswert und die Varianz in der Grundgesamtheit unbekannt sind. Mit zunehmender Stichprobengröße nähern sich die Messwerte der [Normalverteilung](#) an, die nach ihrem Entdecker Carl Friedrich Gauß auch als Gaußsche Glockenkurve bekannt ist.

Die für größere Stichproben zu beobachtende Annäherung der Verteilung von Messwerten an die Normalverteilung kann anhand des Gewichts von Pinguinen aus dem Datensatz `palmerpenguins` gezeigt werden.

**palmerpenguins**

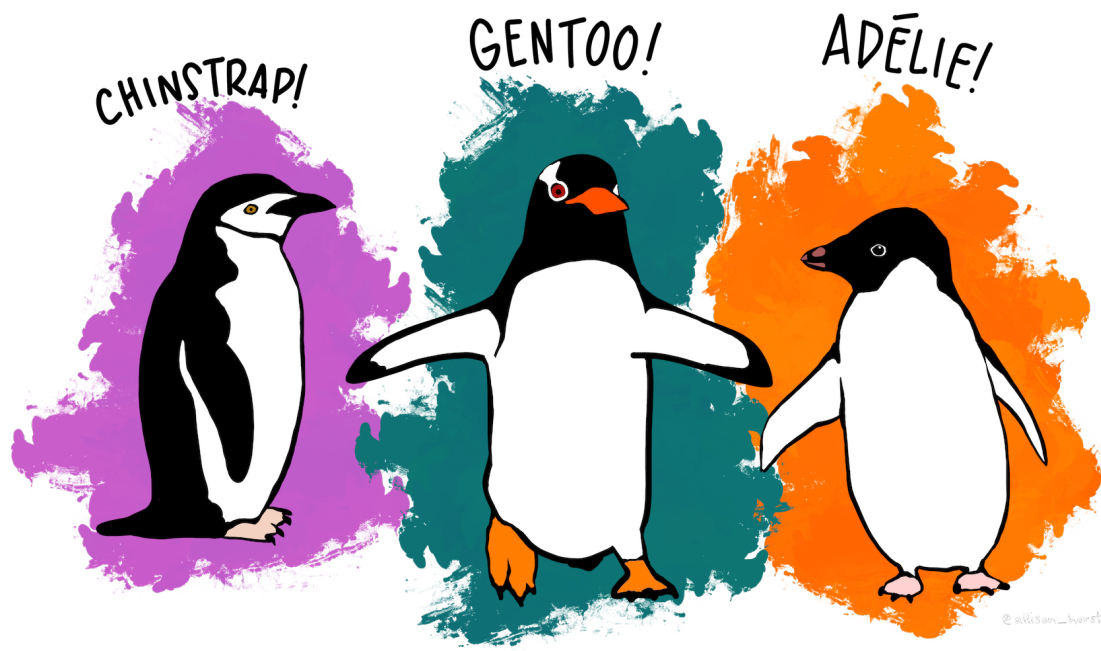


Abbildung 2.1: Pinguine des Palmer-Station-Datensatzes

Meet the Palmer penguins von @allison\_horst steht unter der Lizenz [CC0-1.0](#) und ist auf [GitHub](#) abrufbar. 2020

Der Datensatz steht unter der Lizenz [CCO](#) und ist in R sowie auf [GitHub](#) verfügbar. 2020

```
# R Befehle, um den Datensatz zu laden
install.packages("palmerpenguins")
library(palmerpenguins)
```

Horst AM, Hill AP und Gorman KB. 2020. palmerpenguins: Palmer Archipelago (Antarctica) penguin data. R package version 0.1.0. <https://allisonhorst.github.io/palmerpenguins/>. doi: 10.5281/zenodo.3960218.

```
penguins = pd.read_csv(filepath_or_buffer = "01-daten/penguins.csv")

# Tiere mit unvollständigen Einträgen entfernen
penguins.drop(np.where(penguins.apply(pd.isna).any(axis = 1))[0], inplace = True)

print(penguins.info(), "\n");
```



```

<class 'pandas.core.frame.DataFrame'>
Index: 333 entries, 0 to 343
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species                333 non-null   object
1   island                 333 non-null   object
2   bill_length_mm         333 non-null   float64
3   bill_depth_mm          333 non-null   float64
4   flipper_length_mm      333 non-null   float64
5   body_mass_g            333 non-null   float64
6   sex                    333 non-null   object
7   year                   333 non-null   int64
dtypes: float64(4), int64(1), object(3)
memory usage: 23.4+ KB
None

```

Der Datensatz enthält Daten für drei Pinguinarten.

```
print(penguins.groupby(by = penguins['species']).size())
```

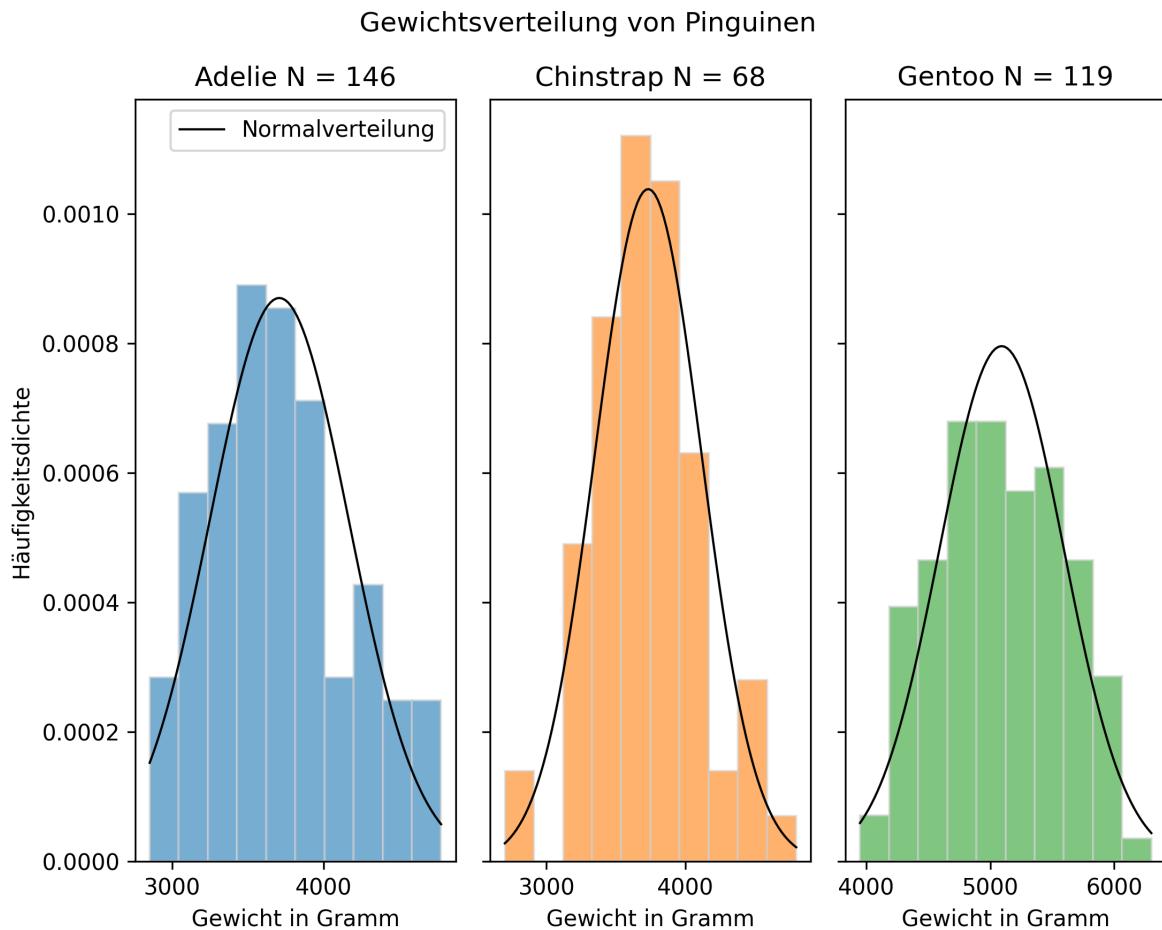
```

species
Adelie      146
Chinstrap   68
Gentoo     119
dtype: int64

```

Unter anderen wurde das Körpergewicht in Gramm gemessen, das in der Spalte 'body\_\_mass\_g' eingetragen ist. Die Gewichtsverteilung der drei Spezies wird jeweils mit einem Histogramm dargestellt. Außerdem werden für jede Spezies der Stichprobenmittelwert und die Stichprobenstandardabweichung bestimmt. Mit diesen Werten kann eine Normalverteilungskurve berechnet und in das Histogramm eingezeichnet werden (wie das geht, wird in Beispiel 3 gezeigt). So kann optisch geprüft werden, ob die empirische Verteilung der Werte in der Stichprobe einer Normalverteilung mit den selben Werten für Mittelwert und Standardabweichung entspricht. (Bei aus Messungen gewonnenen Daten ist dies häufig nicht so eindeutig, wie bei der Häufigkeitsverteilung von Würfelerggebnissen.)

## 2.4 Grafik



## 2.5 Code

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize = (7.5, 6), sharey = True, layout = 'tight')
plt.suptitle('Gewichtsverteilung von Pinguinen')

# Adelie
species = 'Adelie'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogramm
ax1.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C0', density = True)
```

```

ax1.set_xlabel('Gewicht in Gramm')
ax1.set_ylabel('Häufigkeitsdichte')
ax1.set_title(label = str(species) + " N = " + str(data.size))

## Normalverteilungskurve
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (stichprobenstandardabweichung * np.sqrt(2 * np.pi)) * np.exp(- (x_values - s

ax1.plot(x_values, y_values, color = 'black', linewidth = 1, label = 'Normalverteilung')
ax1.legend()

# Chinstrap
species = 'Chinstrap'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogramm
ax2.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C1', density = True)
ax2.set_xlabel('Gewicht in Gramm')
ax2.set_title(label = str(species) + " N = " + str(data.size))

## Normalverteilungskurve
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (stichprobenstandardabweichung * np.sqrt(2 * np.pi)) * np.exp(- (x_values - s

ax2.plot(x_values, y_values, color = 'black', linewidth = 1)

# Gentoo
species = 'Gentoo'
data = penguins['body_mass_g'][penguins['species'] == species]

## Histogramm
ax3.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C2', density = True)
ax3.set_xlabel('Gewicht in Gramm')
ax3.set_title(label = str(species) + " N = " + str(data.size))

## Normalverteilungskurve

```

```

stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = 1 / (stichprobenstandardabweichung * np.sqrt(2 * np.pi)) * np.exp(-(x_values - stichprobenmittelwert)**2 / (2 * stichprobenstandardabweichung**2))

ax3.plot(x_values, y_values, color = 'black', linewidth = 1)

plt.show()

```

Die Normalverteilung ist eine Dichtekurve, an die sich der Verlauf eines Histogramms mit einer gegen unendlich gehenden Anzahl von Messwerten und einer gegen Null gehenden Klassenbreite annähert.

### ! Wichtig 2: Histogramm

Das Histogramm ist eine grafische Darstellung der Häufigkeitsverteilung kardinal skaliertter Merkmale (d. h. mit numerischen, geordneten Merkmalsausprägungen). Die Daten werden in Klassen, die eine konstante oder variable Breite haben können, eingeteilt. Es werden direkt nebeneinanderliegende Rechtecke von der Breite der jeweiligen Klasse gezeichnet, deren Flächeninhalte die (relativen oder absoluten) Klassenhäufigkeiten darstellen. Die Höhe jedes Rechtecks stellt dann die (relative oder absolute) Häufigkeitsdichte dar, also die (relative oder absolute) Häufigkeit dividiert durch die Breite der entsprechenden Klasse.



Die Dichtefunktion der Normalverteilung beschreibt, welcher Anteil der Werte innerhalb eines bestimmten Wertebereichs liegt. Bei der Berechnung der relativen Häufigkeiten in Beispiel 2 haben wir gesehen, dass die Summe der relativen Häufigkeiten 1 ist. Dies entspricht der Fläche unterhalb der Dichtekurve.

Die Dichtefunktion der Normalverteilung ist definiert als:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Die Form der Normalverteilung ergibt sich aus dem Faktor  $e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$  der Funktionsgleichung. Das Maximum der Funktion liegt am Punkt  $x = \mu$ . Von dort fällt sie symmetrisch ab und nähert sich der x-Achse an. Der Abfall der Funktion erfolgt umso schneller, je kleiner  $\sigma$  ist. Die Wendepunkte der Kurve liegen jeweils eine Standardabweichung vom Mittelwert entfernt.

Eine Normalverteilung mit dem Mittelwert  $\mu = 0$  und einer Standardabweichung  $\sigma = 1$  heißt Standardnormalverteilung.

(Baitsch, Matthias 2019, S. 51–54)

## 2.12 Normalverteilung anpassen

Um die Verteilung in einem Datensatz durch eine Normalverteilung anzunähern, werden dessen Mittelwert und Standardabweichung in die Funktionsgleichung der Normalverteilung eingesetzt. Mit Python können die Berechnungen direkt vorgenommen werden. In der Handhabung einfacher sind die vom Paket SciPy bereitgestellten Funktionen, die im nächsten Abschnitt ausführlicher vorgestellt werden. Das folgende Beispiel zeigt die Berechnung und Visualisierung mit Python und mit SciPy.

### **i** Hinweis 3: Dichtekurven berechnen und darstellen

Betrachten wir die Verteilungskennwerte der Gruppe der Meerschweinchen, die eine Dosis von 2 Milligramm Vitamin C erhielten.

```
print(verteilungskennwerte(dose2), "\n");

dose2_mean = verteilungskennwerte(dose2, output = False)[1]
dose2_std = verteilungskennwerte(dose2, output = False)[4]

print("Exakter Mittelwert:", dose2_mean)
print("Exakte Standardabweichung:", dose2_std)
```

N: 20  
arithmetisches Mittel: 26.10  
Stichprobenfehler: 0.84  
Stichprobenvarianz: 14.24  
Stichprobenstandardabweichung: 3.77  
None

Exakter Mittelwert: 26.1  
Exakte Standardabweichung: 3.7741503052098744

Wenn wir die Standardabweichung und das arithmetische Mittel in die Normalverteilungsfunktion einsetzen, erhalten wir:

$$f(x) = \frac{1}{3.7742\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-26.10}{3.7742}\right)^2}$$

$$f(x) = 0.1057 \cdot e^{-\frac{1}{2}\left(\frac{x-26.10}{3.7742}\right)^2}$$

In Python können die Berechnungen umgesetzt und grafisch dargestellt werden:

```
# Histogram der Häufigkeitsdichte zeichnen
plt.hist(dose2, bins = 7, density = True, edgecolor = 'black', alpha = 0.6);
plt.title('Länge zahnbildender Zellen bei Meerschweinchen')

# Achsenbeschriftung
plt.xlabel('Länge der zahnbildenden Zellen (m)')
plt.ylabel('Häufigkeitsdichte')

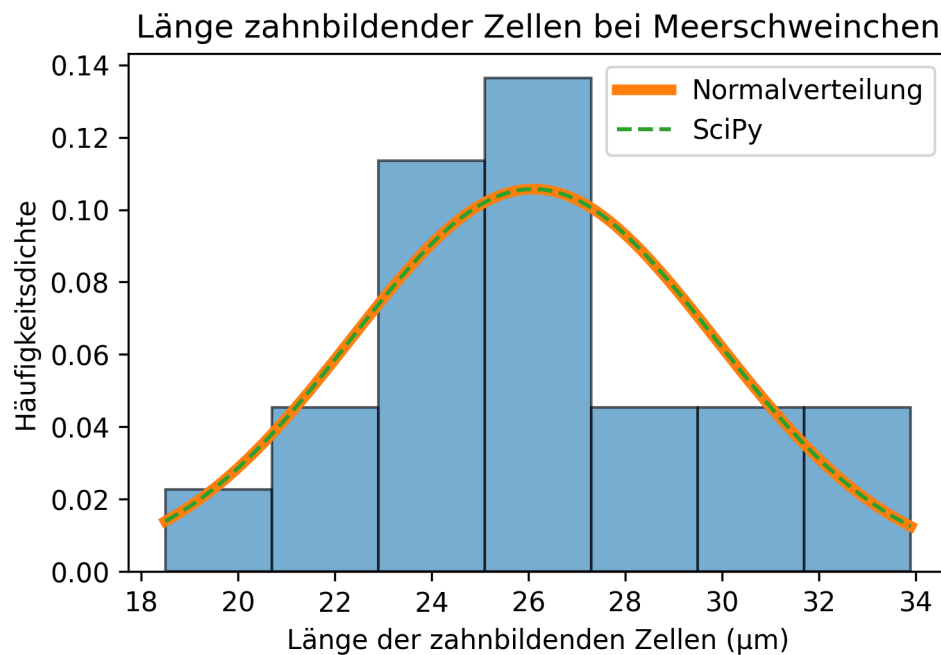
# Normalverteilung berechnen.
hist, bin_edges = np.histogram(dose2, bins = 7)

x_values = np.linspace(min(bin_edges), max(bin_edges), 100)

## Normalverteilungsfunktion mit Python berechnen
y_values = 1 / (dose2_std * np.sqrt(2 * np.pi)) * np.exp(-(x_values - dose2_mean) ** 2 /
plt.plot(x_values, y_values, label = 'Normalverteilung', lw = 4)

## scipy
y_values_scipy = scipy.stats.norm.pdf(x_values, loc = dose2_mean, scale = dose2_std)
plt.plot(x_values, y_values_scipy, label = 'SciPy', linestyle = 'dashed')

plt.legend()
plt.show()
```



Die Verteilung der Länge zahnbildender Zellen bei Meerschweinchen, die eine Dosis von 2 Milligramm Vitamin C erhielten, könnte einer Normalverteilung entsprechen. Aufgrund der geringen Stichprobengröße ist dies aber schwer zu beurteilen.

## 2.13 Das Paket SciPy

Funktionen zur Berechnung von Dichtekurven können über Paket SciPy importiert werden. Das Modul stats (statistical functions) umfasst zahlreiche Funktionen zum Testen von Hypothesen. Funktionen für die Normalverteilung werden wie folgt aufgerufen:

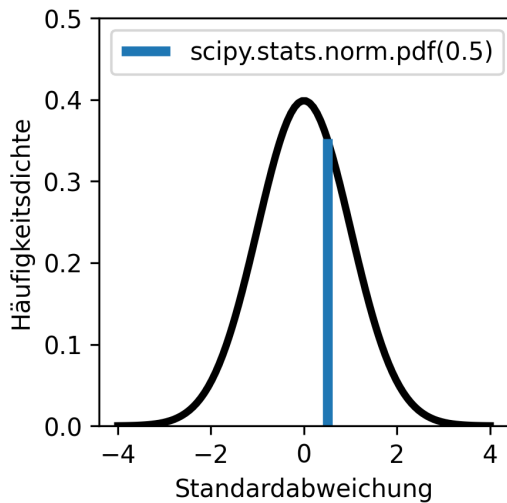
```
import scipy
print("Häufigkeitsdichte der Normalverteilung bei x = 0:", scipy.stats.norm.pdf(0), "\n")
```

Häufigkeitsdichte der Normalverteilung bei x = 0: 0.3989422804014327

Für die Normalverteilung sind vier Funktionen relevant:

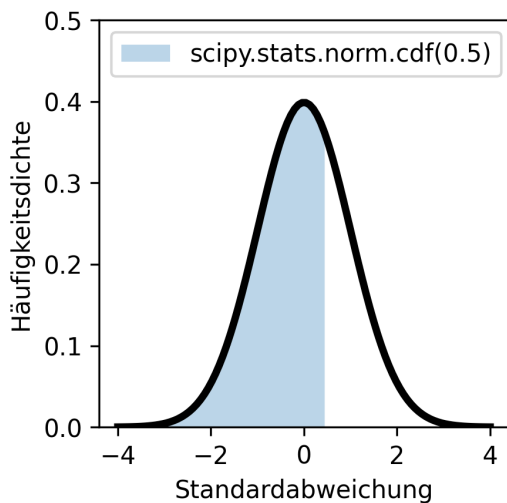
Mit den Parametern `loc = mittelwert` und `scale = standardabweichung` kann die Form der Normalverteilung angepasst werden. Standardmäßig wird die Standardnormalverteilung





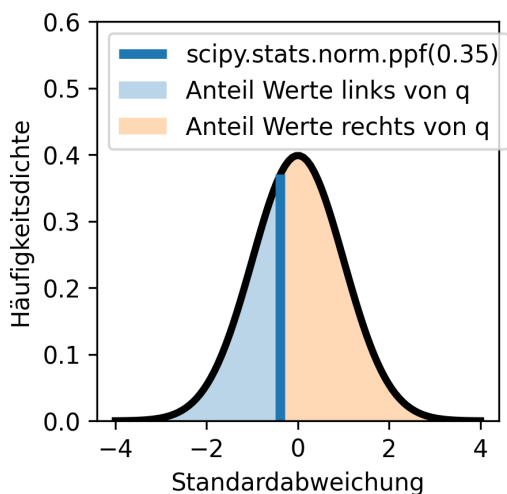
### Beschreibung

Die Funktion `scipy.stats.norm.pdf(x)` berechnet die Dichte der Normalverteilung am Punkt  $x$  (pdf = probability density function).  $x$  kann auch ein array sein - so wurde die linksstehende Kurve mit dem Befehl `scipy.stats.norm.pdf(np.linspace(-4, 4, 100))` berechnet.



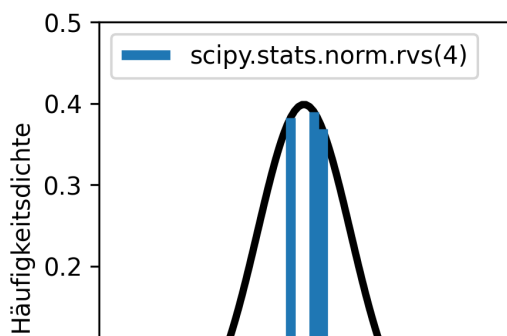
### Beschreibung

Die Funktion `scipy.stats.norm.cdf(x)` berechnet den Anteil der Werte links von  $x$  (cdf = cumulative density function).



### Beschreibung

Die Funktion `scipy.stats.norm.ppf(q)` ist die Quantilfunktion der Normalverteilung und die Umkehrfunktion der kumulativen Häufigkeitsdichtefunktion (cdf). Die Funktion berechnet für  $0 \leq q \leq 1$  den Wert  $x$ , links von dem der Anteil  $q$  aller Werte liegt und rechts von dem der Anteil  $1-q$  liegt (ppf = percentile point function).



### Beschreibung

Die Funktion `scipy.stats.norm.rvs(size)` zieht `size` Zufallszahlen aus der Normalverteilung.

*Hinweis: Die Zufallszahlen werden im Skript dynamisch gezogen.*

mit `loc = 0` und `scale = 1` berechnet. Die Parameter der Funktionen können Einzelwerte (Skalare) oder auch Arrays bzw. Listen sein.

## 2.14 Aufgaben Normalverteilung

Möglicherweise haben Sie schon einmal von Mensa International gehört, einer Vereinigung für Hochbegabte. Wer Mitglied in dieser Vereinigung werden möchte, soll einen höheren Intelligenzquotienten (IQ) haben als 98 % der Bevölkerung seines/ihres Herkunftslandes ([Wikipedia](#)).

1. Wenn der durchschnittliche IQ 100 und die Standardabweichung 15 beträgt, welchen IQ müssten Sie haben, um bei Mensa International aufgenommen zu werden?
2. Mensa International ist nicht die einzige Organisation ihrer Art. Andere Organisationen haben sogar noch strengere Kriterien. Welcher IQ wird benötigt, um hier Mitglied zu werden?
  - Intertel (Kriterium: IQ aus dem höchsten 1 %)
  - Triple Nine Society (Kriterium: IQ aus dem höchsten 0,1 %)
  - Prometheus Society (Kriterium: IQ aus dem höchsten 0,003 %)
3. Der IQ ist nicht mit angeborener Intelligenz gleichzusetzen und auch abhängig davon, wie viel Gelegenheit man zum Gehirntraining hatte, etwa durch den Schulbesuch. Der niedrigste durchschnittliche IQ wurde mit 71 [im Land Niger](#) gemessen. Angenommen Sie hätten einen IQ von 100. Würden Sie in Niger das Kriterium der Mensa International erfüllen?

### Musterlösung Normalverteilung

Aufgabe 1: Einen IQ von mehr als ...

```
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.98))
```

130.80623365947733

Aufgabe 2: Sie benötigen einen IQ von mindestens...

```
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.99))
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 0.999))
print(scipy.stats.norm.ppf(loc = 100, scale = 15, q = 1 - (0.003 / 100)))
```

134.8952181106126

146.3534845925172

160.19216216677682

Aufgabe 3: Nicht ganz.

```
print(scipy.stats.norm.cdf(loc = 71, scale = 15, x = 100))
```

0.9734024259789904

Übrigens: Wie [der Spiegel berichtet](#), schneiden Studierende mit mittelmäßigem Intelligenzquotienten ebenso erfolgreich ab wie Hochbegabte, vorausgesetzt sie sind neugierig genug und arbeiten gewissenhaft.

## 2.15 Konfidenzintervalle

Die schließende Statistik beruht auf dem Prinzip, von Stichprobenwerten auf den tatsächlichen Wert in der Grundgesamtheit zu schließen. Die Überlegung ist wie folgt:

1. Wenn eine Stichprobe aus einer Grundgesamtheit gezogen wird, dann streuen die Stichprobenwerte normalverteilt um den Mittelwert der Grundgesamtheit. Bei einer Normalverteilung liegen
  - 68,27 % aller Werte im Intervall  $\pm 1 s$ ,
  - 95,45 % aller Werte im Intervall  $\pm 2 s$  und
  - 99,73 % aller Werte im Intervall  $\pm 3 s$ .
2. Mit der gleichen Wahrscheinlichkeitsverteilung liegt der unbekannte Mittelwert der Grundgesamtheit um einen zufälligen Wert aus der Stichprobe.
3. Der Erwartungswert kann mit einer gewissen Wahrscheinlichkeit aus dem Standardfehler des Mittelwerts einer Stichprobe geschätzt werden. Man wählt dazu ein Konfidenzniveau, also eine Vertrauenswahrscheinlichkeit, dass der Erwartungswert tatsächlich im Bereich der Schätzung liegt. Der umgekehrte Fall, dass der Erwartungswert nicht im Bereich der Schätzung liegt, wird Signifikanz- oder Alphaniveau genannt und mit dem griechischen Buchstaben  $\alpha$  (alpha) gekennzeichnet.  $\alpha$  liegt im Bereich 0 - 1, das Konfidenzniveau ist  $1 - \alpha$  (siehe: [Fehler 1. und 2. Art](#)).
  - der Erwartungswert liegt in 68,27 % aller Fälle im Intervall  $\pm 1 \frac{s}{\sqrt{n}}$ ,
  - der Erwartungswert liegt in 95,45 % aller Fälle im Intervall  $\pm 2 \frac{s}{\sqrt{n}}$  und
  - der Erwartungswert liegt in 99,73 % aller Fälle im Intervall  $\pm 3 \frac{s}{\sqrt{n}}$ .

Häufig wird das Alphaniveau  $\alpha = 0.05$  bzw. das Konfidenzintervall 95 % gewählt, was  $\pm 1.96 \frac{s}{\sqrt{n}}$  entspricht. Dies gilt aber nur für große Stichproben. Für kleine Stichprobengrößen folgen die Stichprobenmittelwerte der t-Verteilung, die im nächsten Abschnitt vorgestellt wird.

to do Maik: hier könnte / müsste man noch einseitige und zweiseitige Hypothesentests und den Begriff “Alpha-Halbe” einführen. Das ließe sich auch gut grafisch mit nur nach rechts gehenden und beidseitigen Pfeilen darstellen.

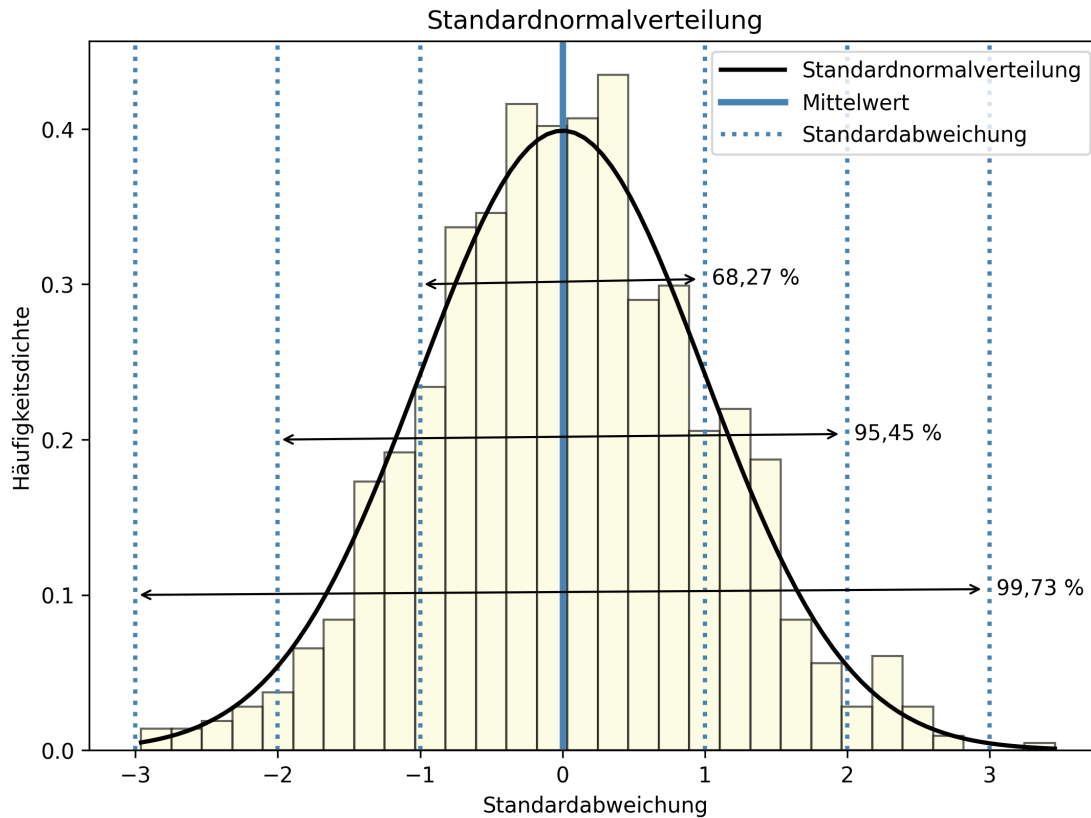
Im folgenden Beispiel wird die Idee, dass mit einer gewissen Wahrscheinlichkeit vom Stichprobenmittelwert auf den Mittelwert der Grundgesamtheit (Erwartungswert) geschlossen werden kann, noch einmal grafisch dargestellt.

#### **i** Hinweis 4: Prinzip der schließenden Statistik

## 2.16 Standardnormalverteilung

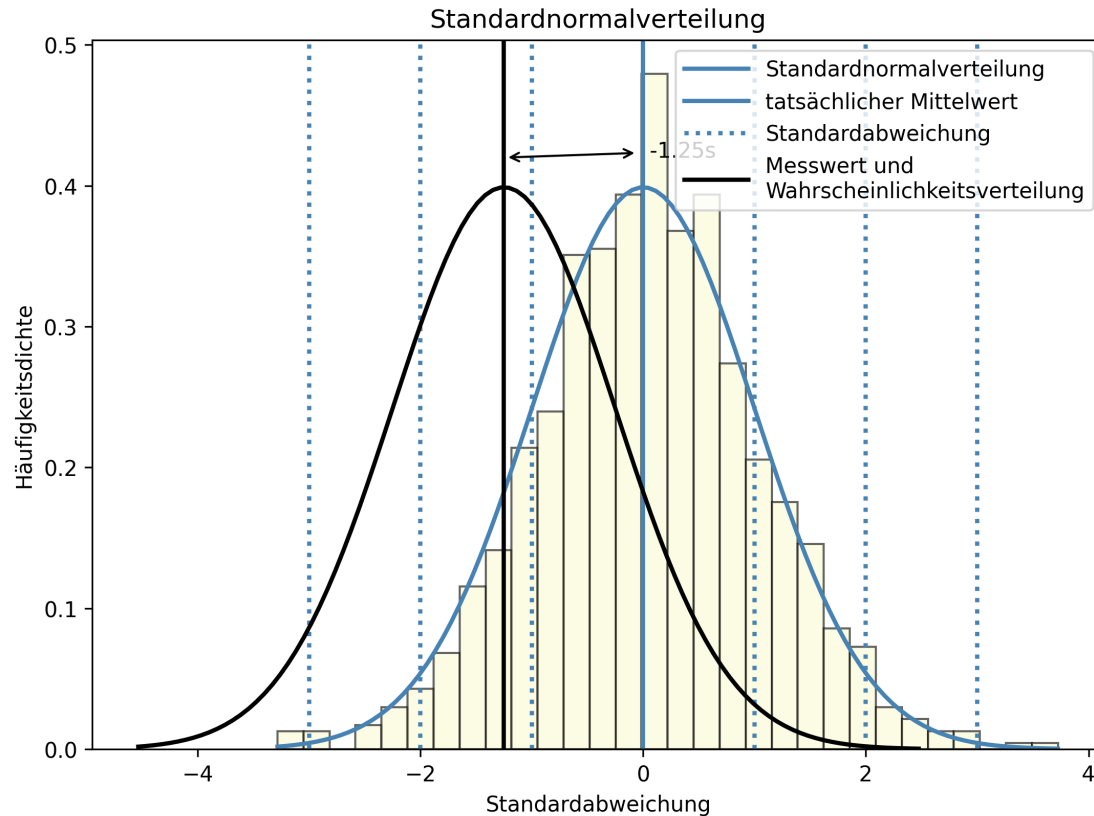
In einer Normalverteilung kommen Werte in der Nähe des Erwartungswerts häufiger vor als weit vom Erwartungswert entfernt liegende Werte. Wie häufig oder selten ein Wert relativ zum Mittelwert der Verteilung vorkommt, kann mit der Standardabweichung ausgedrückt werden.

In der Grafik sehen Sie zufällig gezogene Werte und eine Normalverteilungskurve.



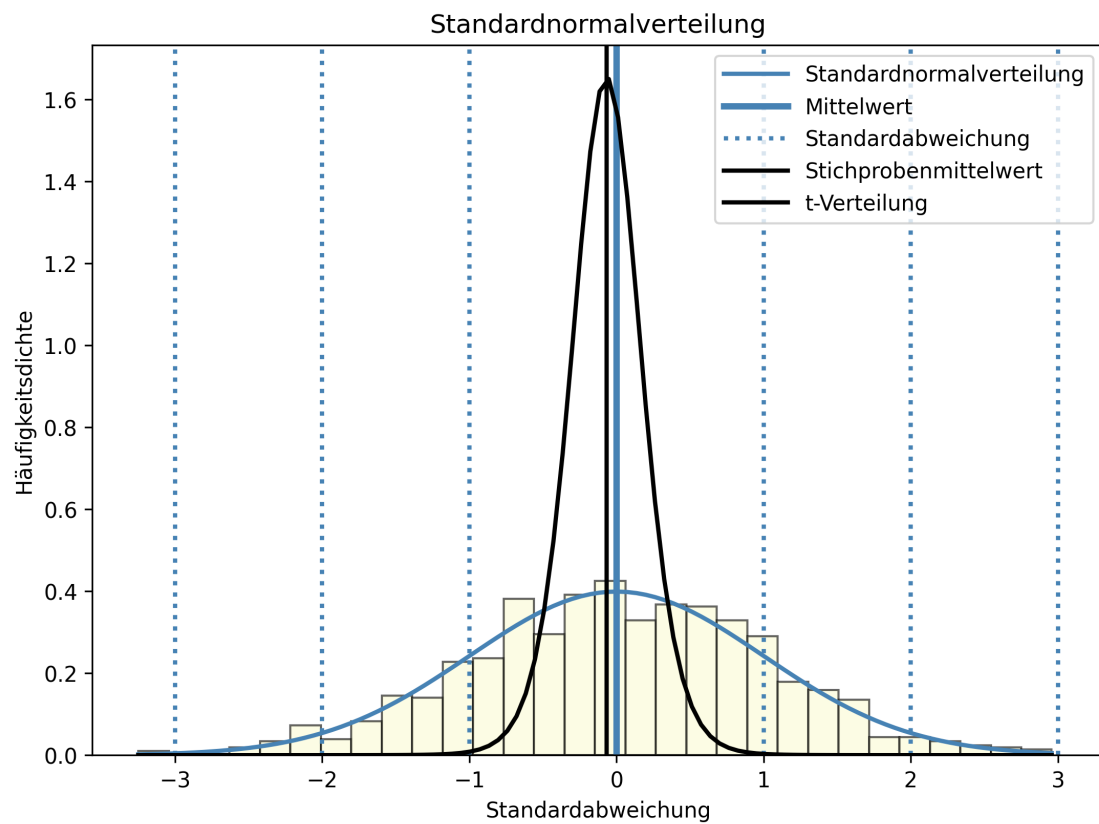
## 2.17 Einzelner Messwert

Ein einzelner Messwert aus der Verteilung entspricht so gut wie nie dem Erwartungswert. Man weiß aber, dass ein Wert nahe des Erwartungswerts häufiger vorkommt, als ein weit entfernter.



## 2.18 Stichprobe $N = 12$

Der Mittelwert einer Stichprobe entspricht ebenfalls so gut wie nie dem Erwartungswert. Der zu erwartende, in Einheiten des Standardfehlers des Mittelwerts ausgedrückte Streubereich ist jedoch erheblich schmäler als der eines einzelnen Messwerts. Dies ist auch grafisch gut zu sehen. Die Dichtekurve ist erheblich schmäler und höher als die Normalverteilungskurve eines einzelnen Messwerts. Dies liegt an der geringen Standardabweichung in der Stichprobe von  $\sim 0.2$ , was die Kurve staucht.



## 2.19 Code

Code für das Panel Stichprobe  $N = 12$

```

import numpy as np
import matplotlib.pyplot as plt
import scipy

# Parameter der Standardnormalverteilung
mu, sigma = 0, 1 # Mittelwert und Standardabweichung

# Daten generieren
seed = 4
np.random.seed(seed = seed)
data = np.random.default_rng().normal(mu, sigma, 1000)

# Grafik
plt.figure(figsize = (8.5, 6))

# Histogramm plotten
array, bins, patches = plt.hist(data, bins = 30, density = True, alpha = 0.6, color = 'lightblue')

# Mittelwert einzeichnen
mean_line = plt.axvline(mu, color = 'steelblue', linestyle = 'solid', linewidth = 3)

# positive und negative Standardabweichungen einzeichnen
pos_std_lines = [plt.axvline(mu + i * sigma, color = 'steelblue', linestyle = 'dotted', linewidth = 3) for i in [1, 2]]
neg_std_lines = [plt.axvline(mu - i * sigma, color = 'steelblue', linestyle = 'dotted', linewidth = 3) for i in [1, 2]]

# Normalverteilungskurve
x_values = np.linspace(min(bins), max(bins), 100)
y_values = 1 / (sigma * np.sqrt(2 * np.pi)) * np.exp(- (x_values - mu) ** 2 / (2 * sigma ** 2))
normal_dist_curve = plt.plot(x_values, y_values, color = 'steelblue', linestyle = 'solid', linewidth = 3)

# Stichprobe
N = 12
np.random.seed(seed = 4)
stichprobe = np.random.default_rng().normal(mu, sigma, N)

stichprobenstandardabweichung = stichprobe.std(ddof = 1)
stichprobenmittelwert = stichprobe.mean()
standardfehler = stichprobenstandardabweichung / np.sqrt(len(stichprobe))

# Histogramm berechnen
# hist, bins = np.histogram(stichprobe, bins = 30, density = True)

# Standardfehlerkurve Stichprobe
# x_values = np.linspace(min(bins), max(bins), 100)
x = np.linspace(stichprobenmittelwert - 4 * stichprobenstandardabweichung, stichprobenmittelwert + 4 * stichprobenstandardabweichung, 100)
y_values = scipy.stats.t.pdf(x = x_values, df = N - 1, loc = stichprobenmittelwert, scale = standardfehler)

# Stichprobenmittelwert einzeichnen
mean_stichprobe = plt.axvline(stichprobenmittelwert, color = 'black', linestyle = 'solid', linewidth = 3)

# Verteilungskurve einzeichnen
stichprobe_dist_curve = plt.plot(x_values, y_values, color = 'black', linestyle = 'solid', linewidth = 3)

```

## 2.20 Die t-Verteilung

[Wikipedia](#)

[Anzahl der Freiheitsgrade](#)

### ! Wichtig 3: Anzahl Freiheitsgrade

“Die Anzahl unabhängiger Information, die in die Schätzung eines Parameters einfließen, wird als Anzahl der Freiheitsgrade bezeichnet. Im Allgemeinen sind die Freiheitsgrade einer Schätzung eines Parameters gleich der Anzahl unabhängiger Einzelinformationen, die in die Schätzung einfließen, abzüglich der Anzahl der zu schätzenden Parameter, die als Zwischenschritte bei der Schätzung des Parameters selbst verwendet werden. Beispielsweise fließen  $n$  Werte in die Berechnung der Stichprobenvarianz ein. Dennoch lautet die Anzahl der Freiheitsgrade  $n - 1$ , da als Zwischenschritt der Mittelwert geschätzt wird und somit ein Freiheitsgrad verloren geht.”

Anzahl der Freiheitsgrade (Statistik). von verschiedenen [Autor:innen](#) steht unter der Lizenz [CC BY-SA 4.0](#) ist abrufbar auf [Wikipedia](#). 2025

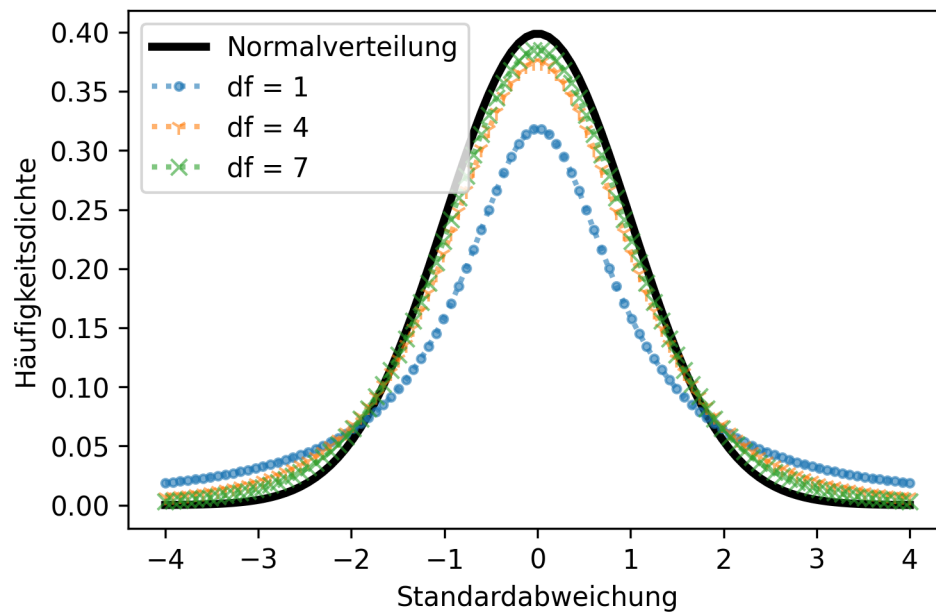
- 
- [Gammafunktion](#)



- 
- 
- 
- 
- 

## 2.21 Grafik

Das Argument df der t-Verteilung



## 2.22 Code

```
x_values = np.linspace(-4, 4, 100)

# Normalverteilung
y_values = scipy.stats.norm.pdf(x_values)
plt.plot(x_values, y_values, color = 'black', lw = 3, label = 'Normalverteilung')
# plt.ylim(bottom = 0, top = 0.5)

# t-Verteilungen
marker = [".", "1", "x"]

[plt.plot(x_values, scipy.stats.t.pdf(x_values, df = (i + (i - 1) * 2)), linestyle = 'dotted',
          marker = marker[i % 3], label = 't-Verteilung, df = %d' % (i + (i - 1) * 2)) for i in range(1, 10)]

plt.suptitle('Das Argument df der t-Verteilung')
plt.xlabel('Standardabweichung')
plt.ylabel('Häufigkeitsdichte')
plt.legend(loc = 'upper left')
plt.show()
```

- - 
  -
- 
- 

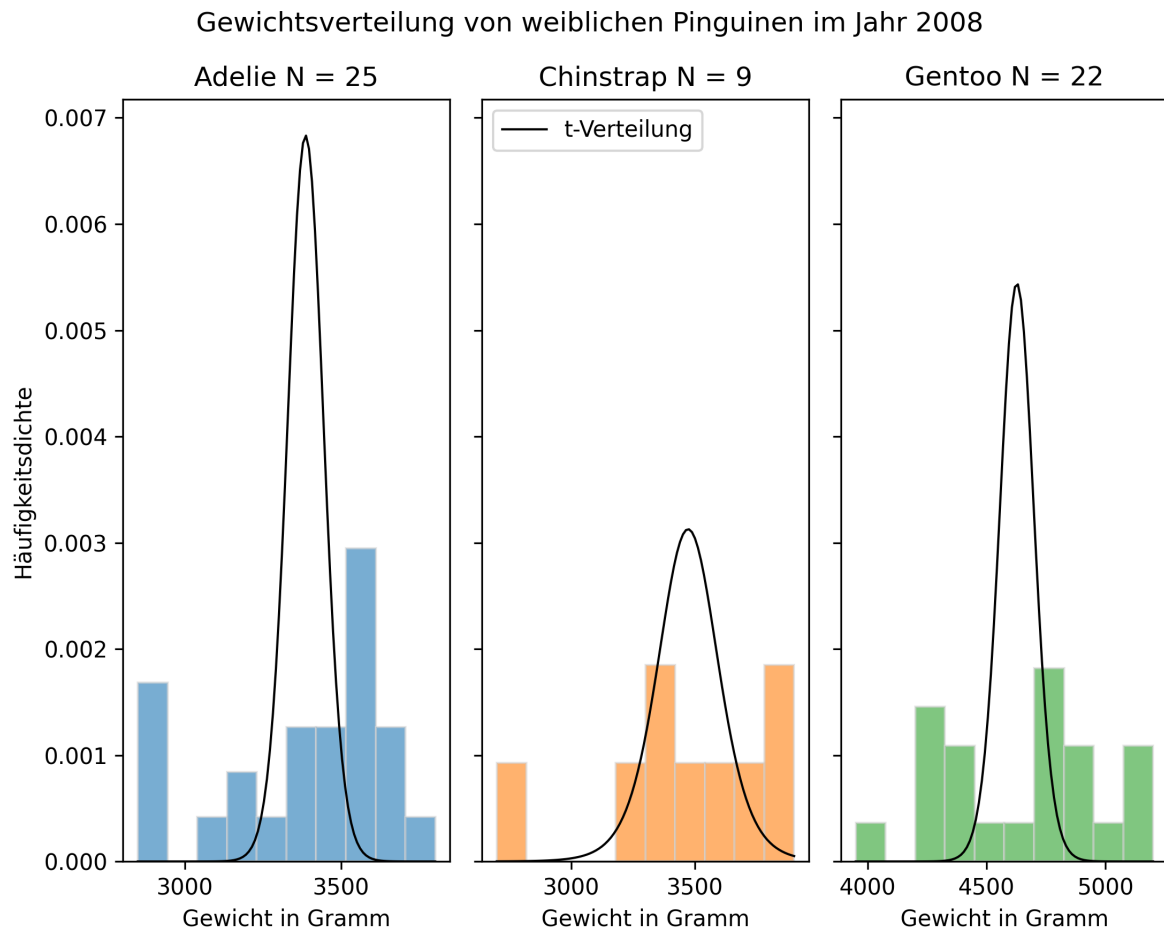
### 💡 Tipp 2: t-Verteilung oder Normalverteilung?

Am Computer ist die t-Verteilung genauso leicht (oder schwer) zu berechnen wie die Normalverteilung. Da sich die t-Verteilung für größere Stichprobengrößen ohnehin der Normalverteilung annähert, empfiehlt es sich, stets die t-Verteilung zu verwenden.

### 2.22.1 Beispiel Gewicht weiblicher Pinguine

```
print(penguins.groupby(by = [penguins['species'], penguins['sex'], penguins['year']]).size())
```

## 2.23 Grafik



## 2.24 Code

```
year = 2008
sex = 'female'
species = 'Adelie'

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize = (7.5, 6), sharey = True, layout = 'tight')
plt.suptitle('Gewichtsverteilung von weiblichen Pinguinen im Jahr 2008')

# Adelie
data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
```

```

stichprobengröße = data.size

## Histogramm
ax1.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C0', density = True)
ax1.set_xlabel('Gewicht in Gramm')
ax1.set_ylabel('Häufigkeitsdichte')
ax1.set_title(label = str(species) + " N = " + str(stichprobengröße))

## t-Verteilung des Stichprobenmittelwerts
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
standardfehler = stichprobenstandardabweichung / np.sqrt(stichprobengröße)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = stichprobenmittelwert, scale = standardfehler, c

ax1.plot(x_values, y_values, color = 'black', linewidth = 1, label = 't-Verteilung')
ax1.legend(loc = 'upper left')

# Chinstrap
species = 'Chinstrap'

data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
stichprobengröße = data.size

## Histogramm
ax2.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C1', density = True)
ax2.set_xlabel('Gewicht in Gramm')
ax2.set_title(label = str(species) + " N = " + str(stichprobengröße))

## t-Verteilung des Stichprobenmittelwerts
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
standardfehler = stichprobenstandardabweichung / np.sqrt(stichprobengröße)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = stichprobenmittelwert, scale = standardfehler, c

ax2.plot(x_values, y_values, color = 'black', linewidth = 1)

# Gentoo
species = 'Gentoo'

```

```

data = penguins['body_mass_g'][(penguins['species'] == species) & (penguins['sex'] == sex) &
stichprobengröße = data.size

## Histogramm
ax3.hist(data, alpha = 0.6, edgecolor = 'lightgrey', color = 'C2', density = True)
ax3.set_xlabel('Gewicht in Gramm')
ax3.set_title(label = str(species) + " N = " + str(stichprobengröße))

## t-Verteilung des Stichprobenmittelwerts
stichprobenmittelwert = data.mean()
stichprobenstandardabweichung = data.std(ddof = 1)
standardfehler = stichprobenstandardabweichung / np.sqrt(stichprobengröße)
hist, bin_edges = np.histogram(data)
x_values = np.linspace(min(bin_edges), max(bin_edges), 100)
y_values = scipy.stats.t.pdf(x_values, loc = stichprobenmittelwert, scale = standardfehler, c

ax3.plot(x_values, y_values, color = 'black', linewidth = 1)

plt.show()

```

## 2.25 Aufgabe Konfidenzintervalle

- 1.
- 2.

### 💡 Tipp 3: Tipp und Musterlösung

Folgende Schritte helfen Ihnen bei der Lösung:

1. Bestimmen Sie den Stichprobenmittelwert  $\bar{x}$ .
2. Bestimmen Sie die Stichprobenstandardabweichung  $s$ , die Stichprobengröße  $N$  und den Standardfehler  $\frac{s}{\sqrt{N}}$ .
3. Bestimmen Sie die z- oder t-Werte der Normal- bzw. t-Verteilung für das gewählte Konfidenzniveau - für einen zweiseitigen Hypothesentest  $\frac{\alpha}{2}$  und  $1 - \frac{\alpha}{2}$
4. Berechnen Sie das Konfidenzintervall  $\bar{x} \pm t_{\alpha/2} \frac{s}{\sqrt{n}}$ .

## 💡 Musterlösung

Alphaniveau definieren und Pinguine auswählen

```
alpha = 1 - 0.9
data = penguins[(penguins['sex'] == sex) & (penguins['year'] == year)]
```

1. Stichprobenmittelwerte bestimmen.

```
penguin_means = data['body_mass_g'].groupby(by = data['species']).mean()
print(penguin_means)
```

```
species
Adelie      3386.000000
Chinstrap   3472.222222
Gentoo      4627.272727
Name: body_mass_g, dtype: float64
```

2. Stichprobenstandardabweichung, Stichprobengröße und Standardfehler bestimmen.

```
penguin_stds = data['body_mass_g'].groupby(by = data['species']).std(ddof = 1)
penguin_sizes = data['body_mass_g'].groupby(by = data['species']).size()
penguin_stderrors = penguin_stds / np.sqrt(penguin_sizes)

print("Stichprobenstandardabweichungen:\n", penguin_stds)
print("\nStichprobengrößen:\n", penguin_sizes)
print("\nStandardfehler:\n", penguin_stderrors)
```

```
Stichprobenstandardabweichungen:
species
Adelie      288.862712
Chinstrap   370.903551
Gentoo      339.722321
Name: body_mass_g, dtype: float64
```

51

```
Stichprobengrößen:
species
Adelie      25
Chinstrap    9
Gentoo      22
Name: body_mass_g, dtype: int64
```

## 3 Lineare Parameterschätzung

- 
- 

### 3.1 Messreihe Hooke'sches Gesetz

Hooke'sche Gesetz



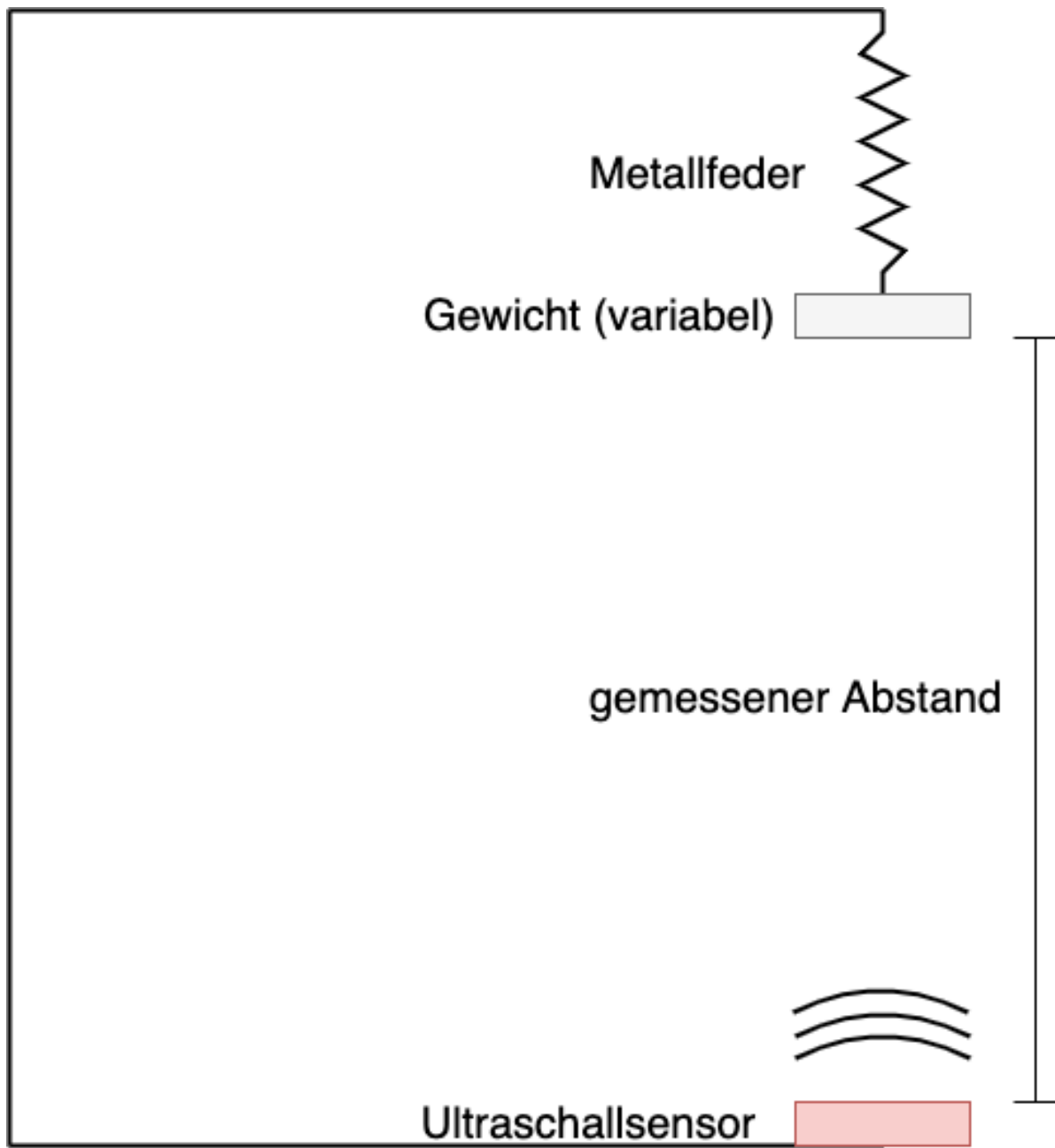


Abbildung 3.1: Versuchsaufbau

```
dateipfad = "01-daten/hooke_data.csv"
hooke = pd.read_csv(filepath_or_buffer = dateipfad, sep = ';')
```

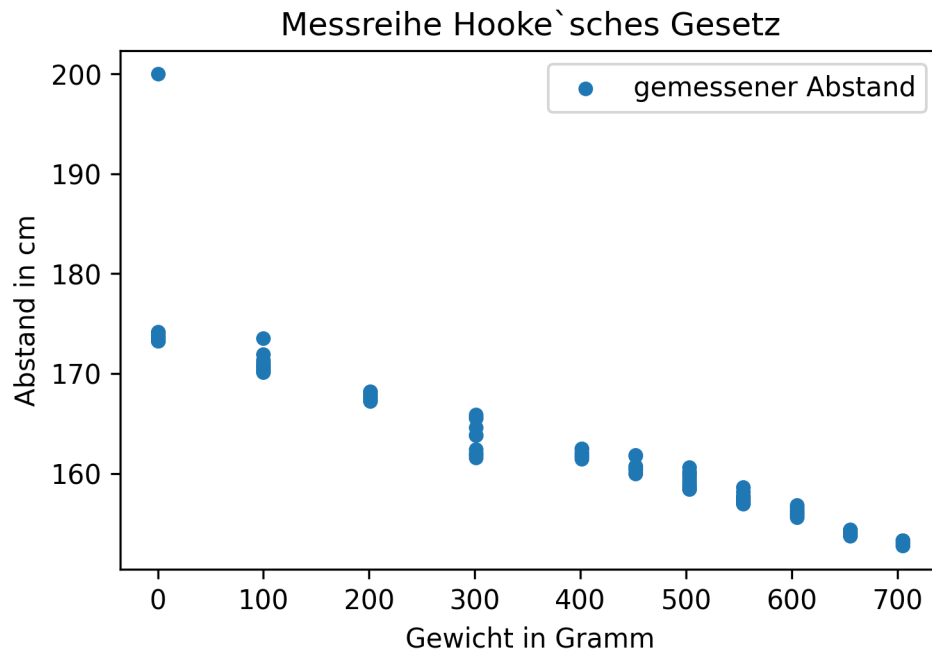
### 3.1.1 Deskriptive Statistik

```
print(hooke.head(), "\n")
print(hooke.tail())
```

```
hooke.describe()
```

```
hooke.groupby(by = 'mass')['distance'].describe()
```

```
hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = "Messreihe Hooke`sches Gesetz")  
plt.legend()  
plt.show()
```



standardisiert in Einheiten der Standardabweichung (z-Werten)

```

gewicht = 0

# z-Transformation manuell berechnen
mittelwert_ausdehnung = hooke[hooke['mass'] == gewicht].loc[:, 'distance'].mean()
standardabweichung_ausdehnung = hooke[hooke['mass'] == gewicht].loc[:, 'distance'].std(ddof=1)

z_values = hooke[hooke['mass'] == gewicht].loc[:, 'distance'].apply(lambda x: (x - mittelwert_ausdehnung) / standardabweichung_ausdehnung)
z_values.name = 'z-values'

# z-Transformation mit scipy
scipy_z_values = scipy.stats.zscore(hooke[hooke['mass'] == gewicht].loc[:, 'distance'], ddof=1)
scipy_z_values.name = 'scipy z-values'

# gemeinsame Ausgabe der Daten
print(pd.concat([hooke[hooke['mass'] == gewicht], z_values, scipy_z_values], axis = 1))

```

## Ausreißer

### ! Wichtig 4: Ausreißer

In der Statistik wird ein Messwert als Ausreißer bezeichnet, wenn dieser stark von der übrigen Messreihe abweicht. In einer Messreihe können auch mehrere Ausreißer auftreten. Diese Werte können zur Verbesserung der Schätzung aus der Messreihe entfernt werden, wenn anzunehmen ist, dass diese durch Messfehler und andere Störgrößen verursacht sind.

Eine Möglichkeit, Ausreißer zu identifizieren, ist die z-Transformation. Dabei muss ein Schwellenwert gewählt werden, ab dem ein Messwert als Ausreißer klassifiziert werden soll, bspw. 2,5 oder 3 Einheiten der Standardabweichung. In der Statistik wurde eine ganze Reihe von Ausreißertests entwickelt (siehe [Ausreißertests](#))

Die Einstufung eines Messwerts als Ausreißer kann aber nicht allein auf der Grundlage statistischer Verfahren erfolgen, sondern ist immer eine Ermessensentscheidung auf der Grundlage Ihres Fachwissens. Denn nicht alle abweichenden Werte sind automatisch ungültig, sondern treten mit einer gewissen statistischen Wahrscheinlichkeit auf (siehe Kapitel Normalverteilung). Man spricht dann von gültigen Extremwerten.

Ausreißer von verschiedenen [Autor:innen](#) steht unter der Lizenz [CC BY-SA 4.0](#) und ist abrufbar auf [Wikipedia](#)

```
hooke.drop(index = 113, inplace = True)

hooke.groupby(by = 'mass')['distance'].describe()
```

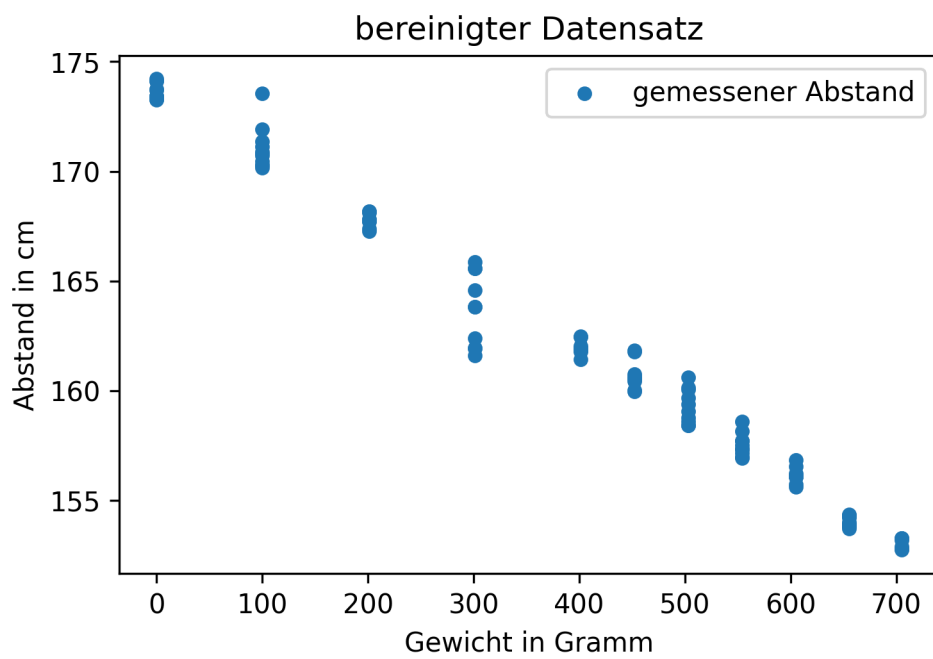
```
gewicht = 301

z_values = scipy_z_values = scipy.stats.zscore(hooke[hooke['mass'] == gewicht].loc[:, 'distance'])
z_values.name = 'z-values'

print(pd.concat([hooke[hooke['mass'] == gewicht], z_values], axis = 1))
```

### 3.1.2 Explorative Statistik

```
hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = 'bereinigter Datensatz', ylab = 'Abstand in cm', xlab = 'Gewicht in Gramm', legend = True)  
plt.legend()  
plt.show()
```



```

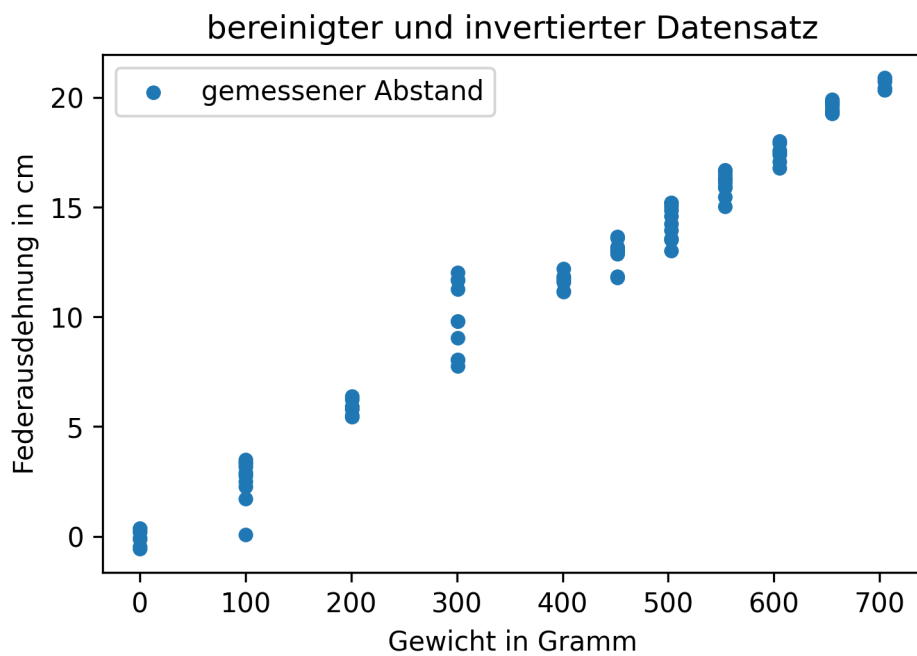
nullpunkt = hooke[hooke['mass'] == 0].loc[:, 'distance'].mean()
print(f"Nullpunkt: {nullpunkt:.2f} cm")

hooke['distance'] = hooke['distance'].sub(nullpunkt).mul(-1)

hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = 'bereinigter und invertierter Datensatz')
plt.legend()

plt.show()

```





```

# Mittelwerte nach Gewicht
distance_means_by_weight = hooke['distance'].groupby(by = hooke['mass']).mean()
distance_means_by_weight.name = 'Federausdehnung'

# Standardfehler nach Gewicht
distance_stderrors_by_weight = hooke['distance'].groupby(by = hooke['mass']).std(ddof = 1).d
distance_stderrors_by_weight.name = 'Standardfehler'

datenpunkte = hooke.plot(x = 'mass', y = 'distance', kind = 'scatter', title = 'bereinigter v

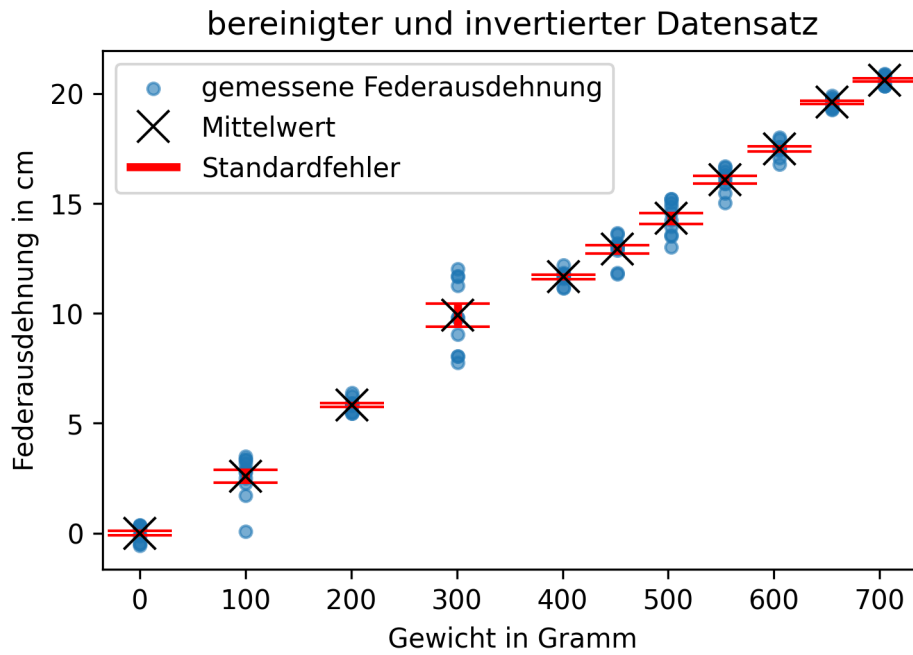
# Legendeneinträge abgreifen
# siehe: https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.get_legend_handles_l
datenpunkte_handle, datenpunkte_label = datenpunkte.get_legend_handles_labels()

errorbar_container = plt.errorbar(
    x = distance_means_by_weight.index, y = distance_means_by_weight, yerr = distance_stderrors
    linestyle = 'none', marker = 'x', color = 'black', markersize = 12, elinewidth = 3, ecolor

# Legende manuell erstellen
# siehe: https://matplotlib.org/stable/api/container_api.html#matplotlib.container.ErrorbarC
plt.legend([datenpunkte_handle[0], errorbar_container.lines[0], errorbar_container.lines[2] [
    [datenpunkte_label[0], 'Mittelwert', 'Standardfehler'],
    loc = 'upper left')
plt.show()

print("\n", pd.concat([distance_means_by_weight, distance_stderrors_by_weight], axis = 1))

```



### 3.2 Federkonstante bestimmen

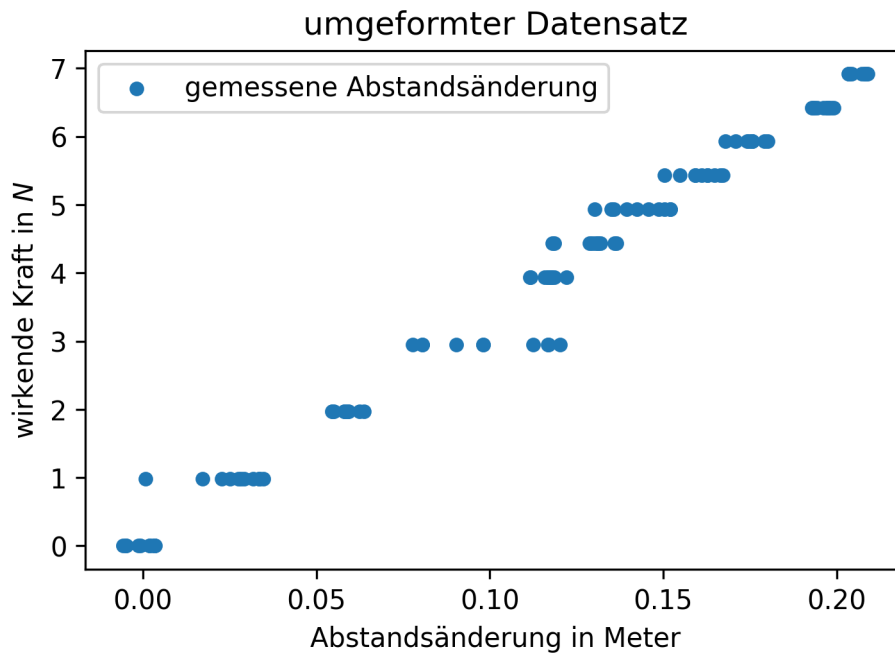
```
hooke['mass'] = hooke['mass'].div(1000).mul(9.81)
hooke.rename(columns = {'mass': 'force'}, inplace = True)

hooke['distance'] = hooke['distance'].div(100)

print(hooke.head())
```

```
hooke.plot(x = 'distance', y = 'force', kind = 'scatter', title = 'umgeformter Datensatz', y=
plt.legend()

plt.show()
```



### 3.2.1 Lineare Ausgleichsrechnung

[Wikipedia](#)

[drate](#)

[kleinsten Qua-](#)

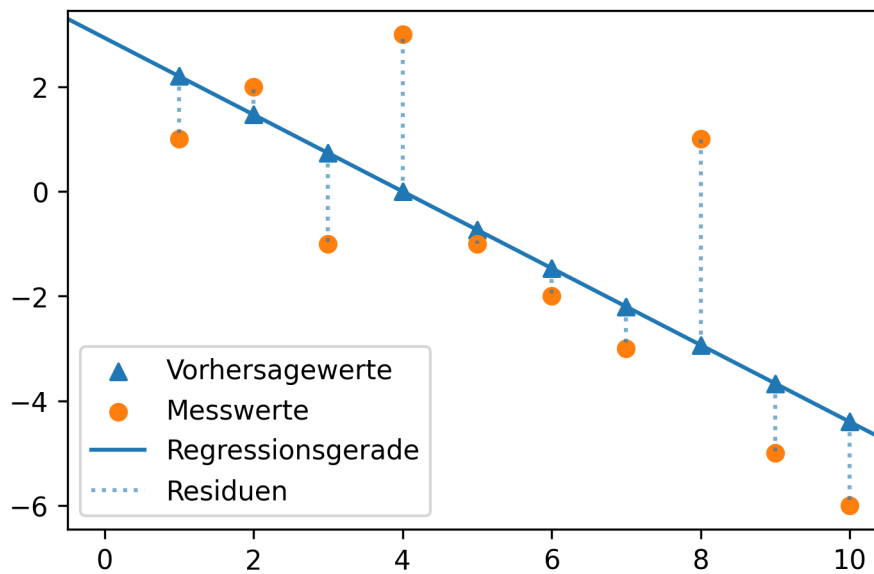
### **i** Hinweis 5: Methode der kleinsten Quadrate

Mit der Methode der kleinsten Quadrate soll diejenige Gerade  $\hat{y} = \beta_0 + \beta_1 \cdot x$  gefunden werden, die die quadrierten Abstände der Vorhersagewerte  $\hat{y}$  von den tatsächlich gemessenen Werten  $y$  minimiert. Die Werte  $y_i - \hat{y}_i$  sind die Residuen  $e_i$ . Es gilt also:

$$\sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N e_i^2 = \min$$

Grafisch kann man sich die Minimierung der quadrierten Abstände so vorstellen.

## 3.3 Grafik



Regressionskoeffizienten: [ 2.93333333 -0.73333333]

## 3.4 Code

```

x = np.arange(1, 11)
y = - x.copy() + 4
y[0] -= 2
y[2] -= 2
y[3] += 3
y[-3] += 5

lm = poly.polyfit(x, y, 1)
vorhersagewerte = poly.polyval(x, lm)

plt.scatter(x, vorhersagewerte, label = 'Vorhersagewerte', marker = "^", color = "tab:blue")
plt.scatter(x, y, label = 'Messwerte', marker = 'o', color = "tab:orange")
plt.axline(xy1 = (0, lm[0]), slope = lm[1], label = "Regressionsgerade", color = "tab:blue")
dotted = plt.vlines(x, ymin = vorhersagewerte, ymax = y, alpha = 0.6, ls = 'dotted', label

plt.legend()
plt.show()

print("Regressionskoeffizienten:", lm)

```

Die eingezeichnete Gerade entspricht der linearen Funktion  $\hat{y} = \beta_0 + \beta_1 \cdot x + e_i$ . Die Dreiecksmarker sind die Vorhersagewerte  $\hat{y}_i$  des linearen Modells für die Werte  $x_i = np.arange(1, 11)$ . Die tatsächlichen Messwerte  $y$  sind mit Kreismarkern markiert. Die Länge der gestrichelten Linien entspricht der Größe der Abweichung zwischen den Mess- und Vorhersagewerten  $y_i - \hat{y}_i$ , also den Residuen  $e_i$ .

Gesucht wird diejenige Gerade, die die Summe der quadrierten Residuen minimiert. Die gesuchten Werte  $\beta_0$  und  $\beta_1$  sind die Kleinst-Quadrate-Schätzer.

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x}) \cdot (y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Der Vollständigkeit halber leiten wir die Kleinst-Quadrate-Schätzer her. Gesucht werden die Werte für  $\beta_0$  und  $\beta_1$ , damit die Summe der Residuenquadrate  $\sum_{i=1}^n e_i^2$  möglichst klein wird. Die Residuenquadratsumme ist die Summe der quadrierten Differenzen aus beobachteten Werten  $y_i$  und der durch die lineare Funktion vorhergesagten Werte.

$$\sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i))^2$$

Wir untersuchen also eine Funktion, die von zwei Variablen abhängig ist.

$$f(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i))^2$$

Das Summenzeichen ist die Kurzschreibweise für eine Summe.

$$f(\beta_0, \beta_1) = (y_1 - (\beta_0 + \beta_1 \cdot x_1))^2 + (y_2 - (\beta_0 + \beta_1 \cdot x_2))^2 + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n))^2$$

Im Minimum der Funktion müssen die beiden partiellen Ableitungen gleich Null sein (Warum das so ist, wird [hier](#) leicht verständlich erklärt.)

#### **i** Hinweis 15: Partielle Ableitung

Die partielle Ableitung ist die Ableitung einer Funktion mit mehreren Variablen nach einer Variablen, wobei die übrigen Variablen als Konstanten behandelt werden. Für eine Funktion  $f(x, y) = 2x + y^2$  wird die partielle Ableitung nach x so ausgedrückt:

$$\frac{\partial f(x, y)}{\partial x}$$

- Das Symbol  $\frac{\partial}{\partial x}$  ist die kursive Darstellung des kyrillischen Kleinbuchstaben (d) und wird als “del” gelesen. Es zeigt an, dass eine partielle Ableitung durchgeführt wird.
- Im Zähler steht die Funktion, die abgeleitet werden soll. Im Nenner steht die Variable nach der abgeleitet wird. Der Term wird gelesen als “del f von x und y nach del x”.

Die partielle Ableitung  $\frac{\partial f(x, y)}{\partial x} = 2$ .  $y^2$  wird als Konstante behandelt (z. B.  $5^2$ ) und ist abgeleitet Null.

Die partielle Ableitung  $\frac{\partial f(x, y)}{\partial y} = 2y$ .  $2x$  wird als Konstante behandelt (z. B.  $2 \cdot 3$ ) und ist abgeleitet Null.

In beiden partiellen Ableitungen sind  $x_i$  und  $y_i$  konstant. In der partiellen Ableitung nach  $\beta_0$  ist außerdem  $\beta_1$  konstant, in der partiellen Ableitung nach  $\beta_1$  ist entsprechend  $\beta_0$  konstant.

### 3.5 partielle Ableitung nach dem y-Achsenschnittpunkt

Für die partielle Ableitung nach  $\beta_0$  gilt also nach der Kettenregel für die äußere Funktion (oben) und die innere Funktion (Mitte):

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_0} = 2 \cdot (y_1 - (\beta_0 + \beta_1 \cdot x_1)) + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n)) = 2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) \cdot (0 - (1 + 0 \cdot 0))$$

Für die partielle Ableitung nach  $\beta_0$  gilt also:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_0} = -2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) = 0$$

Diese kann vereinfacht werden, indem der Vorfaktor  $-2$  entfällt (weil  $-2 \cdot 0 = 0$  gelten muss) und die Vorzeichen aufgelöst werden. Sodass:

$$\sum_{i=1}^n (y_i - \beta_0 - \beta_1 \cdot x_i) = 0$$

Man kann auch schreiben:

$$\sum_{i=1}^n y_i - \sum_{i=1}^n \beta_0 - \sum_{i=1}^n \beta_1 \cdot x_i = 0$$

$\beta_0$  und  $\beta_1$  sind Konstanten, sodass gilt  $\sum_{i=1}^n \beta_0 = \beta_0 \cdot \sum_{i=1}^n 1 = \beta_0 \cdot n$  und  $\sum_{i=1}^n \beta_1 \cdot x_i = \beta_1 \cdot \sum_{i=1}^n 1 \cdot x_i$ . So gilt:

$$\sum_{i=1}^n y_i - n \cdot \beta_0 - \beta_1 \cdot \sum_{i=1}^n x_i = 0$$

Jetzt kann man durch  $n$  teilen. Dabei entspricht  $\frac{\sum_{i=1}^n y_i}{n}$  dem arithmetischen Mittelwert von  $y$  und  $\frac{\sum_{i=1}^n x_i}{n}$  dem arithmetischen Mittelwert von  $x$ . Somit steht:

$$\bar{y} - \beta_0 - \beta_1 \cdot \bar{x} = 0$$

Umgestellt:

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

### 3.6 partielle Ableitung nach dem Anstieg

Für die partielle Ableitung nach  $\beta_1$  ist ebenfalls die Kettenregel anzuwenden, sodass die äußere Funktion (oben) identisch abgeleitet wird:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_1} = 2 \cdot (y_1 - (\beta_0 + \beta_1 \cdot x_1)) + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n)) = 2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) \cdot (0 - (0 + 1 \cdot x_i))$$

Für die partielle Ableitung nach  $\beta_1$  gilt also:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_1} = -2 \sum_{i=1}^n x_i \cdot (y_i - (\beta_0 + \beta_1 \cdot x_i)) = 0$$



Auch diese kann vereinfacht werden, indem der Vorfaktor  $-2$  entfällt (weil  $-2 \cdot 0 = 0$  gelten muss) und die Vorzeichen aufgelöst werden. Außerdem kann ausmultipliziert werden:

$$\sum_{i=1}^n x_i y_i - \sum_{i=1}^n \beta_0 \cdot x_i - \sum_{i=1}^n \beta_1 \cdot x_i x_i = 0$$

Wieder können die Konstanten herausgezogen werden:

$$\sum_{i=1}^n x_i y_i - \beta_0 \cdot \sum_{i=1}^n x_i - \beta_1 \cdot \sum_{i=1}^n x_i x_i = 0$$

Jetzt kann man  $\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$  und  $\sum_{i=1}^n x_i = n \cdot \bar{x}$  einsetzen:

$$\sum_{i=1}^n x_i y_i - (\bar{y} - \beta_1 \cdot \bar{x}) \cdot n \cdot \bar{x} - \beta_1 \cdot \sum_{i=1}^n x_i x_i = 0$$

Der mittlere Term wird ausmultipliziert und  $x_i x_i$  im letzten Term als  $x_i^2$  geschrieben:

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} - \beta_1 \cdot n \bar{x} \bar{x} - \beta_1 \cdot \sum_{i=1}^n x_i^2 = 0$$

Die letzten beiden Terme werden unter Anwendung des Distributivgesetzes  $a - b = -(b - a)$  zusammengefasst.

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} - \beta_1 \cdot (\sum_{i=1}^n x_i^2 - n \bar{x}^2) = 0$$

Jetzt kann nach  $\beta_1$  umgestellt werden. Erst:

$$\beta_1 \cdot (\sum_{i=1}^n x_i^2 - n \bar{x}^2) = \sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}$$

Dann:

$$\beta_1 = \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n x_i^2 - n \bar{x}^2}$$

Nun kann zuerst mit  $\sum_{i=1}^n x_i^2 - n \bar{x}^2 = \sum_{i=1}^n (x_i - \bar{x})^2$  umgeformt werden.

$$\beta_1 = \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Dann - und das wird gleich gezeigt - mit  $\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$ . Sodass steht:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Der letzte Schritt wird ausgehend vom Ergebnis gezeigt und beginnt mit dem Ausmultiplizieren:

$$\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \sum_{i=1}^n (x_i y_i - x_i \bar{y} - \bar{x} y_i + \bar{x} \bar{y})$$

Man kann auch schreiben:

$$\sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \bar{y} - \sum_{i=1}^n \bar{x} y_i + \sum_{i=1}^n \bar{x} \bar{y}$$

$\bar{x}$  und  $\bar{y}$  sind Konstanten, sodass  $\bar{x} \cdot \sum_{i=1}^n y_i$  und  $\bar{y} \cdot \sum_{i=1}^n x_i$  geschrieben werden kann.  $\sum_{i=1}^n x_i$  ist gleich  $n \cdot \bar{x}$  (analog für  $y$ ). So ergibt sich:

$$\sum_{i=1}^n x_i y_i - \bar{y} \cdot n \cdot \bar{x} - \bar{x} \cdot n \cdot \bar{y} + \sum_{i=1}^n \bar{x} \bar{y}$$

Sortieren:

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} - n \bar{x} \bar{y} + \sum_{i=1}^n \bar{x} \bar{y}$$

Der letzte Term  $\sum_{i=1}^n \bar{x} \bar{y}$  kann auch  $n \cdot \bar{x} \bar{y}$  geschrieben werden, sodass sich ergibt:

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} - n \bar{x} \bar{y} + n \bar{x} \bar{y}$$

Die letzten beiden Terme entfallen somit und es bleibt:

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}$$

(Baitsch, Matthias [2019](#), S. 73–74)

## NumPy polyfit und polyeval

```
import numpy.polynomial.polynomial as poly
```

**i** Hinweis 7: polyfit und polyeval erklärt

```
# Beispieldaten erzeugen
x = np.array(list(range(0, 100)))
y = x ** 2

print(poly.polyfit(x, y, 1))
```

```
[-1617.    99.]
```

Die Funktion gibt die geschätzten Regressionsparameter als NumPy-Array zurück. Die Terme sind aufsteigend angeordnet, d. h. der Achsabschnitt steht an Indexposition 0, der Steigungskoeffizient an Indexposition 1. Die Ausgabe für ein Polynom zweiten Grades würde beispielsweise so aussehen:

```
print(poly.polyfit(x, y, 2).round(2))
```

```
[ 0. -0.  1.]
```

Mit den Regressionskoeffizienten können die Vorhersagewerte der linearen Funktion berechnet werden. Dafür wird die Funktion `poly.polyeval(x, c)` verwendet. Diese berechnet die Funktionswerte für in `x` übergebene Werte mit den Funktionsparametern `c`. Aus der Differenz der gemessenen Werte und der Vorhersagewerte können die Residuen bestimmt werden.

```
# 'manuelle' Berechnung
regressions_koeffizienten = poly.polyfit(x, y, 1)
vorhersagewerte = regressions_koeffizienten[0] + x * regressions_koeffizienten[1]
residuen = y - vorhersagewerte
```

```
# Berechnung mit polyeval
lm = poly.polyfit(x, y, 1)
vorhersagewerte_polyval = poly.polyval(x, lm)
```

```
print("Die Ergebnisse stimmen überein:", np.equal(vorhersagewerte, vorhersagewerte_polyval))
print("\nAusschnitt der Vorhersagewerte:", vorhersagewerte[:10])
```

```
Die Ergebnisse stimmen überein: True
```

```
Ausschnitt der Vorhersagewerte: [-1617. -1518. -1419. -1320. -1221. -1122. -1023. -924. -825. -726.]
```

Das **Bestimmtheitsmaß**  $R^2$  gibt an, wie gut die Schätzfunktion an die Daten angepasst ist. Der Wertebereich reicht von 0 bis 1. Ein Wert von 1 bedeutet eine vollständige Anpassung. Für eine einfache lineare Regression mit nur einer erklärenden Variable kann das Bestimmtheitsmaß als Quadrat des **Bravais-Pearson-Korrelationskoeffizienten**  $r$  berechnet werden. Dieser wird mit der Funktion `np.corrcoef(x, y)` ermittelt (die eine Matrix der Korrelationskoeffizienten ausgibt).

```
print(f"r = {np.corrcoef(x, y)[0, 1]:.2f}")
print(f"R\u00b2 = {np.corrcoef(x, y)[0, 1] ** 2:.2f}")
```

```
r = 0.97
R2 = 0.94
```

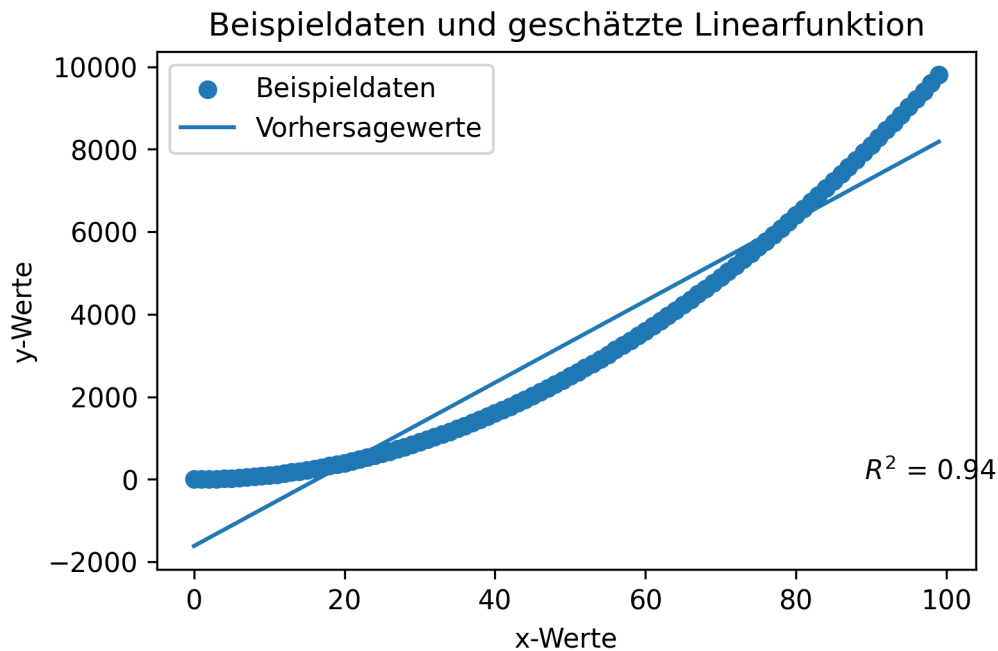
Die Daten und die geschätzte Gerade können grafisch dargestellt werden.

```
import matplotlib.pyplot as plt

plt.scatter(x, y, label = 'Beispieldaten')
plt.plot(x, vorhersagewerte, label = 'Vorhersagewerte')
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

plt.title(label = 'Beispieldaten und geschätzte Linearfunktion')
plt.xlabel('x-Werte')
plt.ylabel('y-Werte')
plt.legend()

plt.show()
```



#### **i** Hinweis 8: np.polyfit & np.polyval

Die Funktionen `np.polyfit(x, y, deg)` und `np.polyval(p, x)` funktionieren wie die vorgestellten Funktionen aus dem Modul `numpy.polynomial.polynomial`. Ein wichtiger Unterschied besteht jedoch darin, dass **die Parameter der Funktion `polyfit` in umgekehrter Reihenfolge** ausgegeben werden.

```
print(poly.polyfit(x, y, deg = 1))
print(np.polyfit(x, y, deg = 1))
```

```
[-1617.    99.]
[   99. -1617.]
```

#### **i** Hinweis

This forms part of the old polynomial API. Since version 1.4, the new polynomial API defined in `numpy.polynomial` is preferred. A summary of the differences can be found in the [transition guide](#).

[NumPy-Dokumentation](#)

```

print(poly.polyfit(hooke['distance'], hooke['force'], 1))

print(f"r = {np.corrcoef(hooke['distance'], hooke['force'])[0, 1]:.2f}")
print(f"R\u00b2 = {np.corrcoef(hooke['distance'], hooke['force'])[0, 1] ** 2:.2f}")

```

```

# Berechnung mit polyeval
lm = poly.polyfit(hooke['distance'], hooke['force'], 1)
vorhersagewerte_hooke = poly.polyval(hooke['distance'], lm)

```

```

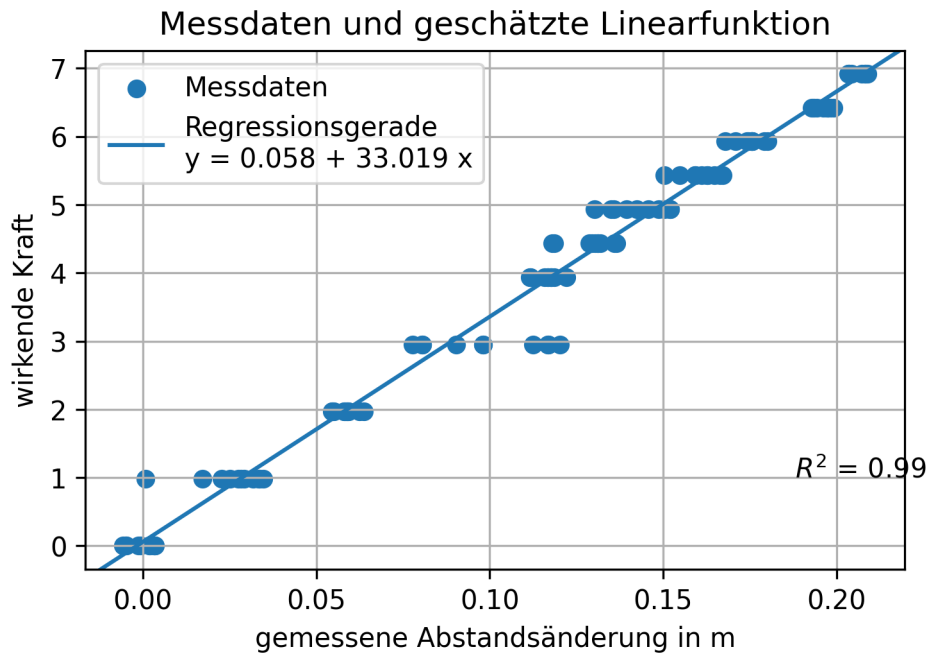
# Platzhalter x & y
x = hooke['distance']
y = hooke['force']

# Plot erstellen
plt.scatter(x, y, label = 'Messdaten')
plt.axline(xy1 = (0, lm[0]), slope = lm[1], label = 'Regressionsgerade\nty = ' + "{beta_0:.3f} + {beta_1:.3f}x")
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

plt.title(label = 'Messdaten und geschätzte Linearfunktion')
plt.xlabel('gemessene Abstandsänderung in m')
plt.ylabel('wirkende Kraft')
plt.legend()

plt.grid()
plt.show()

```



### 3.6.1 Messabweichung quantifizieren

[Wikipedia](#)

- [Residuenquadratsumme](#)
- [Summe der Abweichungsquadrate von  \$x\$](#)
- [Summe der Abweichungsquadrate von  \$y\$](#)

—

```

print(f"Regressionskoeffizient: {lm[1]:.4f}")

# 'manuell' Standardfehler des Regressionskoeffizienten berechnen
standardfehler_beta_1 = np.sqrt( (1 / (len(x) - 2) * sum((y - vorhersagewerte_hooke) ** 2))

print(f"Standardfehler des Regressionskoeffizienten: {standardfehler_beta_1:.4f}")

# Signifikanzniveau (alpha-Niveau) 1 - 95 % wählen
alpha = 0.05
n = len(x)

t_wert = scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 2)
print(f"t-Wert 95%-Intervall (zweiseitig): {t_wert:.4f}")
print(f"Konfidenzintervall 95 %: {lm[1]:.4f} ± {t_wert:.4f} * {standardfehler_beta_1:.4f}")
print(f"untere 95%-Intervallgrenze: {lm[1] - t_wert * standardfehler_beta_1:.4f}")
print(f"obere 95%-Intervallgrenze: {lm[1] + t_wert * standardfehler_beta_1:.4f}")

```

```

# Platzhalter x & y
x = hooke['distance']
y = hooke['force']

# Plot erstellen
plt.scatter(x, y, label = 'Messdaten')
plt.axline(xy1 = (0, lm[0]), slope = lm[1], label = 'Regressionsgerade\
ny = ' + "{beta_0:.3f} + {beta_1:.3f} * x")
plt.annotate("$R^2$ = {:.2f}".format(np.corrcoef(x, y)[0, 1] ** 2), (max(x) * 0.9, 1))

# 95%-Konfidenzintervall einzeichnen
## poly.polyval(hooke['distance'], [lm[0]])
beta1_lower_boundary = lm[1] - (t_wert * standardfehler_beta_1)
beta1_upper_boundary = lm[1] + (t_wert * standardfehler_beta_1)

y_lower_boundary = poly.polyval(hooke['distance'], [lm[0], beta1_lower_boundary])

```



```

y_upper_boundary = poly.polyval(hooke['distance'], [lm[0], beta1_upper_boundary])

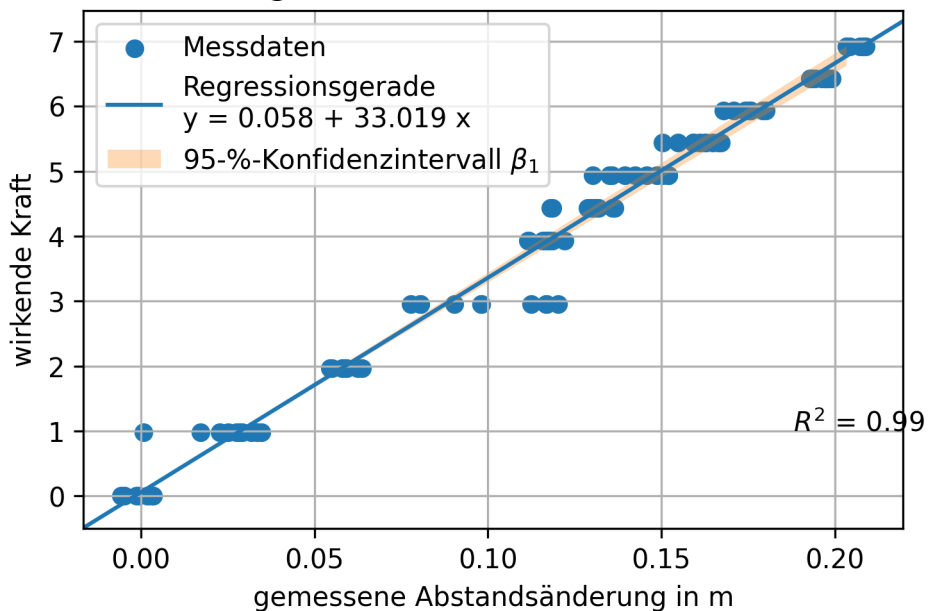
plt.fill_between(x = x, y1 = y_lower_boundary , y2 = y_upper_boundary, alpha = 0.3, label =

plt.title(label = 'Messdaten und geschätzte Linearfunktion im 95%-Intervall')
plt.xlabel('gemessene Abstandsänderung in m')
plt.ylabel('wirkende Kraft')
plt.legend()

plt.grid()
plt.show()

```

Messdaten und geschätzte Linearfunktion im 95%-Intervall



### 3.6.2 Das Modul SciPy

- 
- 
- 
-

- 
- 

```
# Zuweisung mehrerer Objekte
slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
      f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
      f"Standardfehler des Anstiegs: {slope_stderr:.4f}", sep = '')

# Zuweisung eines Objekts
lm = scipy.stats.linregress(x, y)

print("\n", lm, sep = '')
print(f"y-Achsenschnittpunkt: {lm.intercept:.4f}\nStandardfehler des y-Achsenschnittpunkts:{lm.slope_stderr:.4f}")
```

```
alpha = 0.05
n = len(x)

print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}    {slope + scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}")
```

### 3.6.3 Ergebnis Federkonstante

## 4 Fehlerrechnung

“Das Experimentieren ist ein dornenvoller Weg und so ganz sicher ist man sich nie.”  
Wagner, Paul. 2015. “Einführung in die Physik I. PH I - 04 - Die Messgenauigkeit / Fehlerrechnung Teil 1.” Universität Wien Physik, [YouTube](#), Zeitstempel 04:05

### 4.1 Bevor es losgeht

#### 4.1.1 Der Begriff des Fehlers

•  
•

### 4.1.2 Konfidenzniveau

2

3

“In der Physik und der Vermessungstechnik begnügt man sich mit einer statistischen Sicherheit von 68,3% und setzt deshalb  $t = 1$  [...] (In der Industrie wird  $t = 2$ , in der Biologie  $t = 3$  bevorzugt.)” (Hochschule Aalen [2016](#), S. 4)

1995

### 4.1.3 Notation

## 4.2 Fehlerarten

- 

- - 
  - 
  -

Quantifizierungsfehler

- 

- -

- 

- - 
  -

Einschwingzeit

2016

2024

- 

- 

Parallaxenfehler

- 

#### 💡 Tipp 4: Umgang mit verschiedenen Arten von Messabweichungen

1. Grobe Fehler: Betroffene Werte streichen und Messung wiederholen.
2. Systematische Messabweichungen: Systematische Messabweichungen werden, wenn sie bekannt sind, korrigiert.
3. Zufällige Messabweichungen: Die statistische Streuung von Messwerten um den Erwartungswert wird quantifiziert.

Einige systematische und zufällige Messabweichungen können nur mit hohem Aufwand reduziert werden (genauere Messgeräte, größere Anzahl von Messvorgängen). Der Aufwand muss gegen den Gewinn an Genauigkeit abgewogen werden: Ist das Messergebnis

mit der angegebenen Unsicherheit akzeptabel?  
(Hempel (2016), S. 2)

## 4.3 Grundbegriffe nach DIN 1319

- 
- 
- 

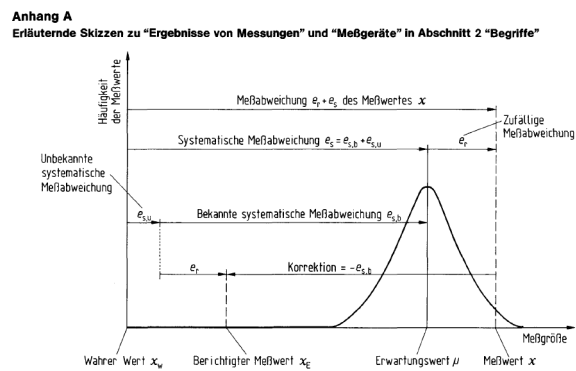


Abbildung 4.1: Messergebnis nach DIN 1319

1995

- 1.
- 2.
- 3.

#### 4.3.1 Messabweichung

! Wichtig 5: Messabweichung nach DIN 1319

Die Messabweichung ist die: “Abweichung eines aus Messungen gewonnenen und der Meßgröße [...] zugeordneten Wertes vom wahren Wert [...]. Ist  $m$  der der Meßgröße zugeordnete Wert und  $x_w$  der wahre Wert, so ist die Meßabweichung des zugeordneten Wertes  $m - x_w$  [...]” (Deutsches Institut für Normung [1995](#), S. 12)

- 
- 
- 

[1995](#)

#### 4.3.2 Messunsicherheit

! Wichtig 6: Messunsicherheit nach DIN 1319

“Kennwert, der aus Messungen [...] gewonnen wird und zusammen mit dem Meßergebnis [...] zur Kennzeichnung eines **Wertebereiches** für den wahren Wert der Meßgröße [...] dient.” (Deutsches Institut für Normung 1995, 14, eigene Hervorhebung)

Während die Messabweichung ein Einzelwert ist, kennzeichnet die Messunsicherheit einen Wertebereich: “Die Meßunsicherheit ist von der Meßabweichung [...] deutlich zu unterscheiden. Letztere ist nur die Differenz zwischen einem der Meßgröße zuzuordnenden Wert [...] und dem wahren Wert. Die Meßabweichung kann gleich Null sein, ohne daß dies bekannt ist. Dieses Unkenntnis drückt sich in einer Meßunsicherheit größer als Null aus.” (Deutsches Institut für Normung 1996, S. 3)



**i** Hinweis 9:  $u(e_s)$  nicht vernachlässigbar

Für den Fall, dass die Unsicherheit der systematischen Messabweichung nicht als vernachlässigbar angesehen wird, verweist die DIN 1319-3 auf Berechnungsregeln in Abschnitt 6.2.5 auf die wir hier nicht eingehen. (Deutsches Institut für Normung (1996), S. 5-6)  
Die Messunsicherheit des berichtigten Messergebnisses berechnet sich aus der quadratischen Kombination der Unsicherheiten der zufälligen und der systematischen Messabweichung.

$$u(y) = \sqrt{u^2(\bar{x}) + u^2(e_s)}$$

(Deutsches Institut für Normung 1996, Gleichung (8), S. 6, Notation angepasst)

**⚠ Warning 2: Notation in der DIN 1319**

In der DIN wird die zufällige Messabweichung statt mit  $u^2(\bar{x})$  mit  $u^2(x_1)$  und die Unsicherheit der systematischen Messabweichung statt mit  $u^2(e_s)$  mit  $u^2(x_2)$  notiert.

### 4.3.3 Vollständiges Messergebnis

1996

1996

2

1996

1995

1996

#### **Warning 3: Notation in the DIN 1319**

In der DIN wird der arithmetische Mittelwert einer Messreihe statt mit  $\bar{x}$  mit  $\bar{v}$  notiert.

### 4.3.4 Übersicht

#### **Wichtig 7: Grundbegriffe nach DIN 1319**

**Messabweichung:** “Abweichung eines aus Messungen gewonnenen und der Meßgröße [...] zugeordneten Wertes vom wahren Wert [...]. Ist  $m$  der der Meßgröße zugeordnete Wert und  $x_w$  der wahre Wert, so ist die Meßabweichung des zugeordneten Wertes  $m - x_w$  [...]” (Deutsches Institut für Normung 1995, S. 12)

- Die Messabweichung des **unberichtigten Messergebnisses** setzt sich additiv aus der zufälligen Messabweichung  $e_r$  und der systematischen Messabweichung  $e_s$  zusammen.
- Die systematische Messabweichung  $e_s$  setzt sich aus einem bekannten  $e_{s,b}$  und einem unbekannten Anteil  $e_{s,u}$  zusammen.
- Die systematische Messabweichung ist die Differenz aus Erwartungswert  $\mu$  und wahren Wert  $x_w$ .  $e_s = \mu - x_w$
- Die bekannte, systematische Messabweichung  $e_{s,b}$  wird mit entgegengesetzten Vorzeichen  $-e_{s,b}$  *Korrektur* genannt.

Das **Messergebnis**  $\bar{x}_E$  ist das um die bekannte, systematische Messabweichung  $e_{s,b}$

berichtigte arithmetische Mittel einer Messreihe  $\bar{x}$ , das als Schätzer des Erwartungswerts  $\mu$  verwendet wird.

$$\bar{x}_E = \bar{x} - e_{s,b}$$

(Deutsches Institut für Normung 1995, S. 11–13)

**Messunsicherheit**  $u$  oder  $u(y)$ : “Kennwert, der aus Messungen [...] gewonnen wird und zusammen mit dem Meßergebnis [...] zur Kennzeichnung eines Wertebereichs für den wahren Wert der Meßgröße [...] dient.” (Deutsches Institut für Normung 1995, S. 14)

- **Standardmessunsicherheit** oder kurz Standardunsicherheit wird verwendet, wenn die Messunsicherheit durch eine Standardabweichung ausgedrückt wird. (Deutsches Institut für Normung 1996, S. 3)
- **erweiterte Messunsicherheit**  $U(y) = k \cdot u(y)$ : Wertebereich, der den wahren Wert der Messgröße wahrscheinlich enthält. Der Erweiterungsfaktor  $k$  sollte zwischen 2 und 3 festgelegt werden, vorzugsweise wird  $k = 2$  verwendet. Der Erweiterungsfaktor  $k$  ist mitzuteilen, damit die Standardmessunsicherheit  $u(y) = U(y)/k$  ermittelt werden kann. (Deutsches Institut für Normung 1996, S. 7)

*Hinweis: Die erweiterte Messunsicherheit unterscheidet sich vom Konfidenzintervall dadurch, dass keine Normalverteilung unterstellt wird - es ist schlicht ein Faktor, ‘um auf Nummer sicher zu gehen’, dass der wahre Wert vom angegebenen Bereich eingeschlossen wird.*

Das **vollständige Messergebnis** ist das Messergebnis mit quantitativen Angaben zur Messunsicherheit  $u$ .  $x = \bar{x}_E \pm u$ . (Deutsches Institut für Normung 1995, S. 16)

- Der **Vertrauensbereich**  $\bar{x} - t \cdot u(y) \leq Y \leq \bar{x} + t \cdot u(y)$  kann zusätzlich zum vollständigen Messergebnis angegeben werden.
  - Ist die systematische Messabweichung bekannt und korrigiert worden, dann liegt der wahre Wert der Messgröße  $Y$  innerhalb des Vertrauensbereichs.
  - Wenn die systematische Messabweichung nicht bekannt ist, liegt der Erwartungswert  $\mu$  der Verteilung im Vertrauensbereich. (Deutsches Institut für Normung 1996, S. 7)

*Hinweis: In der DIN wird der arithmetische Mittelwert einer Messreihe statt mit  $\bar{x}$  mit  $\bar{v}$  geschrieben.*

## 4.4 Absolute und relative Fehler

- 

- 

1995

1996

#### Warnung 4: (Alternative) Notationen

In der DIN 1319 werden Messabweichungen mit  $e$ , die Messunsicherheit mit  $u$  notiert. In der Literatur zur Fehlerrechnung wird zumeist der Begriff des (Mess-)Fehlers verwendet. Die Notation ist allerdings uneinheitlich.

- Der absolute Fehler wird häufig mit  $\epsilon x$  (epsilon x) notiert. Üblich ist aber auch die Notation mit  $\Delta x$  (Delta), (beispielsweise (Hochschule Aalen [2016](#); Dinter, Ralf [2011](#); Schrüfer, Reindl und Zagar [2022](#)).  $\Delta$  wird aber ebenfalls für den Größtfehler verwendet.
- Der relative Fehler wird auch mit  $\delta x$  (delta x) notiert, bspw:  $\delta x = 4\%$ .

## 4.5 Größtfehler

 Tipp 5: Größtfehler

Der Größtfehler ist keine genormte Angabe nach DIN 1319, sondern eine vereinfachte Form der Fehlerabschätzung und überschätzt die wahrscheinliche Messunsicherheit. “Für viele Betrachtungen, speziell in den Grundlagenpraktika der ersten Semester in technischen, naturwissenschaftlichen und ingenieurmäßigen Studiengängen, reicht es aus, den maximalen Fehler (*absoluter Größtfehler*) zu bestimmen.” (Eden und Gebhard (2024), S. 35)

## 4.6 Signifikante Stellen

 Wichtig 8: signifikante Stellen

“Jede zuverlässig bekannte Stelle mit Ausnahme der Nullen, die die Position des Dezimalkommata angeben, wird signifikante Stelle genannt.” (Eden und Gebhard 2024, S. 24)  
“Gerundete numerische Unsicherheitswerte sind **mit zwei (oder bei Bedarf mit drei) signifikanten Ziffern** anzugeben. **Sie sind aufzurunden.** Das Meßergebnis ist an derselben Stelle wie die zugehörige Unsicherheit zu runden, z.B.  $y = 245,5716\text{mm}$  auf  $y = 245,57\text{mm}$ , wenn  $u(y) = 0,4528\text{mm}$  auf  $u(y) = 0,46\text{mm}$  aufgerundet wird.” (Deutsches Institut für Normung 1996, 6, eigene Hervorhebung)

- 1.
- 2.
- 3.
- 4.

#### 💡 signifikante Stellen und wissenschaftliches Runden

Angaben zur Messunsicherheit werden zwar aufgerundet. Trotzdem an dieser Stelle: Kennen Sie den Unterschied zwischen kaufmännischem und wissenschaftlichem Runden?

<https://www.youtube.com/watch?v=s0gPyypGOs>

Runden und signifikante Stellen (Vorkurs Mathematik) von Edmund Weitz ist abrufbar auf [YouTube](#). 2019

Python rundet wissenschaftlich:

```
print(round(2.5), round(3.5))
```

2 4

## 4.7 Methoden der Fehlerrechnung

- 1.
- 2.
- 3.

2016

Normal

2022

- 1.

2.

- 
- 
- 

3.

#### **Warning 5: Formelzeichen und Einheiten**

Im Folgenden werden identische Zeichen für Sekunde und Strecke  $s$ , Masse und Meter  $m$ , Gramm und die Konstante  $g$  sowie  $\Delta$  für die Veränderung einer Größe und den Größtfehler verwendet. Die Formeln sollten also mit großer Aufmerksamkeit gelesen werden.

#### **4.7.1 1. Fehlerabschätzung**

- 
- 

- 

-

2016

### **Aufgabe Temperaturmessung**

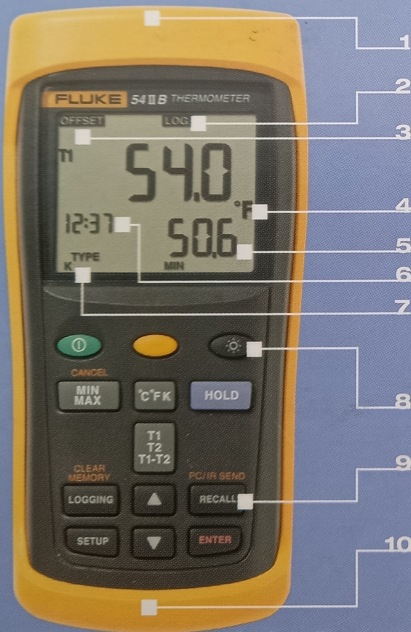




**FLUKE®**

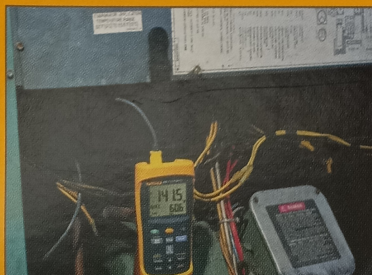
# Thermocouple Thermometer

**53/54 II B**



## Key features

- 1 Accepts magnetic hanger (optional TPAK accessory) for hands-free operation
- 2 Datalog up to 500 points to internal memory
- 3 Electronic offset compensates for thermocouple error
- 4 Selectable °C, °F, or Kelvin readout
- 5 Large dual display shows any combination of T1, T2 (54 II only), T1-T2 (54 II only), plus MIN, MAX, or AVG
- 6 Real time clock captures exact time of day when major events occur
- 7 Thermometer accepts seven different thermocouple types
- 8 Bright backlight turns on for two minutes with a touch of a button
- 9 Recall logged data on-screen or download via IR port to optional PC software
- 10 Rugged design easily withstands a 1 m (3 ft) drop



## Specifications

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <b>Accuracy</b>     |                                                       |
| Above -100 °C       | [0.05 % + 0.3 °C (0.5 °F)] for J, K, T, E and N types |
| Below -100 °C       | [0.05 % + 0.4 °C (0.7 °F)] for R and S types          |
|                     | [0.2 % + 0.3 °C] for J, K, E, and N types             |
|                     | [0.5 % + 0.3 °C] for T type                           |
| <b>Resolution</b>   | 0.1 °C / °F / K < 1000 and 1 °C / °F / K ≥ 1000       |
| <b>Battery Life</b> | 1000 hours typical                                    |
| <b>Warranty</b>     | Three-years                                           |

Abbildung 4.2: Thermometer

| Specifications      |                                                       |
|---------------------|-------------------------------------------------------|
| <b>Accuracy</b>     |                                                       |
| Above -100 °C       | [0.05 % + 0.3 °C (0.5 °F)] for J, K, T, E and N types |
| Below -100 °C       | [0.05 % + 0.4 °C (0.7 °F)] for R and S types          |
|                     | [0.2 % + 0.3 °C] for J, K, E, and N types             |
|                     | [0.5 % + 0.3 °C] for T type                           |
| <b>Resolution</b>   | 0.1 °C / °F / K < 1000 and 1 °C / °F / K ≥ 1000       |
| <b>Battery Life</b> | 1000 hours typical                                    |
| <b>Warranty</b>     | Three-years                                           |

Abbildung 4.3: Spezifikationen

#### 💡 Tipp 6: Halber oder ganzer Skalenstrich?

Messgeräte mit digitaler Anzeige machen es leicht: Die Ablesegenauigkeit beträgt eine Skaleneinheit. Bei Messgeräten mit analoger Anzeige ist es Ermessenssache, welche Messgenauigkeit Sie angeben können bzw. möchten.

Sind Sie sich absolut sicher, dass Sie zwischen “genau auf dem Strich” und “zwischen zwei Strichen” unterscheiden können, beträgt Ihr Ablesefehler einen halben Skalenstrich. Sind Sie sich nicht sicher, beträgt Ihr Ablesefehler einen ganzen Skalenstrich.

([Fehlerrechnung leicht gemacht, FSU Jena](#))

#### 💡 Tipp 7: Musterlösung Temperaturmessung

##### **systematische Messabweichung**

Die absolute Messabweichung beträgt  $u(T) = 0,3 \text{ °C}$ . Die relative Messabweichung beträgt 0,05 %. Das bedeutet:

$$\frac{u(T)}{112,1 \text{ °C}} \cdot 100 \% = 0,05 \%$$

$$\frac{u(T)}{112,1 \text{ °C}} \cdot 100 = 0,05$$

$$u(T) = 0,05 \cdot \frac{112,1 \text{ °C}}{100}$$

$$u(T) = 0,05605 \text{ °C}$$

Die bekannte systematische Messabweichung beträgt also:

$$e(T)_{s,b} = 0,3 \text{ °C} + 0,05605 \text{ °C} = 0,35605 \text{ °C}$$

##### **zufällige Messabweichung**

Die zufällige Messabweichung beträgt eine Skaleneinheit, also:

$$e(T)_r = 0,1 \text{ °C}$$

##### **Größtfehler**

Der Größtfehler beträgt:

$$\Delta T = |e(T)_{s,b}| + |e(T)_r|$$

Die Messunsicherheit ist auf zwei Stellen aufzurunden:

$$\Delta T = |0,35605^\circ\text{C}| + |0,1^\circ\text{C}| = 0,45605^\circ\text{C} \approx 0,46^\circ\text{C}$$

Das vollständige Messergebnis lautet also  $T = 112,1 \pm 0,46^\circ\text{C}$ .

## Beispiel Messbedingungen

- 
- 

### **i** Hinweis 11: Erster Messvorgang Münze

Die Messung wird mit einem **Messschieber mit Nonius** 0,05 mm nach DIN 862 durchgeführt. Die DIN 862 normiert symmetrische Fehlergrenzen von Messschiebern. Die Fehlergrenze ist abhängig von der Skalengenauigkeit des Messschiebers sowie von der gemessenen Länge und ist aus einer Tabelle abzulesen. (DIN 862 wurde 2011 durch DIN EN ISO 13385-1 ersetzt, in der der Begriff der Abweichung verwendet wird. Die DIN 862 ist aber auf dem Messschieber aufgedruckt.)

Messwert:  $x = 23,25\text{mm}$

- Die Skaleneinteilung ist 0,05 mm, die zufällige Ableseabweichung  $e_r$  beträgt also  $\pm 0,025\text{mm}$ .
- Die DIN 862 normiert Fehlergrenzen, die abhängig von der gemessenen Länge zunehmen, mindestens aber  $50\text{ }\mu\text{m}$  oder 0,05 mm betragen. Für den Messwert 23,25 mm gilt somit als bekannte systematische Messabweichung  $e_{s,b} = \pm 0,05\text{mm}$ .

Aus der Summe der Beträge ergibt sich der Größtfehler:  $\Delta x = |0,05|\text{mm} + |0,025|\text{mm} = 0,075\text{mm}$ .

Das vollständige Messergebnis, bestehend aus dem Messwert und der Messunsicherheit lautet also:  $23,25 \pm 0,075\text{mm}$ . Der relative Größtfehler beträgt:  $23,25\text{mm} \pm 0,32\%$ .

Der Daumen besteht

anatomisch aus nur 2 Fingergliedknochen

#### **i** Hinweis 12: Zweiter Messvorgang Daumen

Die Messung wird erneut mit dem Messschieber durchgeführt.

Messwert:  $x = 64,70\text{mm}$

- Die Skaleneinteilung ist  $0,05\text{ mm}$ , die zufällige Ableseabweichung beträgt also  $\pm 0,025\text{mm}$ .
- Die von der DIN 862 für gemessene Längen zwischen  $50$  und  $300\text{ mm}$  normierte Fehlergrenze beträgt  $50\text{ }\mu\text{m}$  oder  $0,05\text{ mm}$ . Für den Messwert  $64,7\text{ mm}$  gilt somit die Fehlergrenze  $\pm 0,05\text{mm}$ .
- zusätzlich wird eine symmetrische Messabweichung von  $3\text{ mm}$  veranschlagt, da
  - das Messgerät nicht gerade angesetzt,
  - die Daumenwurzel nicht exakt lokalisiert und
  - die elastische Fingerkuppe nicht fixiert werden konnte.

Aus der Summe der Beträge ergibt sich der Größtfehler  $\Delta x = |0,025|\text{mm} + |0,05|\text{mm} + |3|\text{mm} = 3,075\text{mm} \approx 3,08\text{mm}$ .

Das Messergebnis, bestehend aus dem Messwert und dem Fehlerintervall lautet also:  $64,70 \pm 3,08\text{mm}$ . Der relative Größtfehler beträgt:  $64,70\text{mm} \pm 4,76\%$ .

#### **4.7.2 2. Fehlerstatistik**

- 
- 

- 1.
- 2.
- 3.

4.

5.

2016

### **i** Hinweis 13: Fehlerstatistik

Aus dem im vorherigen Kapitel behandelten Experiment wird die Messreihe für das Gewicht 503g ausgewählt.

```
dateipfad = "01-daten/hooke_data.csv"
hooke = pd.read_csv(filepath_or_buffer = dateipfad, sep = ';')
hooke.drop(index = 113, inplace = True) # Fehlwert, siehe vorheriges Kapitel

hooke_503g = hooke.loc[hooke['mass'] == 503, :]
print(hooke_503g.head(), hooke_503g.describe(), sep = "\n\n")
```

|    | no | mass | distance |
|----|----|------|----------|
| 40 | 40 | 503  | 158.43   |
| 41 | 41 | 503  | 158.43   |
| 42 | 42 | 503  | 158.60   |
| 43 | 43 | 503  | 158.76   |
| 44 | 44 | 503  | 159.05   |

|       | no       | mass  | distance   |
|-------|----------|-------|------------|
| count | 10.00000 | 10.0  | 10.000000  |
| mean  | 44.50000 | 503.0 | 159.314000 |
| std   | 3.02765  | 0.0   | 0.781099   |
| min   | 40.00000 | 503.0 | 158.430000 |
| 25%   | 42.25000 | 503.0 | 158.640000 |
| 50%   | 44.50000 | 503.0 | 159.220000 |
| 75%   | 46.75000 | 503.0 | 159.965000 |
| max   | 49.00000 | 503.0 | 160.610000 |



Betrachten wir für diese Messreihe noch einmal das arithmetische Mittel (das unbereinigte Messergebnis) und den Standardfehler des Mittelwerts (das statistische Streuungsmaß des Mittelwerts). Die Methode `.sem()` berechnet den empirischen Standardfehler des Mittelwerts (standardmäßig ist der Parameter `ddof = 1` gesetzt.)

Ausgabe auf 3 Dezimalstellen genau.

```
print(f"Gewicht in Gramm: {hooke_503g['mass'].unique()}",
      f"Mittelwert der Messwerte: {hooke_503g['distance'].mean():.3f} cm",
      f"empirischer Standardfehler: {hooke_503g['distance'].sem():.3f} cm",
      sep = "\n")
```

Gewicht in Gramm: [503]

Mittelwert der Messwerte: 159.314 cm

empirischer Standardfehler: 0.247 cm

Die Messunsicherheit wird auf 2 Dezimalstellen genau aufgerundet. Dafür kann die NumPy-Funktion `np.ceil()` verwendet werden. Diese rundet allerdings zur nächsten Ganzzahl, ein Parameter, um auf eine bestimmte Dezimalstelle zu runden, ist nicht verfügbar. Deshalb muss die Dezimalstelle um die gewünschte Anzahl Stellen durch Multiplikation mit der entsprechenden Zehnerpotenz verschoben werden. Für 2 Dezimalstellen entsprechend um  $10^2$ : `np.ceil(wert * 100) / 100`.

```
wert = pd.Series([0.247])
print(np.ceil(wert * 100) / 100)

# mit Pandas-Methoden
print(wert.mul(100).apply(np.ceil).div(100))
```

```
0    0.25
dtype: float64
0    0.25
dtype: float64
```

Für die Verkettung mit weiteren Methoden in Pandas muss die Rückgabe der Funktion, ein NumPy-Skalar oder -Array, wieder in eine `pd.Series` umgewandelt werden.

```
print(f"Gewicht in Gramm: {hooke_503g['mass'].unique()}",
      f"Mittelwert der Messwerte: {hooke_503g['distance'].mean():.3f} cm",
      f"empirischer Standardfehler: { ( sem_hooke_503g:= pd.Series(hooke_503g['distance'].sem()) ):.3f} cm",
      f"vollständiges Messergebnis: {hooke_503g['distance'].mean().round(2):.2f} cm ± {sem_hooke_503g:.2f} cm",
      sep = "\n")
```

Gewicht in Gramm: [503]  
Mittelwert der Messwerte: 159.314 cm  
empirischer Standardfehler: 0.250 cm  
vollständiges Messergebnis: 159.31 cm  $\pm$  0.25 cm

Zusätzliche Angabe des 95-%-Konfidenzintervalls.

```
# t-Wert bestimmen für Signifikanzniveau (alpha-Niveau) 1 - 95 % wählen
alpha = 0.05
n = hooke_503g.shape[0] # Anzahl Zeilen
t_wert = scipy.stats.t.ppf(q = 1 - alpha/2, df = n - 1)
```

```
print(f"95-%-Konfidenzintervall : {hooke_503g['distance'].mean().round(2)} cm  $\pm$  {t_wert:.4} cm")
f"95-%-Konfidenzintervall : {hooke_503g['distance'].mean().round(2)} cm  $\pm$  {( t_wert * sem_503g).round(2)} cm"
f"untere 95-%-Intervallgrenze: {hooke_503g['distance'].mean().round(2) - round(t_wert * sem_503g, 2)} cm"
f"obere 95-%-Intervallgrenze: {hooke_503g['distance'].mean().round(2) + round(t_wert * sem_503g, 2)} cm"
sep = "\n")
```

```
95-%-Konfidenzintervall : 159.31 cm  $\pm$  2.2622 * 0.25 cm
95-%-Konfidenzintervall : 159.31 cm  $\pm$  0.57 cm
untere 95-%-Intervallgrenze: 158.74 cm
obere 95-%-Intervallgrenze: 159.88 cm
```

### 4.7.3 3. Fehlerfortpflanzung

- 
- 

- 
- 

2016



### **i** Hinweis 14: Taylorreihe

<https://www.youtube.com/watch?v=F2IKRaUguBM>

Die Taylorreihe - einfach erklärt von Denkbar ist abrufbar auf [YouTube](#). 2015

### **i** Hinweis 15: Partielle Ableitung

Die partielle Ableitung ist die Ableitung einer Funktion mit mehreren Variablen nach einer Variablen, wobei die übrigen Variablen als Konstanten behandelt werden.

Für eine Funktion  $f(x, y) = 2x + y^2$  wird die partielle Ableitung nach x so ausgedrückt:

$$\frac{\partial f(x,y)}{\partial x}$$

- Das Symbol  $\partial$  ist die kursive Darstellung des kyrillischen Kleinbuchstaben  $\partial$  (d) und wird als “del” gelesen. Es zeigt an, dass eine partielle Ableitung durchgeführt wird.
- Im Zähler steht die Funktion, die abgeleitet werden soll. Im Nenner steht die Variable nach der abgeleitet wird. Der Term wird gelesen als “del f von x und y nach del x”.

Die partielle Ableitung  $\frac{\partial f(x,y)}{\partial x} = 2$ .  $y^2$  wird als Konstante behandelt (z. B.  $5^2$ ) und ist abgeleitet Null.

Die partielle Ableitung  $\frac{\partial f(x,y)}{\partial y} = 2y$ .  $2x$  wird als Konstante behandelt (z. B.  $2 \cdot 3$ ) und ist abgeleitet Null.

2016

2024

#### **i** Hinweis 16: Beispiel Federkonstante

Beispielhaft betrachten wir die erste Messung aus der Messreihe mit dem Gewicht 503 Gramm.

```
hooke_503g_einzelwert = hooke_503g.iloc[0, :].copy()
print(hooke_503g_einzelwert)
```

```
no          40.00
mass        503.00
distance    158.43
Name: 40, dtype: float64
```

Da mehrere Größen betrachtet werden, werden die Messwerte in eine SI-Einheit umgerechnet. Als Nullpunkt wird die erste Messung der Messreihe mit dem Gewicht 0 Gramm aufgefasst.

```
hooke_503g_einzelwert['mass'] = hooke_503g_einzelwert['mass'] / 1000
hooke_503g_einzelwert['distance'] = hooke_503g_einzelwert['distance'] / 100
```

```
nullpunkt = hooke[hooke['mass'] == 0].loc[:, 'distance'].head(n = 1) / 100
print(f"Nullpunkt in Metern: {nullpunkt.item():.3f}\n")
```

```
hooke_503g_einzelwert['ausdehnung'] = (hooke_503g_einzelwert['distance'].item() - nullpunkt.item())
print(hooke_503g_einzelwert)
```

```
Nullpunkt in Metern: 1.733
```

```
no          40.0000
```

```

mass          0.5030
distance      1.5843
ausdehnung    0.1489
Name: 40, dtype: float64

```

Für die Federkonstante gilt (die Abstandsänderung  $\Delta x$  wird mit  $s$  notiert):

$$k = \frac{m \cdot g}{\Delta x} = \frac{m \cdot g}{s}$$

Einsetzen der Werte ( $s$  steht nun für Sekunde,  $m$  für Meter):

$$k = \frac{0,503kg \cdot 9,81m}{0,1489m \cdot s^2} = \frac{0,503 \cdot 9,81}{0,1489} \frac{N}{m} = 33.139 \frac{N}{m}$$

So gilt für den Größtfehler der Funktion:

$$\Delta k(m, s) = \left| \frac{\partial f(m, s)}{\partial m} \right| \cdot \Delta m + \left| \frac{\partial f(m, s)}{\partial s} \right| \cdot \Delta s$$

Im ersten Schritt werden die partiellen Ableitungen der Funktion  $k = \frac{m \cdot g}{s}$  gebildet. Die partielle Ableitung nach  $m$  lautet mit Faktorregel:

$$\frac{\partial f(m, s)}{\partial m} = \frac{\partial}{\partial m} \frac{m \cdot g}{s} = \frac{\partial}{\partial m} m \cdot \frac{g}{s} = 1 \cdot \frac{g}{s} = \frac{g}{s}$$

Die partielle Ableitung nach  $s$  lautet:

$$\frac{\partial f(m, s)}{\partial s} = \frac{\partial}{\partial s} \frac{m \cdot g}{s} = -\frac{m \cdot g}{s^2}$$

Der Größtfehler ergibt sich also durch:

$$\Delta k = \left| \frac{g}{s} \right| \cdot \Delta m + \left| (-) \frac{m \cdot g}{s^2} \right| \cdot \Delta s$$

Im zweiten Schritt wird der Größtfehler der beteiligten Größen ermittelt.

- Die Masse  $m$  der verwendeten Gewichte wurde mit einer Küchenwaage ermittelt, deren Ablesabweichung auf  $e_r(m) = 0,5g = 0,0005kg$  geschätzt wird. Eine systematische Messabweichung ist nicht bekannt.  
Der Größtfehler der Masse beträgt also  $\Delta m = 0,0005kg$ .
- Der Größtfehler der Abstandsmessung beträgt  $e_r(s) = 0,01cm + 0,3cm = 0,0031m$ .

Die Messwerte, die Konstante  $g$  und die Größtfehler werden eingesetzt.

$$\Delta k = \left| \frac{9,81m}{s^2 \cdot 0,1489m} \right| \cdot 0,0005kg + \left| (-) \frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1489m)^2} \right| \cdot 0,0031m$$

Trennen von Werten und Einheiten:

$$\Delta k = \left| \frac{9,81 \cdot 0,0005}{0,1489} \frac{m \cdot kg}{s^2 \cdot m} \right| + \left| (-) \frac{0,503 \cdot 9,81 \cdot 0,0031}{0,02217121} \frac{kg \cdot m^2}{s^2 \cdot m^2} \right|$$

Berechnen und Newton  $\frac{kg \cdot m}{s^2}$  einsetzen:

$$\Delta k = \left| 0,0329 \frac{N}{m} \right| + \left| (-) 0,6899 \frac{N \cdot m}{m^2} \right|$$

Meter im zweiten Term kürzen:

$$\Delta k = \left| 0,0329 \frac{N}{m} \right| + \left| (-) 0,6899 \frac{N}{m} \right|$$

Die Messunsicherheit wird auf zwei Dezimalstellen aufgerundet.

$$\Delta k = 0,7228 \frac{N}{m} \approx 0,73 \frac{N}{m}$$

Das vollständige Messergebnis lautet somit:

$$k = 33,14 \frac{N}{m}; \Delta k = 0,73 \frac{N}{m}$$

### Gauß'sche Fehlerfortpflanzung

**i** Hinweis 17: Beispiel Messreihe 503 Gramm

Betrachten wir noch einmal die Messreihe für das Gewicht 503 Gramm. Es soll das vereinbarte Konfidenzniveau 95 % gelten.

```

# Messdaten aus dem Kapitel hooke.qmd wiederherstellen
## Datei einlesen und Fehler entfernen
dateipfad = "01-daten/hooke_data.csv"
hooke = pd.read_csv(filepath_or_buffer = dateipfad, sep = ';')
hooke.drop(index = 113, inplace = True)

## Abstandsmessung auf Nullpunkt normieren
nullpunkt = hooke[hooke['mass'] == 0].loc[:, 'distance'].mean()
hooke['ausdehnung'] = hooke['distance'].sub(nullpunkt).mul(-1)

# Messreihe 503 Gramm auswählen
hooke_503g = hooke.loc[hooke['mass'] == 503, :].copy()

## Masse in kg, Abstand und Ausdehnung in m umrechnen
hooke_503g['mass'] = hooke_503g['mass'].div(1000)
hooke_503g['distance'] = hooke_503g['distance'].div(100)
hooke_503g['ausdehnung'] = hooke_503g['ausdehnung'].div(100)

# t-Wert bestimmen für Signifikanzniveau (alpha-Niveau) 1 - 95 % wählen
alpha = 0.05
n_ausdehnung = hooke_503g.shape[0] # Anzahl Zeilen
t_wert_ausdehnung = scipy.stats.t.ppf(q = 1 - alpha/2, df = n_ausdehnung - 1)

# Ausgabe
print(f"Gewicht in Kilogramm: {hooke_503g['mass'].unique()}",
      f"Mittlere Ausdehnung: {hooke_503g['ausdehnung'].mean():.4f} m",
      f"empirischer Standardfehler: { (sem_hooke_503g:= pd.Series(hooke_503g['ausdehnung']).sem):.4f} m",
      f"t-Wert der Ausdehnung: {t_wert_ausdehnung:.4f}",
      f"95%-Intervallgrenzen: {(hooke_503g['ausdehnung'].mean() - t_wert_ausdehnung * sem_hooke_503g, hooke_503g['ausdehnung'].mean() + t_wert_ausdehnung * sem_hooke_503g):.4f} m",
      sep = "\n")

```

```

Gewicht in Kilogramm: [0.503]
Mittlere Ausdehnung: 0.1432 m
empirischer Standardfehler: 0.002470 m
t-Wert der Ausdehnung: 2.2622
95%-Intervallgrenzen: 0.1376 m  0.1432 m  0.1488 m

```

Für die Federkonstante gilt (die Abstandsänderung  $\Delta x$  wird mit  $s$  notiert):

$$k = \frac{m \cdot g}{\Delta x} = \frac{m \cdot g}{s}$$

$$k = \frac{0,503 \text{ kg} \cdot 9,81 \text{ m}}{0,1432 \text{ m} \cdot \text{s}^2} = \frac{0,503 \cdot 9,81}{0,1432} \frac{\text{N}}{\text{m}} = 34,458 \frac{\text{N}}{\text{m}}$$

Um die Unsicherheit der Ergebnisgröße  $k$  nach der Gauß'schen Fehlerfortpflanzung zu bestimmen, müsste die Masse des verwendeten Gewichts ebenfalls durch eine Messreihe bestimmt worden sein. (Wie Messreihen und Einzelmessungen kombiniert werden können, wird im nächsten Abschnitt gezeigt.) Deshalb wird angenommen, dass mehrere Messungen zur Ermittlung des Gewichts durchgeführt wurden. Die Messreihe sowie ihr Mittelwert und Standardfehler könnten wie folgt aussehen:

```
messreihe_503g = pd.Series([503 , 503, 504, 502, 503, 502, 503, 504])
messreihe_503g /= 1000

# t-Wert bestimmen für Signifikanzniveau (alpha-Niveau) 1 - 95 % wählen
alpha = 0.05
n_masse = messreihe_503g.shape[0] # Anzahl Elemente
t_wert_masse = scipy.stats.t.ppf(q = 1 - alpha/2, df = n_masse - 1)

# Ausgabe
print(f"arithmetisches Mittel der Masse: {messreihe_503g.mean()} kg",
      f"empirischer Standardfehler: { (sem_messreihe_503g:= pd.Series(messreihe_503g.sem()))",
      f"t-Wert der Masse: {t_wert_masse:.4f}",
      f"95%-Intervallgrenzen: {(messreihe_503g.mean() - t_wert_masse * sem_messreihe_503g",
      sep = "\n")
```

```
arithmetisches Mittel der Masse: 0.503 kg
empirischer Standardfehler: 0.000267 kg
t-Wert der Masse: 2.3646
95%-Intervallgrenzen: 0.5024 kg - 0.5036 kg
```

Die Gauß'sche Fehlerfortpflanzung wird ähnlich zur linearen Fehlerfortpflanzung berechnet.

$$u(k(m, s)) = \sqrt{\left(\frac{\partial f(m, s)}{\partial m} \cdot \sigma_m\right)^2 + \left(\frac{\partial f(m, s)}{\partial s} \cdot \sigma_s\right)^2}$$

Die partiellen Ableitungen werden eingesetzt.

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot \sigma_m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot \sigma_s\right)^2}$$

## 4.8 Vollständiges Messergebnis

Die Messwerte, die Konstante  $g$  und die Standardfehler der Mittelwerte werden eingesetzt.

$$u(k(m, s)) = \sqrt{\left(\frac{9,81m}{s^2 \cdot 0,1432m} \cdot 0,000267kg\right)^2 + \left((-)\frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1432m)^2} \cdot 0,00247m\right)^2}$$

Trennen von Werten und Einheiten:

$$u(k(m, s)) = \sqrt{\left(\frac{9,81 \cdot 0,000267}{0,1432} \frac{m \cdot kg}{s^2 \cdot m}\right)^2 + \left((-)\frac{0,503 \cdot 9,81 \cdot 0,00247}{0,02051} \frac{kg \cdot m^2}{s^2 \cdot m^2}\right)^2}$$

Berechnen, Newton  $\frac{kg \cdot m}{s^2}$  einsetzen und Meter im 2. Term kürzen:

$$u(k(m, s)) = \sqrt{\left(\frac{9,81 \cdot 0,000267}{0,1432} \frac{N}{m}\right)^2 + \left((-)\frac{0,503 \cdot 9,81 \cdot 0,00247}{0,02051} \frac{N}{m}\right)^2}$$

Zusammenfassen:

$$u(k(m, s)) = \sqrt{\left(0,0183 \frac{N}{m}\right)^2 + \left((-)0,594 \frac{N}{m}\right)^2}$$

Ausrechnen:

$$u(k(m, s)) = 0,595 \frac{N}{m} \approx 0,60 \frac{N}{m}$$

Das vollständige Messergebnis lautet somit:

$$k = 34,46 \frac{N}{m} \pm 0,60 \frac{N}{m}$$

## 4.9 95%-Vertrauensintervall

Zusätzlich zum vollständigen Messergebnis kann ein Vertrauensintervall angegeben werden. Entsprechend der vereinbarten Vertrauenswahrscheinlichkeit, sind die entsprechenden t-Werte als Korrekturfaktoren einzusetzen.

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot t_{\alpha/2} \cdot \sigma_m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s\right)^2}$$

Dies sind für die Masse  $t = 2,3646$  und für die Ausdehnung  $t = 2,2622$ .

Die Messwerte, die Konstante  $g$ , der Standardfehler des Mittelwerts und der t-Wert werden in den ersten Term eingesetzt.

$$\left(\frac{g}{s} \cdot t_{\alpha/2} \cdot \sigma_m\right)^2 = \left(\frac{9,81m}{s^2 \cdot 0,1432m} \cdot 2,3646 \cdot 0,000267kg\right)^2$$



Trennen von Werten und Einheiten, Newton  $\frac{kg \cdot m}{s^2}$  einsetzen:

$$\left( \frac{9,81 \cdot 2,3646 \cdot 0,000267}{0,1432} \frac{m \cdot kg}{s^2 \cdot m} \right)^2 = \left( 0,0433 \frac{N}{m} \right)^2$$

Die Messwerte, die Konstante  $g$ , der Standardfehler des Mittelwerts und der t-Wert werden in den zweiten Term eingesetzt.

$$\left( -\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s \right)^2 = \left( (-) \frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1432m)^2} \cdot 2,2622 \cdot 0,00247m \right)^2$$

Trennen von Werten und Einheiten, Newton  $\frac{kg \cdot m}{s^2}$  einsetzen und  $m$  kürzen:

$$\left( (-) \frac{0,503 \cdot 9,81 \cdot 2,2622 \cdot 0,00247}{0,02051} \frac{kg \cdot m^2}{s^2 \cdot m^2} \right)^2 = \left( (-) 1,344 \frac{N}{m} \right)^2$$

Die Formel lautet somit:

$$u(k(m, s)) = \sqrt{\left( 0,0433 \frac{N}{m} \right)^2 + \left( (-) 1,344 \frac{N}{m} \right)^2}$$

Ausrechnen:

$$u(k(m, s)) = 1,345 \frac{N}{m} \approx 1,35 \frac{N}{m}$$

Die Intervallgrenzen lauten somit:

$$k = 34,46 \frac{N}{m} \pm 1,35 \frac{N}{m}$$

Also:

$$k_{95\%} = 33,11 \frac{N}{m} \leq 34,46 \frac{N}{m} \leq 35,81 \frac{N}{m}$$

## Übersicht Fehlerfortpflanzung für Grundrechenarten und Potenzprodukte

Tabelle 3.1: Berechnung der Größtfehler für Potenzprodukte und Grundrechenarten

| $y = f(x_1, x_2)$       | Standardfehler                                                                                                                                    | Größtfehler                                                                            |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| $y = x_1 \pm x_2$       | $\sigma y = \pm \sqrt{\sigma x_1^2 + \sigma x_2^2}$                                                                                               | $\sigma y = \sigma x_1 + \sigma x_2$<br>absolute Fehler werden addiert                 |
| $y = x_1 \cdot x_2$     | $\sigma y = \pm xy \sqrt{\frac{(\sigma x_1)^2}{x_1^2} + \frac{(\sigma x_2)^2}{x_2^2}}$<br>$\delta y = \pm \sqrt{(\delta x_1)^2 + (\delta x_2)^2}$ | $\delta y = \delta x_1 + \delta x_2 = \frac{\delta x_1}{x_1} + \frac{\delta x_2}{x_2}$ |
| $y = \frac{m}{n}$       | $\sigma y = \pm xy \sqrt{\frac{(\sigma x_1)^2}{x_1^2} + \frac{(\sigma x_2)^2}{x_2^2}}$<br>$\delta y = \pm \sqrt{(\delta x_1)^2 + (\delta x_2)^2}$ | $\delta y = \delta x_1 + \delta x_2$                                                   |
| $y = x_1^n \cdot x_2^m$ | $\delta y = \pm \sqrt{(n \delta x_1)^2 + (m \delta x_2)^2}$                                                                                       | $\delta y =  n  \delta x_1 +  m  \delta x_2$                                           |

Abbildung 4.4: Fehlerfortpflanzung für Grundrechenarten und Potenzprodukte

2024

Spezialfall: Fehlerfortpflanzung mit Einzelwerten

- 1.
- 2.

2016

2016

1996

## Hinweis 18: Beispiel Federkonstante

```
# Ausgabe
print(f"Gewicht in Kilogramm: {hooke_503g['mass'].unique()}",
      f"Mittlere Ausdehnung: {hooke_503g['ausdehnung'].mean():.4f} m",
      f"empirischer Standardfehler: { (sem_hooke_503g:= pd.Series(hooke_503g['ausdehnung']).sem()):.4f} m",
      f"t-Wert der Ausdehnung: {t_wert_ausdehnung:.4f}",
      f"95%-Intervallgrenzen: {(hooke_503g['ausdehnung'].mean() - t_wert_ausdehnung * sem_hooke_503g,
                               hooke_503g['ausdehnung'].mean() + t_wert_ausdehnung * sem_hooke_503g)}",
      sep = "\n")
```

Gewicht in Kilogramm: [0.503]  
Mittlere Ausdehnung: 0.1432 m  
empirischer Standardfehler: 0.002470 m  
t-Wert der Ausdehnung: 2.2622  
95%-Intervallgrenzen: 0.1376 m   0.1432 m   0.1488 m

## 4.10 Lineare Fehlerfortpflanzung

Analog zur Fehlerabschätzung ergibt sich der Größtfehler der Federkonstante durch:

$$\Delta k = \left| \frac{g}{s} \right| \cdot \Delta m + \left| (-) \frac{m \cdot g}{s^2} \right| \cdot \Delta s$$

- Die Ablesabweichung der Küchenwaage wird auf  $e_r(m) = 0,5g = 0,0005kg$  geschätzt. Eine systematische Messabweichung ist nicht bekannt. Der Größtfehler der Masse beträgt also  $\Delta m = 0,0005kg$
- Der empirische Standardfehler der Ausdehnung beträgt  $\sigma_x = 0.00247m$  bzw. zum vereinbarten Vertrauensniveau von  $\sigma_{x95\%} = 0.00247m \cdot 2.2622 = 0,00559m$

Die Messwerte, die Konstante  $g$  und die Größtfehler werden eingesetzt.

$$\Delta k = \left| \frac{9,81m}{s^2 \cdot 0,1432m} \right| \cdot 0,0005kg + \left| (-) \frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1432m)^2} \right| \cdot 0,00247m$$

Trennen von Werten und Einheiten:

$$\Delta k = \left| \frac{9,81 \cdot 0,0005}{0,1432} \frac{m \cdot kg}{s^2 \cdot m} \right| + \left| (-) \frac{0,503 \cdot 9,81 \cdot 0,00247}{0,020506} \frac{kg \cdot m^2}{s^2 \cdot m^2} \right|$$

Berechnen und Newton  $\frac{kg \cdot m}{s^2}$  einsetzen:

$$\Delta k = \left| 0,0342 \frac{N}{m} \right| + \left| (-) 0,594 \frac{N \cdot m}{m^2} \right|$$

Meter im zweiten Term kürzen:

$$\Delta k = |0,0342 \frac{N}{m}| + |(-)0,594 \frac{N}{m}|$$

Die Messunsicherheit wird auf zwei Dezimalstellen aufgerundet.

$$\Delta k = 0,628 \frac{N}{m} \approx 0,63 \frac{N}{m}$$

Das Messergebnis und der Größtfehler lauten somit:

$$k = 34,46 \frac{N}{m}; \Delta k = 0,63 \frac{N}{m}$$

## 4.11 Gauß'sche Fehlerfortpflanzung

Die Unsicherheit der Federkonstante berechnet sich durch:

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot \Delta m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot \sigma_s\right)^2}$$

Die Messwerte, die Konstante  $g$  und der abgeschätzte Fehleranteil (Größtfehler) der Masse werden in den ersten Term eingesetzt.

$$\left(\frac{g}{s} \cdot \Delta m\right)^2 = \left(\frac{9,81m}{s^2 \cdot 0,1432m} \cdot 0,0005kg\right)^2$$

Trennen von Werten und Einheiten, Newton  $\frac{kg \cdot m}{s^2}$  einsetzen:

$$\left(\frac{9,81 \cdot 0,0005}{0,1432} \frac{m \cdot kg}{s^2 \cdot m}\right)^2 = \left(0,0342 \frac{N}{m}\right)^2$$

Die Masse des Gewichts, die Konstante  $g$  und der Standardfehler des Mittelwerts der Ausdehnung werden in den zweiten Term eingesetzt.

$$\left(-\frac{m \cdot g}{s^2} \cdot \sigma_s\right)^2 = \left((-) \frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1432m)^2} \cdot 0,00247m\right)^2$$

Trennen von Werten und Einheiten, Newton  $\frac{kg \cdot m}{s^2}$  einsetzen und  $m$  kürzen:

$$\left((-) \frac{0,503 \cdot 9,81 \cdot 0,00247}{0,02051} \frac{kg \cdot m^2}{s^2 \cdot m^2}\right)^2 = \left((-)0,594 \frac{N}{m}\right)^2$$

Die Formel lautet somit:

$$u(k(m, s)) = \sqrt{\left(0,0342 \frac{N}{m}\right)^2 + \left((-)0,594 \frac{N}{m}\right)^2}$$

Ausrechnen:

$$u(k(m, s)) = 0,595 \frac{N}{m} \approx 0,60 \frac{N}{m}$$

Das vollständige Messergebnis lautet somit:

$$k = 34,46 \frac{N}{m}; \Delta k = 0,60 \frac{N}{m}$$

Für das 95%-Konfidenzintervall wird der t-Wert der Ausdehnung  $t = 2,2622$  eingesetzt.

$$u(k(m, s)) = \sqrt{\left(\frac{g}{s} \cdot \Delta m\right)^2 + \left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s\right)^2}$$

Der erste Term lautet unverändert:

$$\left(\frac{g}{s} \cdot \Delta m\right)^2 = \left(\frac{9,81m}{s^2 \cdot 0,1432m} \cdot 0,0005kg\right)^2 = \left(0,0342 \frac{N}{m}\right)^2$$

Die Masse, die Konstante  $g$ , der Standardfehler des Mittelwerts der Ausdehnung und der t-Wert der Ausdehnung  $t = 2,2622$  werden in den zweiten Term eingesetzt.

$$\left(-\frac{m \cdot g}{s^2} \cdot t_{\alpha/2} \cdot \sigma_s\right)^2 = \left((-) \frac{0,503kg \cdot 9,81m}{s^2 \cdot (0,1432m)^2} \cdot 2,2622 \cdot 0,00247m\right)^2$$

Trennen von Werten und Einheiten, Newton  $\frac{kg \cdot m}{s^2}$  einsetzen und  $m$  kürzen:

$$\left((-) \frac{0,503 \cdot 9,81 \cdot 2,2622 \cdot 0,00247}{0,02051} \frac{kg \cdot m^2}{s^2 \cdot m^2}\right)^2 = \left((-)1,344 \frac{N}{m}\right)^2$$

Die Formel lautet somit:

$$u(k(m, s)) = \sqrt{\left(0,0342 \frac{N}{m}\right)^2 + \left((-)1,344 \frac{N}{m}\right)^2}$$

Ausrechnen:

$$u(k(m, s)) = 1,344 \frac{N}{m} \approx 1,35 \frac{N}{m}$$

Die Intervallgrenzen lauten somit:

$$k_{95\%} = 33,11 \frac{N}{m} \leq 34,46 \frac{N}{m} \leq 35,81 \frac{N}{m}$$

## 5 Übung Hooke'sches Gesetz

- 
- 
- 
- 
- 

- 1.
- 2.
- 3.

5.1

5.2

### 5.1 Dateien einlesen

Einlesen strukturierter Datensätze

Methodenbaustein

```
ordnerpfad = '01-daten/hooke'

pfadliste = glob.glob(pathname = '*', root_dir = ordnerpfad, recursive = False)
print(pfadliste)
print(f"Anzahl Dateien: {len(pfadliste)}")
```

```
for pfad in pfadliste:
    zwischenspeicher = pd.read_csv(filepath_or_buffer = ordnerpfad + '/' + pfad, nrows = 3)
    print(pfad, "\n", zwischenspeicher, "\n", sep = '')
```

## 5.2 Aufgabe Dateien einlesen

- 
- 

a)

b)

Methodenbaustein Einlesen struk-

turierter Datensätze

c)

### Tipp 8: Schrittweises Vorgehen

Das Einlesen der Dateien wird voraussichtlich der aufwändigste und fehleranfälligste Arbeitsschritt sein. Entwickeln Sie Ihre Lösung Schritt für Schritt. Beginnen Sie mit der Variante a). Wenn Sie die Dateien eingelesen haben, können Sie sich durch die Weiterentwicklung zur Variante b) mit dem Modul glob vertraut machen. Darauf aufbauend können Sie mit der Variante c) die Automatisierung für beliebig viele Dateien umsetzen.

### Tipp 9: Musterlösung Dateien einlesen

Der erste Versuch, die Dateien einzulesen, scheitert mit einer Fehlermeldung. Die Anweisungen werden deshalb in die Struktur zur Ausnahmebehandlung eingebettet und die verursachende Datei abgefangen. (Sollten mehrere Dateien Fehler erzeugen, müssten die Dateien in einer Liste gespeichert und später - falls möglich - mit einer Schleife weiter



behandelt werden.)

```
hooke = pd.DataFrame(columns = ['Zeit', 'Abstand', 'Gewicht', 'Team']) # ein leerer DataFr

for pfad in pfadliste:

    try:
        zwischenspeicher = pd.read_csv(filepath_or_buffer = ordnerpfad + '/' + pfad, sep = '\t'

        # Dateiname als Spalte einfügen
        zwischenspeicher['Team'] = pfad[5:-4] # 'team_' und die Dateiendung abschneiden

        hooke = pd.concat([hooke, zwischenspeicher], ignore_index = True)

    except Exception as error:
        print(pfad, error, sep = "\n")
        pfad_problem_datei = pfad

print(hooke.info(), "\n")
print("Erfolgreich einglesen:\n", hooke['Team'].unique(), sep = '')
```

team\_ma.txt

Error tokenizing data. C error: Expected 3 fields in line 135, saw 4

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 359 entries, 0 to 358
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Zeit        355 non-null   object
1   Abstand     355 non-null   object
2   Gewicht     355 non-null   float64
3   Team        359 non-null   object
dtypes: float64(1), object(3)
memory usage: 11.3+ KB
None
```

Erfolgreich einglesen:

```
['kreativkoepfe' 'fabi' 'die_ahnungslosen']
```

Der Fehlermeldung zufolge besteht Zeile 135 aus 4 statt aus 3 Spalten. Die fehlerverursachende Datei wird deshalb zeilenweise durchlaufen und jede Zeile ausgegeben, die mehr

als 3 Einträge hat. Zur Kontrolle werden auch die ersten 5 Zeilen ausgegeben.

```
# einen leeren DataFrame mit 3 Spalten erstellen
df = pd.DataFrame(data = [], columns = ['Zeit', 'Abstand', 'Gewicht'])

dateiobjekt_problem_datei = open(file = ordnerpfad + '/' + pfad_problem_datei, mode = 'r')

index = 0
for zeile in dateiobjekt_problem_datei:
    try:
        zwischenspeicher = zeile.split(sep = "\t")
        if len(zwischenspeicher) > 3:
            print("Index =", index, ":", zwischenspeicher)
        elif index <= 5:
            print(zeile)
        index += 1
    except Exception as error:
        print(error)

dateiobjekt_problem_datei.close()
```

```
10:09:38    109.26 cm    0
```

```
10:09:41    109.26 cm    0
```

```
10:09:44    109.28 cm    0
```

```
Index = 134 : ['10:29:27', '105.49 cm', '300', '\n']
```

Jede zweite Zeile ist leer. In Zeile 134 wird ein Zeilenumbruch ‘\n’ eingelesen. Die Datei wird deshalb mit einer angepassten Schleife erneut durchlaufen. Aus der betreffenden Zeile wird der zusätzliche Zeilenumbruch ‘\n’ entfernt. Leere Zeilen werden übersprungen. Die korrekten Zeilen werden an den DataFrame hooke angefügt.

**Der Code muss ggf. noch angepasst werden, weil vermutlich so leere zeilen angefügt werden**

```

dateiobjekt_problem_datei = open(file = ordnerpfad + '/' + pfad_problem_datei, mode = 'r')

for zeile in dateiobjekt_problem_datei:
    try:
        zwischenspeicher = zeile.split(sep = "\t")
        if len(zwischenspeicher) > 3:
            zwischenspeicher = zwischenspeicher[:3]
        elif len(zwischenspeicher) < 3: # leere Zeilen überspringen
            continue
        # Dateinamen anfügen
        zwischenspeicher.append(pfad[5:-4])

        hooke.loc[len(hooke)] = pd.Series(zwischenspeicher).values

    except Exception as error:
        print(error)
        print(pd.Series(zwischenspeicher).values)

dateiobjekt_problem_datei.close()

print("Erfolgreich einglesen:\n", hooke['Team'].unique(), "\n", sep = '')
print(hooke.info())

```

Erfolgreich einglesen:

```
['kreativkoepfe' 'fabi' 'die_ahnungslosen' 'ma']
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 537 entries, 0 to 536
```

```
Data columns (total 4 columns):
```

| # | Column  | Non-Null Count | Dtype  |
|---|---------|----------------|--------|
| 0 | Zeit    | 533 non-null   | object |
| 1 | Abstand | 533 non-null   | object |
| 2 | Gewicht | 533 non-null   | object |
| 3 | Team    | 537 non-null   | object |

```
dtypes: object(4)
```

```
memory usage: 21.0+ KB
```

```
None
```

Anschließend werden zum einen die Zeilen mit Nullwerten betrachtet ...

```
print(hooke.loc[hooke.apply(pd.isna).any(axis = 1), :])
```

|     | Zeit | Abstand | Gewicht | Team             |
|-----|------|---------|---------|------------------|
| 3   | NaN  | NaN     | NaN     | kreativkoepfe    |
| 23  | NaN  | NaN     | NaN     | kreativkoepfe    |
| 28  | NaN  | NaN     | NaN     | kreativkoepfe    |
| 322 | NaN  | NaN     | NaN     | die_ahnungslosen |

... und entfernt.

```
hooke.drop(np.where(hooke.apply(pd.isna).any(axis = 1))[0], inplace = True)
```

Zum anderen werden die Datentypen kontrolliert.

- Die Zeit kann als string stehen bleiben, da sie für die Auswertung nicht benötigt wird.
- Der gemessene Abstand ist mit ' cm' notiert - diese Zeichenkette wird entfernt. Anschließend sollte die Spalte als numerisch erkannt werden.
- Das Gewicht sollte numerische Werte enthalten, wird aber als Datentyp object eingelesen und muss weiter untersucht werden.
- Der Spalte Team könnte der [Pandas Datentyp category](#) zugewiesen werden, notwendig ist es aber nicht.

String ' cm' entfernen.

```
hooke.replace(' cm', '', regex = True, inplace = True)
```

Ob alle Elemente einer Zelle numerisch sind, kann mit der Pandas-Methode `pd.Series.str.isnumeric()` überprüft werden. Ein Blick auf die Daten zeigt die Ursache.

```
print(hooke['Gewicht'].str.isnumeric().sum())
```

```
print(hooke['Gewicht'].head())
```

```
print(hooke['Gewicht'].tail())
```

|   |     |
|---|-----|
| 1 |     |
| 0 | 0.0 |
| 1 | 0.0 |
| 2 | 0.0 |
| 4 | 0.0 |
| 5 | 0.0 |

```
Name: Gewicht, dtype: object
```

```
532      850\n
```

```
533      850\n
```

```
534      850\n
```

```
535      850\n
```

```
536      850\n
```

```
Name: Gewicht, dtype: object
```

Die Zeilenumbrüche werden ebenfalls entfernt.

```
hooke.replace('\n', '', regex = True, inplace = True)
print(hooke['Gewicht'].str.isnumeric().sum())
print(hooke['Gewicht'].tail())
```

```
178
```

```
532      850
```

```
533      850
```

```
534      850
```

```
535      850
```

```
536      850
```

```
Name: Gewicht, dtype: object
```

```
print(hooke.info(), "\n")
```

```
# explizite Zuweisung
```

```
hooke['Abstand'] = hooke['Abstand'].astype('float')
```

```
hooke['Gewicht'] = hooke['Gewicht'].astype('float')
```

```
print(hooke.info())
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 533 entries, 0 to 536
```

```
Data columns (total 4 columns):
```

| # | Column  | Non-Null Count | Dtype  |
|---|---------|----------------|--------|
| 0 | Zeit    | 533 non-null   | object |
| 1 | Abstand | 533 non-null   | object |
| 2 | Gewicht | 533 non-null   | object |
| 3 | Team    | 533 non-null   | object |

```
dtypes: object(4)
```

```
memory usage: 20.8+ KB
```

None

```
<class 'pandas.core.frame.DataFrame'>  
Index: 533 entries, 0 to 536  
Data columns (total 4 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   Zeit        533 non-null    object  
1   Abstand     533 non-null    float64  
2   Gewicht     533 non-null    float64  
3   Team        533 non-null    object  
dtypes: float64(2), object(2)  
memory usage: 20.8+ KB  
None
```

```
hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].describe()
```

### **5.3 Musterlösung Daten aufbereiten**

- 
-

- 

- 1.

#### **5.3.1 Auf Ausreißer prüfen**

### **5.4 Ausreißer bestimmen**

### **5.5 Gemeinsame Ausgabe der Messreihen**





## 5.6 Code

```
# z-Werte größer gleich abs(3) finden
z_values_ge3_sum = hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: scipy.stats.zscore(x))
print("Anzahl der studentisierten z-Werte mit Betrag >= 3:", z_values_ge3_sum)

# z-Werte größer gleich abs(2.5) finden
z_values_ge25_sum = hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: scipy.stats.zscore(x))
print("Anzahl der studentisierten z-Werte mit Betrag >= 2.5:", z_values_ge25_sum)

# Die Zeilen mit z-Werten größer abs(2.5) ausgeben
bool_index = hooke.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: scipy.stats.zscore(x) >= 2.5)
z_values_ge_25 = hooke.iloc[bool_index, :]
print(z_values_ge_25, "\n")

# Kombinationen aus Gewicht & Team bestimmen

teams = z_values_ge_25['Team'].unique()

## teams durchlaufen und jeweils die Gewichte speichern
team_gewichte = [] # leere liste
for i in range(len(teams)):
    print(teams[i])
    print(z_values_ge_25.loc[z_values_ge_25['Team'] == teams[i], 'Gewicht'], "\n")
    team_gewichte.append(z_values_ge_25.loc[z_values_ge_25['Team'] == teams[i], 'Gewicht'].values)

print(team_gewichte, "\n")

# Messreihen auswählen
messreihen = pd.DataFrame()
for i in range(len(teams)):
    for j in range(len(team_gewichte[i])):

        print(teams[i], team_gewichte[i][j])

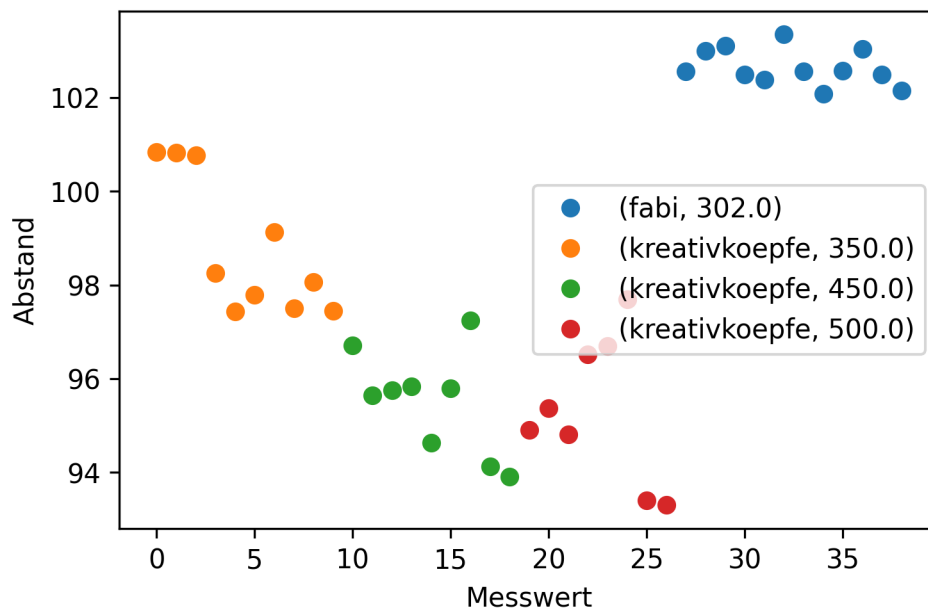
        messreihen = pd.concat([messreihen, hooke.loc[ (hooke['Team'] == teams[i]) & (hooke['Gewicht'] == team_gewichte[i][j]) ]])

# studentisierte z-Werte der Messreihen bilden
messreihen_z_scores = messreihen.groupby(by = ['Team', 'Gewicht'])['Abstand'].apply(lambda x: scipy.stats.zscore(x))

# gemeinsame Ausgabe der Daten
messreihen.insert(loc = 2, column = 'z-Werte Abstand', value = messreihen_z_scores.values)
```

```
print(messreihen)
```

```
messreihen.reset_index(drop = True).groupby(by = ['Team', 'Gewicht'])['Abstand'].plot(marker='o')  
  
plt.xlabel('Messwert')  
plt.ylabel('Abstand')  
plt.legend()  
  
plt.show()
```



### 5.6.1 Umwandlung der Rechengrößen

### 💡 Tipp 10: Musterlösung

#### Abstandsmessung auf Meter und auf den Nullpunkt normieren

Abstandsmessung auf den Nullpunkt normieren. Die Spalte Abstand wird in Abstandsänderung umbenannt.

```
nullpunkte = hooke.loc[hooke['Gewicht'] == 0, :].groupby(by = 'Team')['Abstand'].mean()

print("Nullpunkte", nullpunkte, sep = '\n')

teams = nullpunkte.index

for i in range(len(teams)):

    hooke.loc[hooke['Team'] == teams[i] , 'Abstand'] = hooke.loc[hooke['Team'] == teams[i] ,

hooke.rename(columns = {'Abstand': 'Abstandsänderung'}, inplace = True)
hooke['Abstandsänderung'] = hooke['Abstandsänderung'].div(100)
```

```
Nullpunkte
Team
die_ahnungslosen    109.759000
fabi                 110.366000
kreativkoepfe       109.676667
ma                  109.258000
Name: Abstand, dtype: float64
```

#### Gewicht in wirkende Kraft umrechnen

Gewicht in  $g$  in die wirkende Kraft in  $N$  umrechnen. Die Spalte wird in den Datensatz eingefügt.

```
hooke['Kraft'] = hooke['Gewicht'].div(1000).mul(9.81)
```

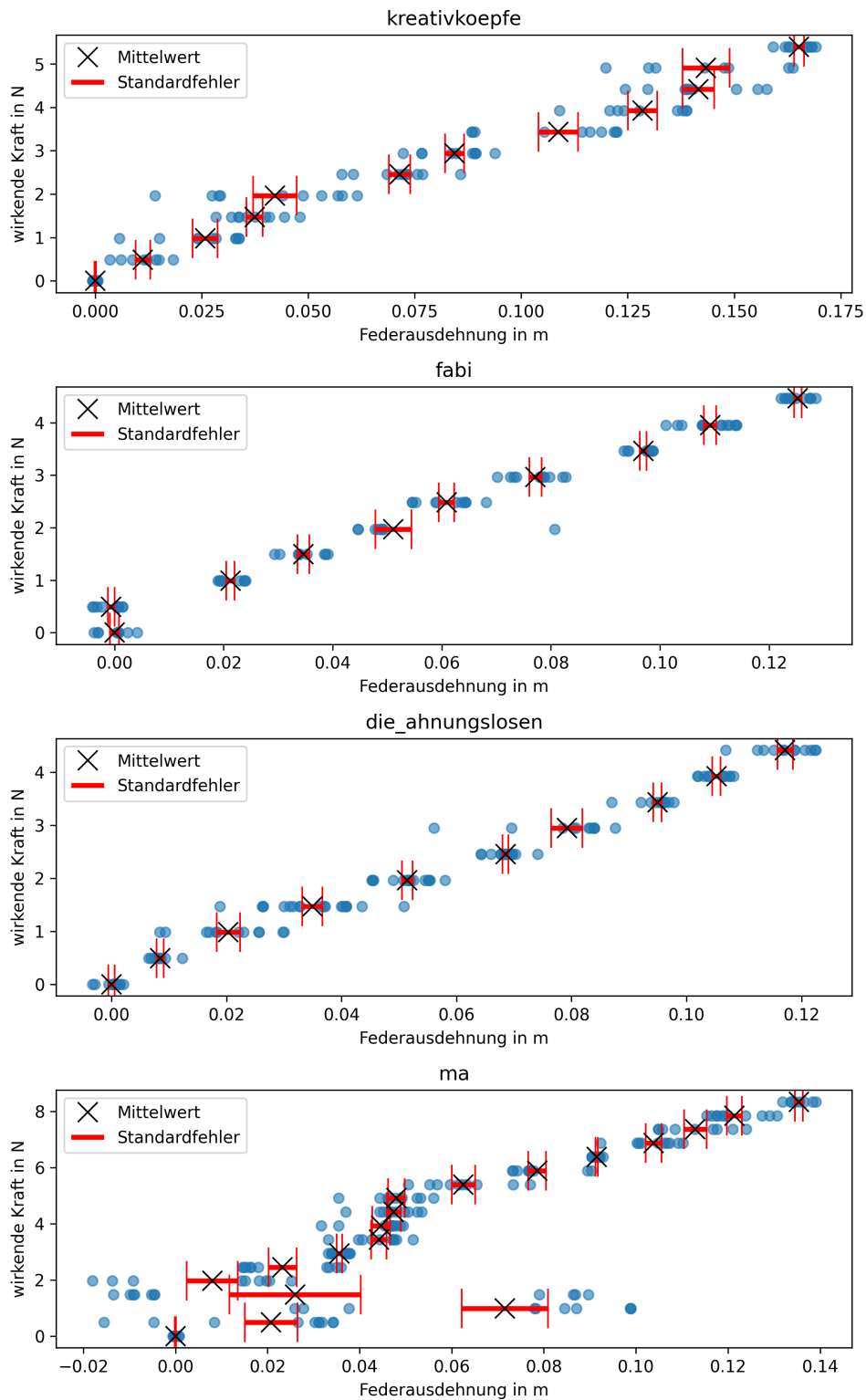
```
hooke.groupby(by = ['Team', 'Kraft'])['Abstandsänderung'].describe()
```



### **5.6.2 Grafische Darstellung**



## 5.7 Grafische Darstellung





## 5.8 Mittelwerte und Standardfehler berechnen

## 5.9 Code

```
# Mittelwerte der Teams nach Kraft
distance_means_by_team_and_force = hooke.groupby(by = [hooke['Team'], hooke['Kraft']])['Absta
distance_means_by_team_and_force.name = 'Federausdehnung'

print("Mittelwerte der Abstandsänderung nach Team und Kraft:", distance_means_by_team_and_force)

print() # leere Zeile

# Standardfehler der Teams nach Kraft
distance_stderrors_by_team_and_force = hooke.groupby(by = [hooke['Team'], hooke['Kraft']])['A
distance_stderrors_by_team_and_force.name = 'Standardfehler'

print("Standardfehler der Mittelwerte nach Team und Kraft", distance_stderrors_by_team_and_force)

# grafische Darstellung
anzahl_teams = hooke['Team'].unique().size
```

```

plt.figure(figsize = (7.5, 12))
for i in range(anzahl_teams):

    plt.subplot(4, 1, i + 1) # plt.subplot zählt ab 1

    # Punktdiagramm
    plotting_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
    plt.scatter(x = plotting_data['Abstandsänderung'], y = plotting_data['Kraft'], alpha = 0.6)

    plt.title(label = hooke['Team'].unique()[i])
    plt.xlabel("Federausdehnung in m")
    plt.ylabel("wirkende Kraft in N")

    # # Fehlerbalken
    distance_means_by_force = plotting_data.groupby(by = plotting_data['Kraft'])['Abstandsänderung'].mean()
    distance_stderrors_by_force = plotting_data.groupby(by = plotting_data['Kraft'])['Abstandsänderung'].std()

    errorbar_container = plt.errorbar(x = distance_means_by_force, y = distance_means_by_force,
                                     linestyle = 'none', marker = 'x', color = 'black', markersize = 12, elinewidth = 3, ecolor = 'black')

    # siehe: https://matplotlib.org/stable/api/container_api.html#matplotlib.container.ErrorbarContainer
    plt.legend([errorbar_container.lines[0], errorbar_container.lines[2][0]],
               ['Mittelwert', 'Standardfehler'],
               loc = 'upper left')

plt.tight_layout()
plt.show()

```

## 5.10 Auswertung

- 
- 
- 
-

```
hooke.drop(index = hooke.loc[hooke['Team'] == 'ma', :].index, inplace = True)
hooke.drop(index = hooke.loc[(hooke['Team'] == 'fabi') & (hooke['Gewicht'] == 50), :].index,
```

#### Tipp 11: Vorgehen bei vielen Datensätzen

Bei einer großen Anzahl an Datensätzen kann auch die grafische Kontrolle an Grenzen stoßen. In diesem Fall empfiehlt es sich, die visuellen und kennzahlenbasierten Methoden zusammen zu nutzen, um Muster zu identifizieren und für eine große Zahl von Messungen zu überprüfen. Beispielsweise könnten nach einer visuellen Inspektion von Messreihen mit Extremwerten bzw. Ausreißern alle Messreihen daraufhin überprüft werden, ob mit zunehmenden Gewicht stets auch die mittlere Federausdehnung größer als für leichtere Gewichte ist. Abweichende Messreihen könnten dann grafisch kontrolliert werden.

## 5.11 Musterlösung Federkonstanten bestimmen

2.

3.

## 5.12 $\alpha = 0.10$

### 5.13 $\alpha = 0.05$

### 5.14 Code

```
anzahl_teams = hooke['Team'].unique().size
alpha = 0.10

for i in range(anzahl_teams):

    reg_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
    x = reg_data['Abstandsänderung']
    y = reg_data['Kraft']
    n = len(x)
```

```

print("\n", hooke['Team'].unique()[i], " Konfidenzniveau: ", 1 - alpha, sep = '')

slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
      f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
      f"Standardfehler des Anstiegs: {slope_stderr:.4f}", sep = '')

print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}    {s

alpha = 0.05

for i in range(anzahl_teams):

    reg_data = hooke.loc[hooke['Team'] == hooke['Team'].unique()[i], :]
    x = reg_data['Abstandsänderung']
    y = reg_data['Kraft']
    n = len(x)

    print("\n", hooke['Team'].unique()[i], " Konfidenzniveau: ", 1 - alpha, sep = '')

    slope, intercept, rvalue, pvalue, slope_stderr = scipy.stats.linregress(x, y)
    print(f"y = {intercept:.4f} + {slope:.4f} * x\n",
          f"r = {rvalue:.4f} R2 = {rvalue ** 2:.4f} p = {pvalue:.4f}\n",
          f"Standardfehler des Anstiegs: {slope_stderr:.4f}", sep = '')

    print(f"{slope - scipy.stats.t.ppf(q = 1 - alpha / 2, df = n - 2) * slope_stderr:.3f}    {s

```

## 5.15 Auswertung

 **Warning 6: Ergebnisse**

Abhängig vom gewählten Vorgehen sind andere Ergebnisse möglich, beispielsweise durch das Entfernen von als Ausreißern eingestuften Einzelwerten oder einer anderen Behandlung der Messreihe vom Team fabi für das angehängte Gewicht 50 Gramm.

## 6 Kalibrierung

### 6.1 Beispiel Pt100

2023

- 
- 
- 

—  
—  
—

2023

hier

| Tabelle A.1 – Beziehung zwischen Temperatur und Widerstand unter 0 °C; $R_0 = 100,00\ \Omega$ |                                                                      |       |       |       |       |       |       |       |       |       |                                 |
|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|---------------------------------|
| $t_{\text{rel}}/^\circ\text{C}$                                                               | Widerstand in Ohm bei der Temperatur $t_{\text{rel}}/^\circ\text{C}$ |       |       |       |       |       |       |       |       |       | $t_{\text{rel}}/^\circ\text{C}$ |
|                                                                                               | 0                                                                    | -1    | -2    | -3    | -4    | -5    | -6    | -7    | -8    | -9    |                                 |
| -200                                                                                          | 18,52                                                                |       |       |       |       |       |       |       |       |       | -200                            |
| -190                                                                                          | 22,83                                                                | 22,40 | 21,97 | 21,54 | 21,11 | 20,68 | 20,25 | 19,82 | 19,38 | 18,95 | -190                            |
| -180                                                                                          | 27,10                                                                | 26,67 | 26,24 | 25,82 | 25,39 | 24,97 | 24,54 | 24,11 | 23,68 | 23,25 | -180                            |
| -170                                                                                          | 31,34                                                                | 30,91 | 30,49 | 30,07 | 29,64 | 29,22 | 28,80 | 28,37 | 27,95 | 27,52 | -170                            |

Abbildung 6.1: Ausschnitt der Tabelle A.1

2023

2023

### 6.1.1 Aufgabe Pt100-Tabelle einlesen

hier



```
'skript/01-daten/pt100-table-below-zero.csv'
'skript/01-daten/pt100-table-below-zero.csv'
```

### 💡 Tipp 12: Musterlösung Pt100-Tabelle einlesen

```
dateipfad_belowzero = '01-daten/pt100-table-below-zero.csv'
dateipfad_abovezero = '01-daten/pt100-table-above-zero.csv'
```

Dateien einlesen und betrachten. Zunächst die Datei mit Temperaturen unter 0 Grad Celsius.

```
# belowzero
belowzero = pd.read_csv(filepath_or_buffer = dateipfad_belowzero, sep = ',')

print(belowzero.head(), "\n")
print(belowzero.tail())
```

|   | Temperatur in °C | Unnamed: 1 | Unnamed: 2 | Unnamed: 3 | Unnamed: 4 | Unnamed: 5 |
|---|------------------|------------|------------|------------|------------|------------|
| 0 | NaN              | 0          | -1         | -2         | -3         | -4         |
| 1 | -200.0           | 18,52      | NaN        | NaN        | NaN        | NaN        |
| 2 | -190.0           | 22,83      | 22,4       | 21,97      | 21,54      | 21,11      |
| 3 | -180.0           | 27,1       | 26,67      | 26,24      | 25,82      | 25,39      |
| 4 | -170.0           | 31,34      | 30,91      | 30,49      | 30,07      | 29,64      |

|   | Unnamed: 6 | Unnamed: 7 | Unnamed: 8 | Unnamed: 9 | Unnamed: 10 |
|---|------------|------------|------------|------------|-------------|
| 0 | -5         | -6         | -7         | -8         | -9          |
| 1 | NaN        | NaN        | NaN        | NaN        | NaN         |
| 2 | 20,68      | 20,25      | 19,82      | 19,38      | 18,95       |
| 3 | 24,97      | 24,54      | 24,11      | 23,68      | 23,25       |
| 4 | 29,22      | 28,8       | 28,37      | 27,95      | 27,52       |

|    | Temperatur in °C | Unnamed: 1 | Unnamed: 2 | Unnamed: 3 | Unnamed: 4 | Unnamed: 5 |
|----|------------------|------------|------------|------------|------------|------------|
| 17 | -40.0            | 84,27      | 83,87      | 83,48      | 83,08      | 82,69      |
| 18 | -30.0            | 88,22      | 87,83      | 87,43      | 87,04      | 86,64      |
| 19 | -20.0            | 92,16      | 91,77      | 91,37      | 90,98      | 90,59      |
| 20 | -10.0            | 96,09      | 95,69      | 95,3       | 94,91      | 94,52      |
| 21 | 0.0              | 100        | 99,61      | 99,22      | 98,83      | 98,44      |

|    | Unnamed: 6 | Unnamed: 7 | Unnamed: 8 | Unnamed: 9 | Unnamed: 10 |
|----|------------|------------|------------|------------|-------------|
| 17 | 82,29      | 81,89      | 81,5       | 81,1       | 80,7        |

|    |       |       |       |       |       |
|----|-------|-------|-------|-------|-------|
| 18 | 86,25 | 85,85 | 85,46 | 85,06 | 84,67 |
| 19 | 90,19 | 89,8  | 89,4  | 89,01 | 88,62 |
| 20 | 94,12 | 93,73 | 93,34 | 92,95 | 92,55 |
| 21 | 98,04 | 97,65 | 97,26 | 96,87 | 96,48 |

Es gibt 201 Werte (von -200 bis 0). Die Daten beginnen in der Zeile mit dem Index 1 (wenn die erste Zeile als header eingelesen wird). Diese enthält nur einen Eintrag in der Spalte '0'. Die folgenden Zeilen sind von rechts nach links einzulesen. Das Dezimaltrennzeichen ist das ,.

Das Einlesen der Daten von rechts nach links kann auf viele Arten bewerkstelligt werden. Eine einzeilige Lösung beginnt damit, der Methode `pd.iloc[::-1]` eine negative Schrittweite zu übergeben.

```
df = pd.DataFrame(np.array([[3, 2, 1], [6, 5, 4]]))
print(df, "\n")
print(df.iloc[::-1])
```

```

    0  1  2
0  3  2  1
1  6  5  4
```

```

    0  1  2
1  6  5  4
0  3  2  1
```

Das führt dazu, dass die *Zeilen* des DataFrame in umgekehrter Reihenfolge ausgegeben werden. Um die *Spalten* in umgekehrter Reihenfolge auszugeben, wird der DataFrame mit der Methode `pd.T` zwei mal transponiert.

```
print(df.T.iloc[::-1].T)
```

```

    2  1  0
0  1  2  3
1  4  5  6
```

Mit der NumPy-Methode `np.flatten()` kann ein Array in eine eindimensionale Struktur reduziert werden. Dafür wird mit der Methode `pd.to_numpy()` der DataFrame als NumPy-Array ausgegeben.

```
print(df.T.iloc[::-1].T.to_numpy())
print() # leere Zeile
print(df.T.iloc[::-1].T.to_numpy().flatten())
```

```
[[1 2 3]
 [4 5 6]]
```

```
[1 2 3 4 5 6]
```

Versuchen wir es mit dem Kopf der Pt100-Daten.

```
belowzero = pd.read_csv(filepath_or_buffer = dateipfad_belowzero, sep = ',', decimal = ',',
print(belowzero.head())
print() # leere Zeile
print(belowzero.head().T.iloc[:, :-1].T.to_numpy().flatten())
```

|      | 0     | -1    | -2    | -3    | -4    | -5    | -6    | -7    | -8    | -9    |
|------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| -200 | 18.52 | NaN   | NaN   | NaN   | NaN   | NaN   | NaN   | NaN   | NaN   | NaN   |
| -190 | 22.83 | 22.40 | 21.97 | 21.54 | 21.11 | 20.68 | 20.25 | 19.82 | 19.38 | 18.95 |
| -180 | 27.10 | 26.67 | 26.24 | 25.82 | 25.39 | 24.97 | 24.54 | 24.11 | 23.68 | 23.25 |
| -170 | 31.34 | 30.91 | 30.49 | 30.07 | 29.64 | 29.22 | 28.80 | 28.37 | 27.95 | 27.52 |
| -160 | 35.54 | 35.12 | 34.70 | 34.28 | 33.86 | 33.44 | 33.02 | 32.60 | 32.18 | 31.76 |

```
[ nan  nan  nan  nan  nan  nan  nan  nan  nan 18.52 18.95 19.38
 19.82 20.25 20.68 21.11 21.54 21.97 22.4  22.83 23.25 23.68 24.11 24.54
 24.97 25.39 25.82 26.24 26.67 27.1  27.52 27.95 28.37 28.8  29.22 29.64
 30.07 30.49 30.91 31.34 31.76 32.18 32.6  33.02 33.44 33.86 34.28 34.7
 35.12 35.54]
```

Um die fehlenden Werte zu überspringen, wandeln wir das bisherige Ergebnis wieder in eine `pd.Series()` und verwenden die Methode `pd.Series.dropna()`.

```
pd.Series(belowzero.head().T.iloc[:, :-1].T.to_numpy().flatten()).dropna()
```

|    |       |
|----|-------|
| 9  | 18.52 |
| 10 | 18.95 |
| 11 | 19.38 |
| 12 | 19.82 |
| 13 | 20.25 |
| 14 | 20.68 |
| 15 | 21.11 |
| 16 | 21.54 |
| 17 | 21.97 |
| 18 | 22.40 |
| 19 | 22.83 |

```
20    23.25
21    23.68
22    24.11
23    24.54
24    24.97
25    25.39
26    25.82
27    26.24
28    26.67
29    27.10
30    27.52
31    27.95
32    28.37
33    28.80
34    29.22
35    29.64
36    30.07
37    30.49
38    30.91
39    31.34
40    31.76
41    32.18
42    32.60
43    33.02
44    33.44
45    33.86
46    34.28
47    34.70
48    35.12
49    35.54
dtype: float64
```

Und jetzt für die gesamte Datei:

```
belowzero_ohm = pd.Series(belowzero.T.iloc[:, :-1].T.to_numpy().flatten()).dropna()
print(belowzero_ohm.head(), belowzero_ohm.tail(), sep = '\n')
```

```
9      18.52
10     18.95
11     19.38
12     19.82
13     20.25
```

```
dtype: float64
205    98.44
206    98.83
207    99.22
208    99.61
209   100.00
dtype: float64
```

Die Temperatur können wir einfach durch ein range-Objekt erzeugen:

```
belowzero_temperatur = pd.Series(range(-200, 1)) # range stop ist exklusiv
print(belowzero_temperatur.head(), belowzero_temperatur.tail(), sep = '\n')
```

```
0   -200
1   -199
2   -198
3   -197
4   -196
dtype: int64
196    -4
197    -3
198    -2
199    -1
200     0
dtype: int64
```

Für die Temperaturen oberhalb von 0 Grad kann das Vorgehen vereinfacht werden, da die Datei gleich aufgebaut ist und die Werte aufsteigend sortiert sind.

```
abovezero = pd.read_csv(filepath_or_buffer = dateipfad_abovezero, sep = ',', decimal = ',')

abovezero_ohm = pd.Series(abovezero.to_numpy().flatten()).dropna()
print(abovezero_ohm.head(), abovezero_ohm.tail(), sep = '\n')
```

```
0    100.00
1    100.39
2    100.78
3    101.17
4    101.56
dtype: float64
846   389.31
847   389.60
```

```
848    389.90
849    390.19
850    390.48
dtype: float64
```

Beide Datensätze enthalten einen Eintrag für die Temperatur 0 Grad, sodass dieser in der zweiten Datei entfernt werden kann.

```
abovezero_ohm = abovezero_ohm[1:]
print(abovezero_ohm.head(), abovezero_ohm.tail(), sep = '\n')
```

```
1    100.39
2    100.78
3    101.17
4    101.56
5    101.95
dtype: float64
846    389.31
847    389.60
848    389.90
849    390.19
850    390.48
dtype: float64
```

Die Temperatur wird wieder durch ein range-Objekt erzeugt.

```
abovezero_temperatur = pd.Series(range(1, 851)) # range stop ist exklusiv
```

Die aus beiden Datensätzen erzeugten Series können zusammengefasst werden:

```
pt100 = pd.DataFrame({'Temperatur': pd.concat([belowzero_temperatur, abovezero_temperatur])})
print(pt100.head(), pt100.tail(), sep = '\n')

print(pt100.info())
```

|   | Temperatur | Ohm   |
|---|------------|-------|
| 0 | -200       | 18.52 |
| 1 | -199       | 18.95 |
| 2 | -198       | 19.38 |
| 3 | -197       | 19.82 |
| 4 | -196       | 20.25 |

|  | Temperatur | Ohm |
|--|------------|-----|
|--|------------|-----|

```

1046      846  389.31
1047      847  389.60
1048      848  389.90
1049      849  390.19
1050      850  390.48
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1051 entries, 0 to 1050
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Temperatur  1051 non-null   int64
1   Ohm         1051 non-null   float64
dtypes: float64(1), int64(1)
memory usage: 16.6 KB
None

```

### 6.1.2 Nicht-Linearität darstellen

12

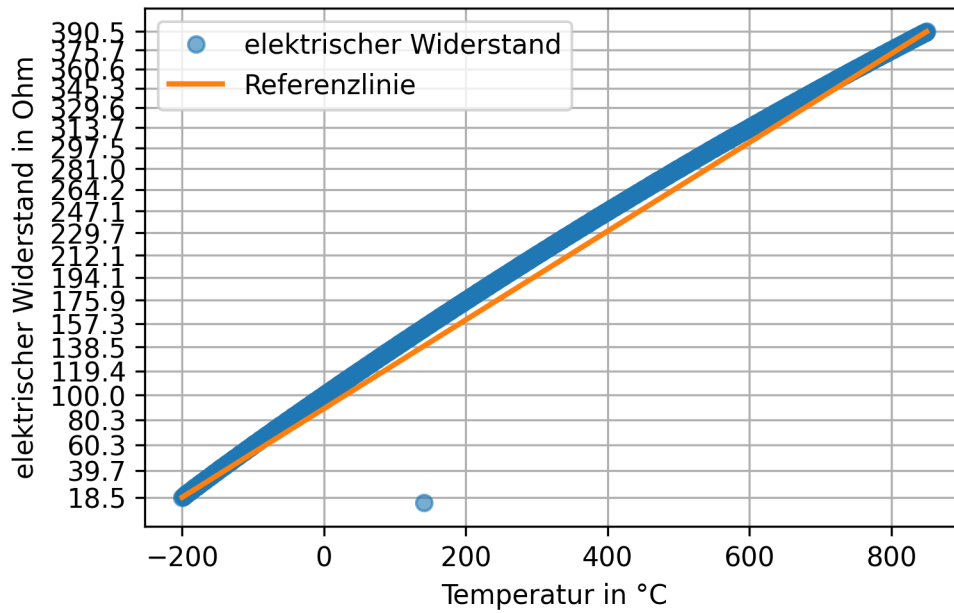
```

plt.plot(pt100['Temperatur'], pt100['Ohm'], marker = 'o', linestyle = '', label = 'elektrisch')
plt.plot([pt100['Temperatur'].iloc[0], pt100['Temperatur'].iloc[-1]], [pt100['Ohm'].iloc[0],
pt100['Ohm'].iloc[-1]], label = 'Temperaturbereich', color = 'red', linestyle = 'solid');

plt.xlabel('Temperatur in °C')
plt.ylabel('elektrischer Widerstand in Ohm')
plt.grid()
plt.legend()

plt.show()

```



```
print(( position_fehlwert := pt100['Ohm'].diff().idxmin() )) # := ist der sog. Walross-Operator
print(pt100.iloc[list(range(position_fehlwert - 2, position_fehlwert + 3))])
```

2023



## Hinweis 19: Interpolation

Die einfachste Form der Interpolation ist die lineare Interpolation.

```
interpolate_me = pt100.loc[position_fehlwert - 1 : position_fehlwert + 1, 'Ohm'].copy()
interpolate_me[position_fehlwert] = np.nan
print(interpolate_me)

print("linear interpolierter Wert:", interpolate_me.interpolate(), sep = "\n")
```

```
340    153.58
341         NaN
342    154.33
Name: Ohm, dtype: float64
linear interpolierter Wert:
340    153.580
341    153.955
342    154.330
Name: Ohm, dtype: float64
```

Durch Runden wird das korrekte Ergebnis erreicht (darauf verlassen sollte man sich aber nicht).

```
print("linear interpolierter Wert:", interpolate_me.interpolate().round(2), sep = "\n")
korrekter_wert = interpolate_me.interpolate().round(2)[position_fehlwert]
```

```
linear interpolierter Wert:
340    153.58
341    153.96
342    154.33
Name: Ohm, dtype: float64
```

Eine andere Option ist die nicht-lineare Interpolation (die nicht Inhalt dieses Bausteins ist).

### **i** Hinweis 20: Nicht-lineare Interpolation

Die nicht-lineare Interpolation benötigt mehr Datenpunkte. Wir versuchen die kubische Interpolation:

```
interpolate_me_again = pt100.loc[position_fehlwert - 3 : position_fehlwert + 4, 'Ohm'].copy()
interpolate_me_again[position_fehlwert] = np.nan
print("polynomial interpolierter Wert:", interpolate_me_again.interpolate(method = 'polynomial', order = 3))
```

polynomial interpolierter Wert:

```
338    152.830000
339    153.210000
340    153.580000
341    153.952213
342    154.330000
343    154.710000
344    155.080000
345    155.460000
```

Name: Ohm, dtype: float64

Das Ergebnis ist nicht unbedingt besser.

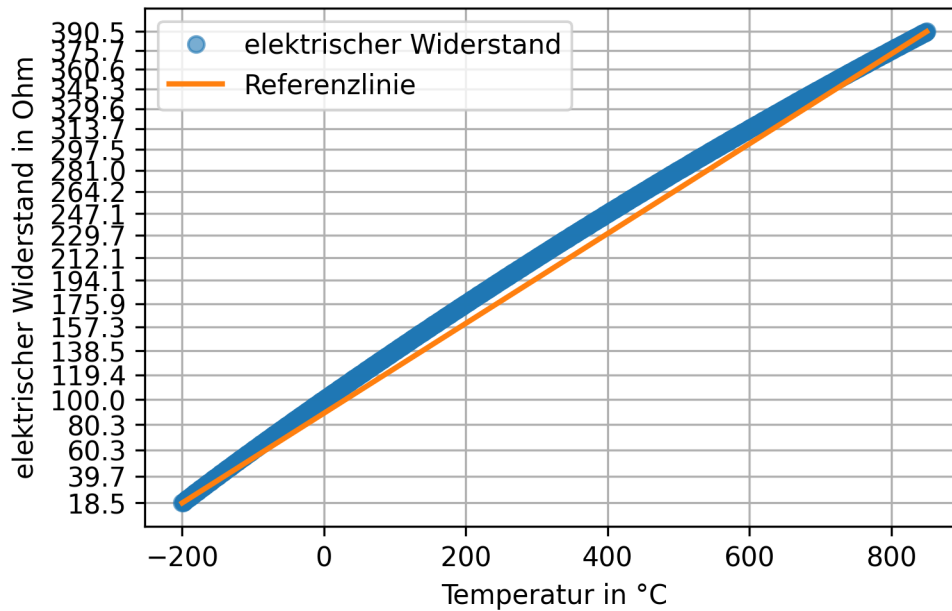
```
pt100.loc[position_fehlwert, 'Ohm'] = korrekter_wert
print(pt100.loc[position_fehlwert - 1 : position_fehlwert + 1, 'Ohm'])
```

```
plt.plot(pt100['Temperatur'], pt100['Ohm'], marker = 'o', linestyle = '', label = 'elektrischer Widerstand')
plt.plot([pt100['Temperatur'].iloc[0], pt100['Temperatur'].iloc[-1]], [pt100['Ohm'].iloc[0], pt100['Ohm'].iloc[-1]],
         label = 'Temperaturbereich', color = 'red', linestyle = 'solid', marker = 'none')
plt.yticks(pt100['Ohm'][:50]);

plt.xlabel('Temperatur in °C')
plt.ylabel('elektrischer Widerstand in Ohm')
```

```
plt.grid()
plt.legend()

plt.show()
```



### 6.1.3 Nicht-Linearität quantifizieren

#### ! Wichtig 9: Festpunktmethode

Die Endpunkte der realen Kennlinie werden durch eine Gerade verbunden. Der Linearitätsfehler ist das Maximum der Abweichung der Kennlinie zu dieser Geraden. (Grote und Feldhusen 2011, W2 (S. 1661))

```
lm = poly.polyfit(
    x = [pt100['Temperatur'].iloc[0], pt100['Temperatur'].iloc[-1]],
    y = [pt100['Ohm'].iloc[0], pt100['Ohm'].iloc[-1]],
    deg = 1)
```

```

print(lm.round(2)) # intercept + slope

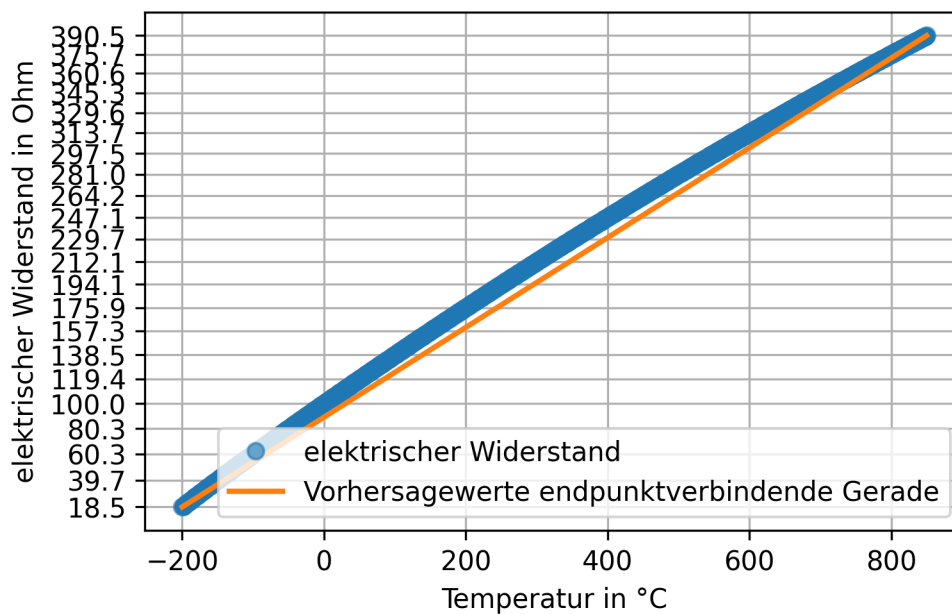
vorhersagewerte = poly.polyval(x = pt100['Temperatur'], c = lm)

plt.plot(pt100['Temperatur'], pt100['Ohm'], marker = 'o', linestyle = '', label = 'elektrischer Widerstand')
plt.plot(pt100['Temperatur'], vorhersagewerte, linewidth = 2, label = 'Vorhersagewerte endpunktverbindende Gerade')
plt.yticks(pt100['Ohm'][:50]);

plt.xlabel('Temperatur in °C')
plt.ylabel('elektrischer Widerstand in Ohm')
plt.grid()
plt.legend()

plt.show()

```



```
linearitätsfehler = (pt100['Ohm'] - vorhersagewerte).abs().max()
print(f"Linearitätsfehler nach Festpunktmethode: {linearitätsfehler:.2f} Ohm.")
```

### ! Wichtig 10: Toleranzbandmethode

Bei der Toleranzbandmethode wird eine Gerade so durch die Messpunkte gelegt, dass die Summe der quadrierten Abweichungen der Messpunkte zu dieser Geraden (Methode der kleinsten Quadrate) oder die größte einzelne Abweichung ([Tschebyscheff-Approximation](#)) minimiert wird. Die Größe des Linearitätsfehlers ist die maximale senkrechte Entfernung der Kennlinie zu dieser Ausgleichsgeraden. (Grote und Feldhusen [2011](#), W3 (S. 1662))

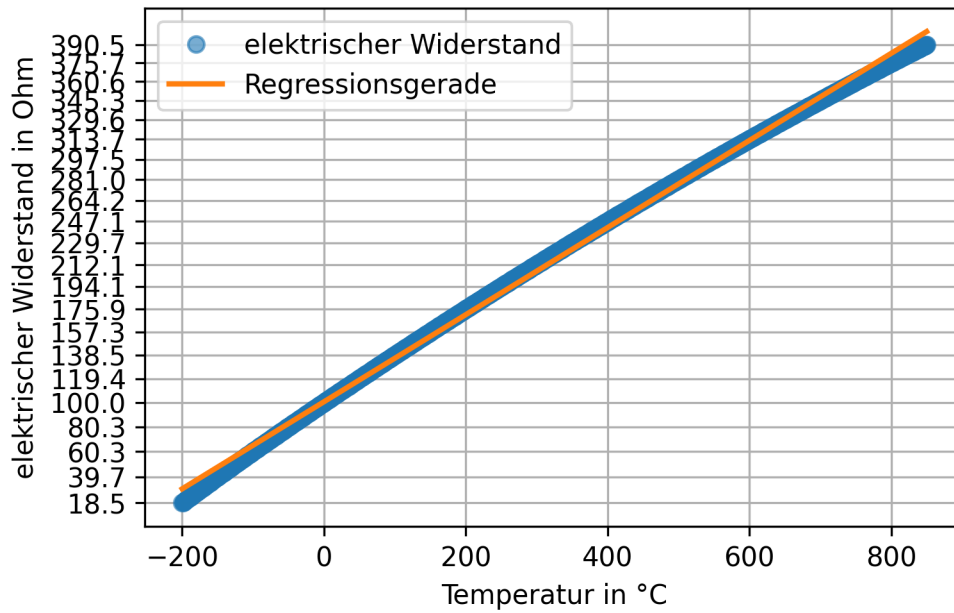
```
lm = poly.polyfit(
    x = pt100['Temperatur'],
    y = pt100['Ohm'],
    deg = 1)
print(lm.round(2)) # intercept + slope

vorhersagewerte = poly.polyval(x = pt100['Temperatur'], c = lm)

plt.plot(pt100['Temperatur'], pt100['Ohm'], marker = 'o', linestyle = '', label = 'elektrisch')
plt.plot(pt100['Temperatur'], vorhersagewerte, linewidth = 2, label = 'Regressionsgerade')
plt.yticks(pt100['Ohm'][:50]);

plt.xlabel('Temperatur in °C')
plt.ylabel('elektrischer Widerstand in Ohm')
plt.grid()
plt.legend()

plt.show()
```



```
linearitätsfehler = (pt100['Ohm'] - vorhersagewerte).abs().max()
print(f"Linearitätsfehler nach Toleranzbandmethode: {linearitätsfehler:.2f} Ohm.")
```

## 6.2 Kalibrieren und Justieren

### ! Wichtig 11: Kalibrierung nach DIN 1319

“Ermitteln des Zusammenhangs zwischen Meßwert [...] oder Erwartungswert [...] der Ausgangsgröße [...] und dem zugehörigen wahren [...] oder richtigen Wert [...] der als Ein-

gangsgröße vorliegenden Meßgröße für eine betrachtete Meßeinrichtung [...]” (Deutsches Institut für Normung [1995](#), S. 22)

[1995](#)

! Wichtig 12: Justierung nach DIN 1319

“Einstellen oder Abgleichen eines Meßgeräts [...], um systematische Meßabweichungen [...] soweit zu beseitigen, wie es für die vorgesehene Anwendung erforderlich ist.” (Deutsches Institut für Normung [1995](#), S. 22)

## 6.3 Kalibriermethoden

- 1.
- 2.
- 3.

### **6.3.1 Korrekturstabelle**

### **6.3.2 Ermittlung von Kalibrierfaktoren**

### **6.3.3 Ermittlung einer (empirischen) Kalibrierfunktion**

## **6.4 Resterampe**

Hysteresis

1.

2.

•





# Quellen

10.31030/2713411

10.31030/7204542

10.31030/3405985

<https://www2.physnet.uni-hamburg.de/TUHH/Versuchsanleitung/Fehlerrechnung.pdf>

10.1007/978-3-658-44779-3

10.1007/978-3-642-17306-6 <https://doi.org/10.1007/978-3-642-17306-6>

<https://www.tu-braunschweig.de/index.php?eID=dumpFile&t=f&f=45992&token=b041478add5f7a3e136b69905b72bfa0e192d82b>

[https://www.hs-aalen.de/uploads/mediapool/media/file/13430/F\\_Fehlerrechnung.pdf](https://www.hs-aalen.de/uploads/mediapool/media/file/13430/F_Fehlerrechnung.pdf)